

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

О. А. Молчанов

**ВСТУП ДО ФУНКЦІОНАЛЬНОГО
ПРОГРАМУВАННЯ
КОНСПЕКТ ЛЕКЦІЙ**

Навчальний посібник

*Рекомендовано Методичною радою КПІ ім. Ігоря Сікорського
як навчальний посібник для здобувачів першого (бакалаврського) рівня вищої
освіти за освітньо-професійною програмою
«Системне програмування та спеціалізовані комп'ютерні системи»
спеціальності F7 Комп'ютерна інженерія*

Електронне мережеве навчальне видання

Київ
КПІ ім. Ігоря Сікорського
2025

УДК 004.41

Автори: *Молчанов Олексій Андрійович, доктор філософії, асистент*

Рецензент *Заболотня Т. М., канд. техн. наук., доц., доцент кафедри програмного забезпечення комп'ютерних систем ФПМ*

Відповідальний редактор *Тарасенко-Клятченко О. В., канд. техн. наук, доц.*

*Гриф надано Методичною радою КПП ім. Ігоря Сікорського
(протокол № 6 від 30.03.2025 р.)
за поданням вченої ради факультету прикладної математики
(протокол № 7 від 27.02.2025 р.)*

Молчанов О. А.

Вступ до функціонального програмування. Конспект лекцій [Електронний ресурс]: навч. посіб. для здобувачів ступеня бакалавра за освітньо-професійною програмою «Системне програмування та спеціалізовані комп'ютерні системи» спеціальності F7 «Комп'ютерна інженерія» / О. А. Молчанов ; КПП ім. Ігоря Сікорського. – Електронні текстові дані (1 файл). – Київ : КПП ім. Ігоря Сікорського, 2025. – 209 с.

Навчальний посібник розроблено для ознайомлення студентів із лекційним курсом дисципліни «Вступ до функціонального програмування». Навчальний посібник містить теоретичний матеріал та практичні завдання для вивчення основ функціонального програмування мовою програмування Common Lisp. Навчальний посібник призначений для студентів очної форми навчання за спеціальністю F7 «Комп'ютерна інженерія» факультету прикладної математики КПП ім. Ігоря Сікорського.

УДК 004.41

Реєстр. № НП ХХ/ХХ-ХХХ. Обсяг 6,2 авт. арк.
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
проспект Берестейський, 37, м. Київ, 03056
<https://kpi.ua>

Свідоцтво про внесення до Державного реєстру видавців, виготовлювачів
і розповсюджувачів видавничої продукції ДК № 5354 від 25.05.2017 р.

© О. А. Молчанов 2025
© КПП ім. Ігоря Сікорського, 2025

ЗМІСТ

ВСТУП.....	11
РОЗДІЛ 1 ПОНЯТТЯ ФУНКЦІОНАЛЬНОГО ПРОГРАМУВАННЯ.....	13
Тема 1.1 Класифікація мов програмування.....	13
Тема 1.2 Основні принципи функціонального програмування.....	14
Тема 1.3 Мова програмування LISP.....	15
1.3.1 Особливості мови Lisp.....	17
1.3.2 Чому саме Lisp?.....	18
1.3.3 Налаштування середовища розробки для Common Lisp.....	22
1.3.3.1 Простий спосіб налаштування.....	22
1.3.3.2 Emacs.....	23
1.3.3.3 IntelliJ IDEA.....	23
1.3.3.4 Visual Studio Code.....	23
1.3.3.5 Після встановлення.....	24
1.3.4 REPL.....	24
1.3.4.1 Деякі особливості роботи з REPL SBCL.....	25
1.3.4.2 Завантаження файлів з кодом.....	27
1.3.4.3 Завантаження файлів зі Slime в Emacs.....	27
1.3.5 Виклики функцій в Lisp та інтерпретація s-виразів.....	28
1.3.5.1 Відміна обчислень.....	30
1.3.5.2 Обчислення форм з EVAL.....	31
1.3.6 Коментарі.....	32
1.3.7 Завдання на СРС.....	33
РОЗДІЛ 2 ОСНОВИ МОВИ COMMON LISP.....	34
Тема 2.1 Базові типи даних та функції.....	34
2.1.1 Символи.....	34

2.1.1.1	Константи.....	35
2.1.2	Числа.....	37
2.1.3	Списки.....	38
2.1.4	Символьні вирази.....	39
2.1.5	Основні функції роботи зі списками.....	40
2.1.5.1	Отримання елементів списків.....	40
2.1.5.2	Примітивні предикати.....	42
2.1.5.3	Додавання елементів до голови списку.....	43
2.1.6	Предикати для порівняння двох об'єктів.....	44
2.1.6.1	Порівняння з EQL.....	44
2.1.6.2	Порівняння з EQUAL.....	45
2.1.6.3	Порівняння з EQUALP.....	45
2.1.7	Операції з числами.....	46
2.1.7.1	Арифметичні і інші операції.....	46
2.1.7.2	Числові предикати.....	47
2.1.8	Логічні операції.....	48
2.1.9	Інші функції для роботи зі списками.....	49
2.1.10	Встановлення значення символу.....	50
2.1.11	Завдання на СРС.....	52
Тема 2.2	Символи та їх властивості.....	53
2.2.1	Структура символу.....	53
2.2.1.1	Встановлення значень полів символьних структур.....	55
2.2.1.2	Встановлення значень полів символьних структур з SETF.....	55
2.2.1.3	Символи зі значенням і функцією.....	56
2.2.1.4	Редагування списку атрибутів символу.....	57
2.2.2	Ключові символи.....	58
Тема 2.3	Списки та їх будова.....	59

2.3.1 Внутрішнє представлення списків.....	59
2.3.1.1 Спискові комірки та точкові пари.....	59
2.3.1.2 Конструктивний підхід.....	60
2.3.1.3 Деструктивний підхід.....	62
2.3.2 Види списків.....	64
2.3.2.1 Список атрибутів (plist).....	64
2.3.3 Деструктивні операції.....	65
2.3.3.1 Універсальний макрос SETF.....	66
2.3.3.2 Інші оператори, що оперують "місцями".....	68
2.3.3.3 INCF та DECF.....	68
2.3.3.4 PUSH та POP.....	69
2.3.3.5 ROTATEF.....	70
2.3.4 Функції для роботи зі списками.....	70
2.3.4.1 Функції пошуку елемента MEMBER, MEMBER-IF та MEMBER-IF-NOT.....	70
2.3.4.2 Функції конструктивного складання списків MAPCAR та MAPLIST.....	72
2.3.4.3 Функції обходу списків MAPC та MAPL.....	73
2.3.4.4 Функції деструктивного складання списків MAPCAN та MAPCON.....	74
Тема 2.4 Визначення функції.....	74
2.4.1 Іменовані функції.....	75
2.4.2 Спеціальні параметри функцій.....	76
2.4.2.1 Опційні параметри.....	77
2.4.2.2 Ключові параметри.....	78
2.4.2.3 "Решта" параметрів.....	79
2.4.2.4 Допоміжні параметри.....	80

2.4.2.5 Комбінування різних видів спеціальних параметрів.....	81
2.4.3 Локальні функції.....	82
2.4.4 "Багато значень" та робота з ними.....	83
2.4.5 Завдання на СРС.....	86
Тема 2.5 Зв'язування змінних.....	89
2.5.1 Спеціальні оператори LET та LET*.....	89
2.5.2 Види зв'язування змінних та область дії зв'язування.....	91
2.5.2.1 Види змінних.....	91
2.5.2.2 Лексичне зв'язування.....	92
2.5.2.3 Динамічне зв'язування.....	94
2.5.2.4 Особливості зв'язування змінних для замикань.....	95
Тема 2.6 Форми керування потоком виконання.....	95
2.6.1 Послідовне виконання.....	95
2.6.2 Розгалуження обчислень.....	97
2.6.2.1 IF, WHEN та UNLESS.....	97
2.6.2.2 COND.....	99
2.6.2.3 CASE.....	100
2.6.3 Цикли.....	101
2.6.3.1 DO та DO*.....	101
2.6.3.2 DOLIST.....	104
2.6.3.3 DOTIMES.....	105
2.6.3.4 Макрос LOOP.....	106
2.6.4 Інші форми керування потоком виконання.....	106
2.6.4.1 BLOCK, RETURN, RETURN-FROM.....	106
РОЗДІЛ 3 РЕКУРСІЯ І ЇЇ ВИКОРИСТАННЯ У ФУНКЦІОНАЛЬНОМУ	
ПРОГРАМУВАННІ.....	108
Тема 3.1 Проста рекурсія.....	108

3.1.1 Хвостова та не хвостова рекурсія.....	113
Тема 3.2 Інші види рекурсії.....	116
3.2.1 Паралельна рекурсія.....	116
3.2.2 Взаємна рекурсія.....	118
3.2.3 Рекурсія вищого порядку.....	120
3.2.4 Реалізація циклів за допомогою рекурсії.....	121
3.2.4.1 Дворівневі цикли.....	122
3.2.4.2 Трирівневі цикли.....	125
3.2.5 Завдання на СРС.....	126
РОЗДІЛ 4 ВИДИ ФУНКЦІЙ ТА ЇХ ЗАСТОСУВАННЯ.....	130
Тема 4.1 Функції вищого порядку.....	130
4.1.1 Отримання функціонального об'єкта.....	131
4.1.2 Виклик функціонального об'єкта.....	132
4.1.3 Стандартні функції вищого порядку.....	134
Тема 4.2 Лямбда-вирази.....	135
Тема 4.3 Замикання.....	136
РОЗДІЛ 5 ІНШІ МОЖЛИВОСТІ МОВИ COMMON LISP.....	139
Тема 5.1 Послідовності, масиви.....	139
5.1.1 Функції отримання елементів з послідовностей.....	141
5.1.2 Функція копіювання послідовності.....	142
5.1.3 Функції об'єднання послідовностей.....	142
5.1.4 Функції аналізу послідовностей.....	143
5.1.4.1 LENGTH.....	143
5.1.4.2 COUNT, COUNT-IF та COUNT-IF-NOT.....	144
5.1.4.3 Предикати EVERY, NOTEVERY та NOTANY.....	145
5.1.5 Функції пошуку елементів у послідовностях.....	146
5.1.5.1 FIND, FIND-IF та FIND-IF-NOT.....	146

5.1.5.2	POSTION, POSITION-IF, POSITION-IF-NOT.....	147
5.1.5.3	SOME.....	148
5.1.5.4	SEARCH.....	149
5.1.5.5	MISMATCH.....	149
5.1.6	Конструктивні функції зміни послідовностей.....	149
5.1.6.1	REVERSE.....	149
5.1.6.2	REMOVE, REMOVE-IF та REMOVE-IF-NOT.....	149
5.1.6.3	SUBSTITUTE, SUBSTITUTE-IF та SUBSTITUTE-IF-NOT.....	150
5.1.6.4	REMOVE-DUPPLICATES.....	150
5.1.7	Деструктивні функції зміни послідовностей.....	150
5.1.8	Функція перетворення послідовності.....	151
5.1.9	Масиви.....	153
5.1.9.1	Створення масиву.....	153
5.1.9.2	Доступ до елементів з масиву.....	155
5.1.9.3	Встановлення значень у масив.....	156
5.1.10	Завдання на СРС.....	156
Тема 5.2	Рядки, рядкові символи.....	157
5.2.1	Робота з рядковими символами.....	158
5.2.1.1	Створення і перетворення рядкових символів.....	158
5.2.1.2	Порівняння рядкових символів.....	159
5.2.1.3	Інші предикати для рядкових символів.....	160
5.2.2	Робота з рядками.....	161
5.2.2.1	Порівняння рядків.....	162
5.2.2.2	Інші функції для роботи з рядками.....	164
5.2.3	Завдання на СРС.....	167
Тема 5.3	Асоціативні списки.....	167
Тема 5.4	Геш-таблиці.....	170

5.4.1 Створення геш-таблиці.....	170
5.4.2 Додавання значень та пошук за ключем.....	171
5.4.3 Обхід геш-таблиці.....	173
5.4.4 Видалення значень та очищення геш-таблиці.....	174
5.4.5 Завдання на СРС.....	174
Тема 5.5 Структури.....	175
5.5.1 Визначення структур.....	175
5.5.2 Зміна значень слотів структур.....	177
5.5.3 Спадкування структур.....	178
Тема 5.6 Файли.....	179
5.6.1 Принтер.....	181
5.6.1.1 Форматований вивід.....	181
5.6.2 Макрос WITH-OPEN-FILE.....	183
Тема 5.7 Макроси.....	185
5.7.1 Визначення макросів.....	185
5.7.2 Прості макроси.....	185
5.7.3 Створення складних макросів.....	186
РОЗДІЛ 6 ОСОБЛИВОСТІ ФУНКЦІОНАЛЬНОГО ПРОГРАМУВАННЯ.....	188
Тема 6.1 Комбінування функціонального та об'єктного програмування.....	188
Тема 6.2 Лінійні обчислення.....	189
Тема 6.3 Комбінатори.....	191
Тема 6.4 Простий вебсервер.....	192
РЕКОМЕНДОВАНА ЛІТЕРАТУРА.....	193
Базова література.....	193
Додаткова література.....	193
Інформаційні ресурси.....	193
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	194

ДОДАТОК 1 ФОРМАТУВАННЯ КОДУ МОВОЮ COMMON LISP.....	200
1. Іменування символів.....	200
1.1 Верхній та нижній регістри.....	200
1.2 Багатослівні символи.....	200
1.3 Небажані імена.....	201
2. Пробіли у списках.....	201
3. Відступи вкладених форм.....	201
4. Розміщення гілок конструкцій керування.....	203
5. Визначення функцій.....	204
ДОДАТОК 2 РОБОТА З EMACS/PORTACLE і SLIME.....	206
cua-mode.....	206
paredit-mode.....	207
Робота з текстом в буфері.....	207
REPL.....	208
Slime.....	209

ВСТУП

Навчальний посібник призначений для студентів, які вивчають дисципліну «Вступ до функціонального програмування». «Вступ до функціонального програмування» є вибірковою дисципліною з циклу загальної підготовки навчального плану спеціальності F7 «Комп'ютерна інженерія» при навчанні за освітньо-професійною програмою «Системне програмування та спеціалізовані комп'ютерні системи».

Метою дисципліни є оволодіння студентами основ функціонального програмування на прикладі функціональної мови програмування Common Lisp, а також формування у студентів здатностей:

- аналізувати поставлену задачу;
- виконувати вибір парадигми програмування;
- визначати найкращий спосіб вирішення поставленої задачі у функціональному стилі.

Предметом вивчення дисципліни є базові принципи функціонального програмування; окремі визначення і поняття, що притаманні саме функціональним мовам програмування; структури та алгоритми, що застосовуються при програмуванні у функціональному стилі мовою Common Lisp.

Вивчення кредитного модуля «Вступ до функціонального програмування» дає змогу сформувати у студентів компетенції, необхідні для розв'язання задач професійної діяльності, пов'язаної із здатністю аналізувати поставлену задачу і визначати найкращі методи програмування для її вирішення, зокрема випадки, коли доцільним є застосування функціонального програмування, а також застосування на практиці принципів функціонального програмування при розробці програм.

Крім того, вивчення дисципліни «Вступ до функціонального програмування» підсилює формування у студентів наступних компетентностей:

- здатність виявляти, ставити та вирішувати проблеми;
- здатність вести дослідницьку діяльність, включаючи аналіз проблем, постановку цілей і завдань, вибір способу й методів дослідження, а також оцінку його якості;
- здатність гнучко адаптуватися до різних професійних ситуацій, проявляти творчий підхід, ініціативу;
- здатність критично оцінювати й переосмислювати накопичений досвід (власний і чужий), рефлексувати професійну й соціальну діяльність, тощо.

Лекційний курс з кредитного модулю «Вступ до функціонального програмування» розраховано на 30 академічних години аудиторних занять, вивчається на факультеті прикладної математики у 7 семестрі.

РОЗДІЛ 1

ПОНЯТТЯ ФУНКЦІОНАЛЬНОГО ПРОГРАМУВАННЯ

В дисципліні "Вступ до функціонального програмування" розглядаються базові принципи функціонального програмування на прикладі мови Lisp.

Тема 1.1 Класифікація мов програмування

На сьогодні у світі існує багато різних мов програмування. Вони створювались в різний час, для вирішення різних задач. Якісь з цих мов використовуються і є дуже популярними (напр. JavaScript, Python, Java тощо), в той час як інші мови або не використовуються взагалі, або ж існують лише за рахунок підтримки старих (legacy) систем (напр. SmallTalk, ALGOL, COBOL [3] тощо).

Для класифікації мов програмування використовуються *парадигми програмування* [2]. **Парадигма програмування** — це система ідей і понять, які визначають стиль написання комп'ютерних програм [4]. Існує багато парадигм програмування і переважна кількість сучасних мов програмування реалізують у собі багато парадигм, що дає змогу використовувати такі мови для вирішення різних типів задач.

З багатьох парадигм програмування часто виокремлюють дві як загальні (або основні) — це *імперативне* та *декларативне* програмування [1, 2].

При імперативному підході до написання програми програміст вказує (commands) як необхідно отримати результат. Тобто, програміст визначає послідовність інструкцій, послідовне виконання яких призводить до отримання бажаного результату. Наприклад, "для того, щоб потрапити у парк, необхідно пройти сто метрів на північ, повернути ліворуч і пройти ще 50 метрів". Типовим

прикладом мови, що реалізує парадигму імперативного програмування є будь-яка C-подібна мова — C, C++, Java, Python тощо [5].

При декларативному підході до написання програми програміст визначає (declares) *що* необхідно отримати в якості результату, а не те, як саме це зробити. Наприклад, "для того, щоб потрапити у парк, необхідно пройти за адресою вул. Паркова, 1". Прикладом мов, що реалізують парадигму декларативного програмування є функціональні мови (Lisp, Clojure, Haskell, Erlang та ін.), мови логічного програмування (Prolog) тощо [5].

Тема 1.2 Основні принципи функціонального програмування

Парадигма *функціонального (або функційного) програмування* є похідною від декларативного програмування [6]. В мовах, що реалізують цю парадигму, програми створюються за рахунок композиції викликів функцій, і часто мають вигляд "дерев" викликів, в яких "піддерева" формують проміжні результати.

Принципи функційного програмування такі:

1. Чистота функцій (purity of functions). Цей принцип передбачає, що на утворення результату функції впливають лише її вхідні аргументи і не впливає глобальний (або зовнішній) стан. Тобто, так як в математиці: функція є відображенням однієї множини значень в іншу: $f: X \rightarrow Y$; $f(x) \rightarrow y$, де $x \in X$, $y \in Y$. Для одного і того ж значення x функція завжди повертатиме те саме значення y , оскільки лише x впливає на результат функції f . Ця особливість також передбачає, що функція не впливає ні на що, окрім того, що всередині неї. Тобто, стан "зовнішніх" об'єктів не змінюється зсередини функції. Для цього також використовується *незмінюваність даних*.
2. Незмінюваність даних. Зазвичай програмування в функційному стилі передбачає використання незмінюваних типів даних (immutable). Всі об'єкти, які необхідно створити або оновити (змінити їхній стан) — конструюються

"з нуля" або отримуються внаслідок копіювання зі зміною вже наявних об'єктів. Це дає змогу [7]:

- a) забезпечити передбачуваність при отриманні даних — оскільки старі дані не змінюються;
 - b) забезпечити безпечну роботу в багатьох потоках, оскільки незмінювані дані можна легко передавати між потоками не переймаючись синхронізацією для зміни певного "загального" стану;
 - c) тестування і налагодження спрощується, оскільки область коду, в якій могли відбутись якісь зміни (через які потенційно виникає помилкова ситуація) — обмежена.
3. Композиція і рекурсія. Як і було зазначено, програми у функційному стилі зазвичай мають вигляд "дерева" виразів, утвореного композицією викликів функцій і констант. Досить часто можуть застосовуватись різні види рекурсії — коли функція викликає саму себе (прямо чи опосередковано). Типовим є використання функцій вищого порядку, оскільки функції розглядаються як об'єкти.

Тема 1.3 Мова програмування LISP

Мова програмування Lisp виникла ще в 1958 році і є однією з перших високорівневих мов програмування, а також першою функціональною мовою [8]. При створенні мови Lisp було створено перший збірник сміття (garbage collector). Слово "lisp" походить від "LISt Processing", тобто "обробка списків": однією з основних структур даних мови є списки, обробка яких є типовою в програмах, написаних на Lisp.

Першим застосуванням мови були дослідження у сфері штучного інтелекту. Проте не того штучного інтелекту, про який часто йде мова останніми роками (машинне навчання (machine learning), великі мовні моделі (LLM)), а штучного

інтелекту для забезпечення автономної роботи роботів. Сьогодні використання різних діалектів мови Lisp є радше нішевим. Вони використовуються в різних компаніях для створення застосунків для обробки великих обсягів даних, багатопоточних застосунків, хмарних технологіях, для програмування медичного обладнання, в трансляторах та статичних аналізаторах і оптимізаторах коду тощо.

Найвідомішими реалізаціями мови Lisp сьогодні є такі мови як Clojure, Common Lisp, Emacs Lisp, Clojure Script, Scheme, Racket та інші [9] (рис. 1.1). Станом на 2025 рік різні діалекти мови Lisp використовуються в таких компаніях, як Grammarly, Keepit, Apptopia, Freshcode та ін.

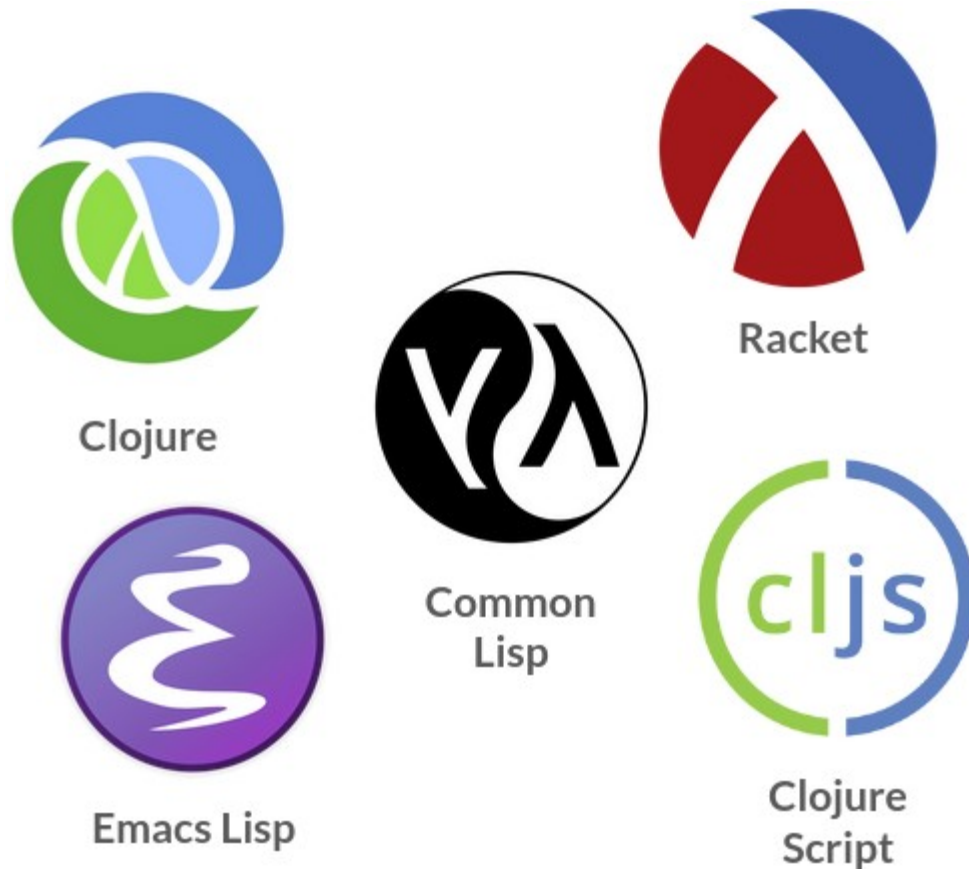


Рисунок 1.1 — Діалекти мови Lisp

1.3.1 Особливості мови Lisp

Особливостями мови Lisp, які вирізняють її з-поміж інших мов є такі:

1. Це функційна мова програмування, в якій реалізовані принципи функційного програмування, розглянуті вище. В залежності від діалекту мови, реалізація цих принципів може мати ті чи інші відмінності (наприклад, в Common Lisp є можливість писати програми в імперативному стилі, а не в функціональному, хоча це і є нетиповим).
2. Простий синтаксис: програми мовою Lisp — це набір функцій, що визначаються як дерева виразів, (s-виразів), що є списками, тобто послідовністю символів, констант та інших виразів, розміщених в круглих дужках. Завдяки простому синтаксису і префіксній формі запису, програми і дані в Lisp записуються однаково, що дає змогу формувати і виконувати програми динамічно.
3. Безтиповість або динамічна типізація: при написанні програм мовою Lisp не потрібно явно визначати типи змінних, формальних параметрів функцій чи тип результату функції. Тип даних значень визначається під час виконання програми динамічно і, якщо буде виявлено несумісність типів даних при виконанні тих чи інших функцій, — буде видано помилку.
4. Інтерактивна розробка, що забезпечується REPL (Read-Eval-Print Loop), завдяки чому розробка і відлагодження програм є досить простим і швидким процесом.
5. Потужна система макросів дає змогу розробникам писати власні синтаксичні конструкції, створювати власні предметно-орієнтовні мови (domain-specific languages, DSL). Макроси в Lisp можуть виконуватись як на етапі компіляції, так і на етапі зчитування коду, що значено розширює можливості розробника при написанні коду. (Хоча варто зауважити, що з великими можливостями,

застосовувати макроси варто зважено, бо код може перетворитись в щось складно зрозуміле.)

6. Метaproграмування — тобто можливість інтерпретування програм як даних. Це реалізовано у макросах, а також досягається завдяки рефлексії (reflection).

В різних діалектах мови певні особливості можуть відрізнятись, проте в більшості Lisp-подібних мов все зазначене реалізовано повністю або частково.

1.3.2 Чому саме Lisp?

Закономірним питання при вивченні будь-якої мови програмування є питання "чому саме ця мова?". Як і було зазначено, мова Lisp має нішеве комерційне та академічне застосування. Вона не є такою популярною, як, наприклад, мови Python, JavaScript (і діалекти), Java, C++ чи C#. Чому Lisp не має (або ж втратив) свою популярність зараз? На це є кілька відповідей [13]:

- Стиль програмування
 - *Теза:* програми мовою Lisp сильно відрізняються від програм, що написані C-подібними мовами — через відмінність основної парадигми, яка в них закладена (декларативна проти імперативної). А через свою популярність і вживаність, яка з різних причин була сформована роками, саме C-подібні мови є першими (і, можливо, єдиними), з якими знайомляться майбутні програмісти в школах та університетах.
 - *Пояснення:* якщо раніше в мовах програмування реалізовували якусь одну чи кілька парадигм, то сьогодні часто в мовах програмування є підтримка багатьох парадигм, і, як наслідок, область їхнього застосування розширюється. Проте часто парадигми, що не є (не були) основними в мові, не вивчаються або вивчаються обмежено. Мова Lisp є чудовим прикладом парадигми функційного програмування, яке не є основним в

більшості С-подібних мов. Тому вивчення мови Lisp дасть змогу краще зрозуміти як і коли застосовувати функційний підхід, в т. ч. при програмуванні іншими мовами.

- **Поріг входження:**
 - *Теза:* програмування в декларативному стилі є дещо складнішим за імперативний стиль. Через це збільшується поріг входження у розробку Lisp-подібними мовами.
 - *Пояснення:* це твердження виходить із попереднього. Написання програм є складнішим, бо воно є значно менш звичним за написання програм в імперативному стилі. Такі концепції, як функції вищого порядку, рекурсія, метапрограмування та інші часто не є потрібними програмістам для вирішення типових задач. Проте знання цих концепцій і вміння їх застосовувати може значно підвищити рівень програміста, розширити його можливості, що прямо впливає на якість коду, який він пише. Більш того, вищий ступінь абстракції (що досягається за рахунок простого синтаксису) дає змогу більш творчо підходити до вирішення різних задач.
- **Синтаксис:**
 - *Теза:* багатьох новачків відлякує велика кількість дужок в програмах, написаних мовою Lisp. Особливо проблемним видається пошук помилки, коли одна відкриваюча або закриваюча дужка пропущена, і компілятор не каже, де саме.
 - *Пояснення:* так, в програмах мовою Lisp багато дужок. В деяких дужки лише одного виду (круглі в Common Lisp), в деяких — різного (Clojure), але через те, що всі вирази мови записуються в префіксній формі — дужок таки багато. Проте, насправді, синтаксис мови значно простіший за той самий С. Одноманітність запису різних операцій певною мірою навіть спрощує написання коду. Тому "питання дужок" — це радше

питання звички. Після програмування іншими мовами це виглядає дивно, проте не менш дивно, ніж у випадку переходу з C/C++ на Python, коли дужки для позначення блоків відсутні (для визначення вкладеності блоків використовуються відступи), а крапки з комою взагалі можуть не використовуватись. Питання ж з парністю дужок вирішується сучасними редакторами, які не допускають втрати парності дужок при написанні програми.

- Кількість бібліотек та величина спільноти:
 - *Теза:* Lisp має невелику спільноту та мало бібліотек для комерційних проєктів, часто асоціюється з академічним застосуванням, а не комерцією.
 - *Пояснення:* це лише частково правда і це виходить з того, що Lisp ніколи не був і навряд буде (на жаль) таким же популярним, як, наприклад, JavaScript чи Python. Проте бібліотеки і спільнота є. Якщо казати про окремі діалекти, як, наприклад, Clojure, тоді бібліотек навіть більше ніж в Java, оскільки Clojure працює на JVM, і в ньому можна використовувати код з Java, Scala, або інших мов, що базуються на JVM.

За що варто вивчити Lisp, так це за чудовий приклад функційного підходу до написання програм; концепції "функції як об'єкту", застосування макросів (точніше, **відповідального** застосування макросів), метапрограмування та багато іншого. Хоча Lisp і вважається функційною мовою програмування, проте в ньому також реалізовано й інші парадигми (як от ООП в Common Lisp).

Згідно з опитування від StackOverflow за 2024 рік [14], Lisp та Clojure займають одні з найвищих сходинок за величиною заробітної плати (до речі, як і інші функційні мови — Erlang та Elixir) (рис. 1.2). З іншого боку, зазвичай розробники Lisp мають позицію Middle, Senior та вище.

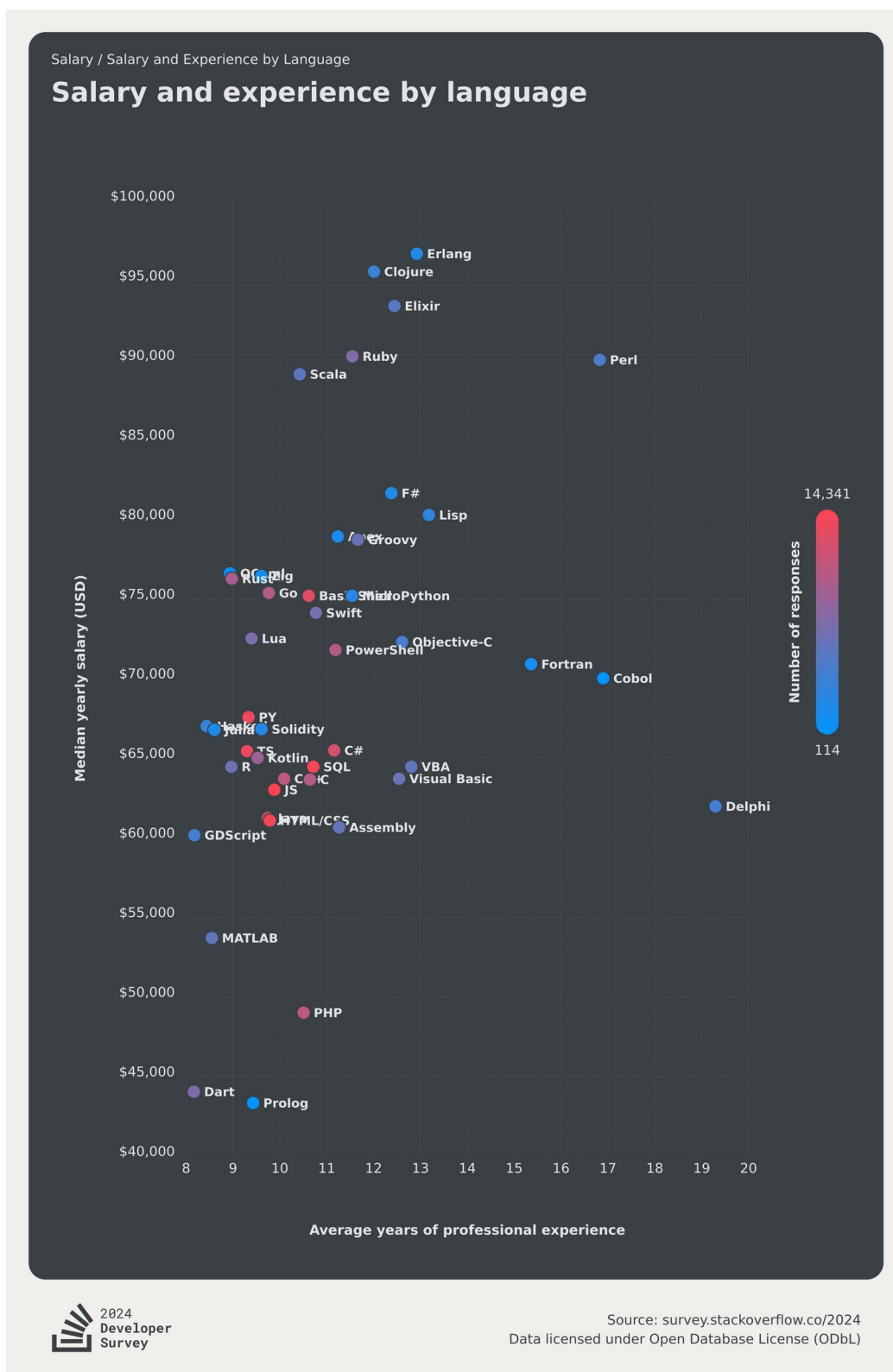


Рисунок 1.2 — Рівень зарплат програмістів різними мовами
(StackOverflow 2024 Survey)

В рамках курсу ми вивчатимемо мову Common Lisp [10]. Чому вивчатимемо саме цей діалект Lisp, а не, наприклад, Clojure? Common Lisp є досить простим і тому його простіше опанувати програмістам, які раніше не використовували суто функційні мови. Clojure потребує більше знань та досвіду, в т. ч. деяких знань з Java (бажано). Тому, щоб спростити процес вивчення концепцій функційного програмування, розглянемо їх на базі саме Common Lisp.

1.3.3 Налаштування середовища розробки для Common Lisp

Як і зазначено вище, в рамках даної дисципліни ми ознайомимось із принципами функційного програмування на прикладі мови програмування Common Lisp [10]. Common Lisp — це стандарт ANSI, одна з найвідоміших спроб стандартизувати мову Lisp. Із описом функцій і макросів мови можна ознайомитись на сайті LispWorks [17].

Для розробки програм мовою Common Lisp в рамках курсу ми використовуватимемо компілятор SBCL (Steel Bank Common Lisp) [11] та одне із середовищ розробки мовою, яке необхідно встановити на свій комп'ютер.

1.3.3.1 Простий спосіб налаштування

Для спрощення процесу встановлення середовища розробки, рекомендується встановити [Portacle](#) [18] — середовище розробки, створене на базі редактора Emacs. Для роботи в ньому необхідно трохи розібратись із [комбінаціями клавіш Emacs](#) [19], але в іншому це готовий інструмент, в якому є все (або майже все), що знадобиться для вивчення та ефективної розробки мовою Common Lisp (в т. ч. Slime [12]).

Також варіантом може бути встановлення лише SBCL та будь-якого текстового редактора. В такому випадку розробка вестиметься шляхом редагування файлів в редакторі і тоді запуску програми після завантаження написаного файлу в REPL SBCL.

1.3.3.2 Emacs

Чи не найпопулярнішим середовищем розробки мовою Common Lisp є [Emacs](#) [15]. Emacs написаний на одному з діалектів Lisp — Emacs Lisp. Це повністю налаштовний редактор, який в комбінації з різними модами перетворюється в повноцінне і зручне середовище розробки мовою Lisp. Emacs можна розширювати різними модами та власними утилітами. Моді поділяються на основні (major) та другорядні (minor). Основний мод, який дає змогу значно пришвидшити розробку та відлагодження програм мовою Common Lisp є [Slime] (<https://slime.common-lisp.dev/>) [12]. Детальніше можливості Emacs та Slime розглядатимуться на окремому лекційному занятті.

Для встановлення Emacs достатньо встановити його з "магазину застосунків" на вашій ОС (якщо він там є), або ж скориставшись інструкцією на сайті: <https://www.gnu.org/software/emacs/>. Для встановлення і налаштування SBCL та Slime можна скористатись інструкцією: <https://lisp-lang.org/learn/getting-started/>.

Примітка: Portacle вже містить в собі SBCL, Slime та інші моди та бібліотеки.

1.3.3.3 IntelliJ IDEA

Для IDE IntelliJ IDEA існує плагін для роботи з Lisp — SLT [16]. Інструкцію по його встановленню можна знайти на GitHub за посиланням: <https://github.com/Enerccio/SLT>.

1.3.3.4 Visual Studio Code

Для VS Code існують різні розширення, наприклад <https://marketplace.visualstudio.com/items?itemName=rheller.alive>. Проте огляд їх функціональності не охоплюється в цьому курсі.

1.3.3.5 Після встановлення

В разі успішного встановлення SBCL та/або середовища розробки (що містить SBCL), після запуску інтерпретатора має з'явитись термінал з пропозицією вводу:

```
CL-USER>
```

Після цього все готове для того, щоб починати писати перші програми мовою Common Lisp.

1.3.4 REPL

Ознайомившись із базовими типами даних, можна перейти до опановування базових функцій Common Lisp. Проте перед цим варто розібратись із тим, що таке REPL і як відбувається інтерпретація форм.

Як було зазначено в розділі 1.3.3.5, після запуску SBCL в Emacs відкривається консоль, з пропозицією вводу: CL-USER> CL-USER — це назва пакета за замовчуванням. *Пакети* в Common Lisp є аналогами просторів імен (namespace). (Детальніше про пакети можна дізнатись в [51].) Після того, як з'явилась пропозиція вводу, можна починати вводити програму (символьні вирази). А називається ця консоль Read-Eval(uate)-Print Loop, або ж *REPL*.

Примітка: якщо запустити просто SBCL (поза середовищем розробки), назва пакету може не з'являтися. Надалі приклади наводитимуться з середовища Emacs + Slime / Portacle, тож вигляд прикладів може відрізнятися від того, що можна побачити в SBCL, що використовується безпосередньо, або в інших середовищах розробки. Різні особливості роботи з Emacs наведені в Додатку 2.

REPL циклічно виконує три дії:

- зчитує (read) всі символьні вирази, які отримує на вхід...;

- ...і виконує їх (evaluate) — тобто інтерпретує;
- після завершення виконання, REPL виводить (print) результат та пропонує ввести наступний s-вираз — починається наступна ітерація.

```
CL-USER> 42
42
CL-USER>
```

REPL завжди виконує всі три етапи. Тобто він завжди намагається виконати або *обчислити* значення того символьного виразу, який він зчитав. Результатом обчислення чисел є самі числа, адже вони позначають самі себе. Результатом обчислення символу буде його значення. У випадку, коли REPL зчитує список, він інтерпретує список як виклик функції і запускає її виконання.

1.3.4.1 Деякі особливості роботи з REPL SBCL

Якщо ввести кілька s-виразів одразу — всі будуть виконані по черзі. Якщо натиснути "Ввести" до того, як s-вираз введено повністю, відбудеться перенесення курсора на новий рядок, де можна закінчити введення виразу.

```
CL-USER> (print 1) (print 2) (print 3
)
1
2
3
3
```

В прикладі вище спочатку послідовно були виведені три числа (виконалась функція print), після чого REPL повернув результат виконання останнього s-виразу (print 3). Функція print друкує свій аргумент і повертає його ж.

Деякі символи та комбінації клавіш в REPL дають змогу отримати один з останніх результатів (*, **, ***) чи одну з останніх введених команд (+, ++, +++). В

разі, якщо щось пішло не так, можна припинити виконання поточного s-виразу за допомогою комбінації клавіш C-c C-c (C відповідає Ctrl). Завершити роботу REPL можна викликом функції `exit`. Комбінація клавіш C-j дає змогу перенести курсор на наступний рядок не виконуючи вирази, які наразі введені.

В разі виникнення помилки, буде виведено повідомлення про помилку в окремому буфері Emacs, а також перелік можливих дій (продовжити, відмінити та ін.) та зворотне трасування (`backtrace`) (рис. 1.3). Для відміни виконання поточного виразу достатньо обрати варіант `*ABORT` (1) або натиснувши на нього, або натиснувши на відповідну цифру на клавіатурі.

Загалом, Slime надає досить широкі можливості для відлагодження програм в REPL. Можливості відлагоджувача (`debugger`) в Slime наведені в [52].

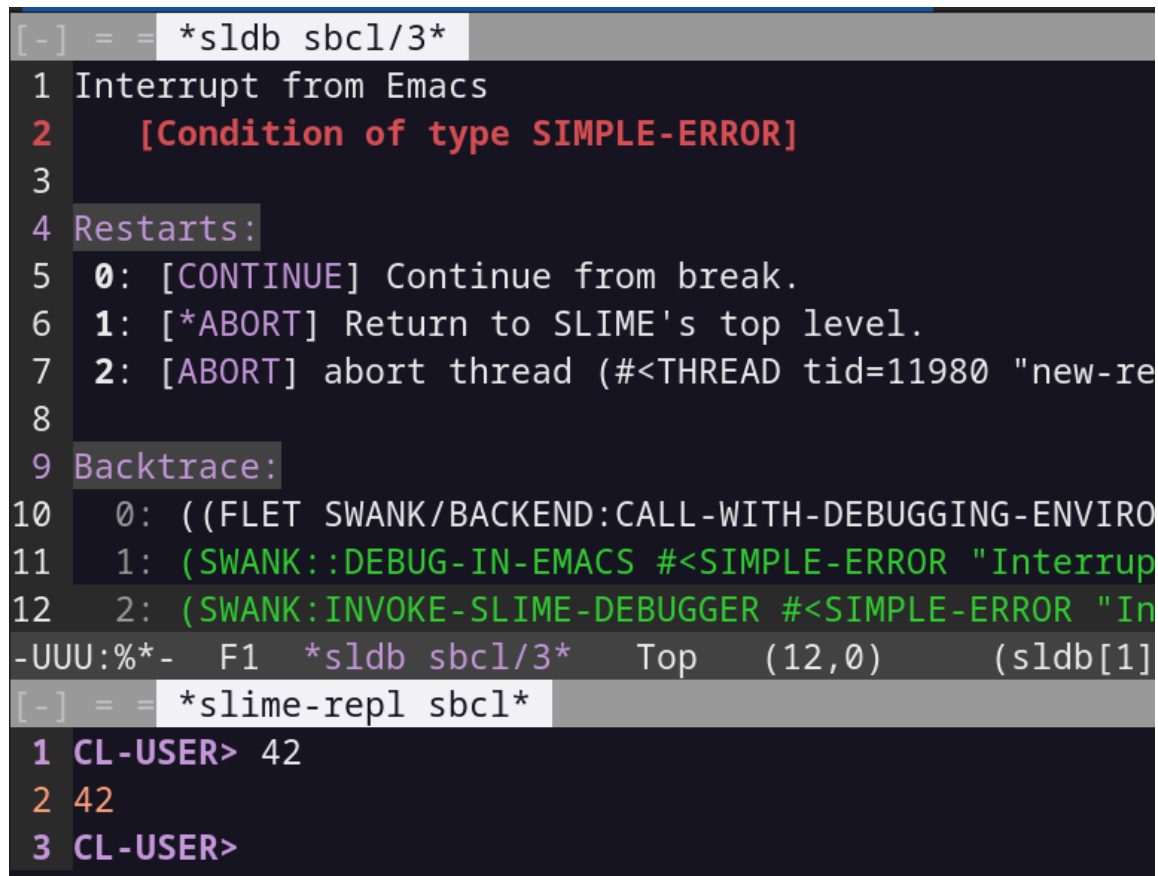
The image is a screenshot of the Emacs REPL interface. At the top, a command line shows `[ - ] = = *sldb sbcl/3*`. Below it, a list of numbered options is displayed:
1 Interrupt from Emacs
2 [Condition of type SIMPLE-ERROR]
3
4 Restarts:
5 0: [CONTINUE] Continue from break.
6 1: [*ABORT] Return to SLIME's top level.
7 2: [ABORT] abort thread (#<THREAD tid=11980 "new-re
8
9 Backtrace:
10 0: ((FLET SWANK/BACKEND:CALL-WITH-DEBUGGING-ENVIRO
11 1: (SWANK::DEBUG-IN-EMACS #<SIMPLE-ERROR "Interrup
12 2: (SWANK:INVOKE-SLIME-DEBUGGER #<SIMPLE-ERROR "In
Below the list, a status line shows `-UUU:%*- F1 *sldb sbcl/3* Top (12,0) (sldb[1]`. The next command line is `[ - ] = = *slime-repl sbcl*`. The final part of the screenshot shows the REPL prompt `CL-USER>` with the input `42` and the output `42`.

Рисунок 1.3 — Помилка та зворотне трасування в Slime REPL

1.3.4.2 Завантаження файлів з кодом

Звісно, що писати програми безпосередньо в REPL може бути незручно. Особливо, якщо мова йде про великі за обсягом програми. Значно простіше написати програму у редакторі з підсвіткою синтаксису та, якщо це спеціалізоване середовище розробки, допоміжними функціями.

Файли з програми мовою Common Lisp зазвичай мають розширення `.lsp` або `.lisp`. Для завантаження файлу у REPL можна скористатись функцією `load`. Нехай маємо файл з визначенням змінної `hero-name` та функції `my-print`:

```
(defparameter hero-name "Mad Max" "Name of a hero") ; визначення змінної
(defun my-print (arg) ; визначення функції
  (format t "out> ~S~%" arg) ; виклик функції форматованого виводу
  arg)
```

Завантаження файлу в REPL виглядає ось так:

```
CL-USER> (load "~/tmp/tasks.lsp")
T
CL-USER> hero-name
"Mad Max"
CL-USER> (my-print "Hello world!")
out> "Hello world!" ; надруковано функцією format
"Hello world!" ; результат функції my-print
```

1.3.4.3 Завантаження файлів зі Slime в Emacs

При роботі в середовищі Emacs (в т. ч. в Portacle) з встановленим модом Slime, завантажувати файл явним чином не обов'язково. Після запуску Slime з міні-буфера Emacs (`M-x slime RET` — тобто необхідно ініціювати введення команди натиснувши `Alt-x`, далі ввести назву команди (`slime`) і натиснути "ввести" (`RET`)) буде запущено REPL (в Portacle це відбувається одразу після запуску автоматично),

і тоді за допомогою комбінації клавіш C-c C-k можна вказати Slime зчитати весь поточний файл (буфер). При цьому буфер з файлом має бути активним. Також можна зчитати лише окрему форму — в якій або перед якою наразі знаходиться курсор. Для цього можна скористатись командою C-c C-s. Більше про роботу з Emacs та Slime можна прочитати за посиланнями:

- Emacs: <https://www.gnu.org/software/emacs/tour/index.html> та <https://www.gnu.org/software/emacs/refcards/pdf/refcard.pdf> [19]
- Slime: <https://slime.common-lisp.dev/doc/html/> та <https://dept-info.labri.fr/~idurand/enseignement/lst-info/PFS/Common/slimer-refcard.pdf>

Крім того в Emacs варто встановити і активувати (в Portacle вже є) мод `paredit` [26], який слідкує за тим, щоб кількість дужок в програмі завжди була парною, що значно спрощує написання програм на Common Lisp в Emacs.

Примітка: для спрощення роботи з Emacs можна також встановити мод CUA. Він додасть чотири комбінації клавіш, що є типовими для редагування коду в інших редакторах: C-c (`copy`), C-v (`paste`), C-x (`cut`), C-z (`undo`). Детальніше можна дізнатись за посиланням [53]. Аналогічні комбінації є й в Emacs, але вони відрізняються [19]: M-w (`copy active region to the kill ring`), C-y (`paste — copy last thing from kill ring`), C-w (`cut — kill region and put it to the kill ring`), C-x у або C-_ або C-/ (`undo`). "region" — це виділений наразі текст (в термінології Emacs).

1.3.5 Виклики функцій в Lisp та інтерпретація s-виразів

Виклик функції у Lisp, на відміну від, наприклад, C-подібних мов, таких як C чи Java, має префіксний запис. Тобто, виклик функції має вигляд списку, першим елементом якого є назва функції:

```
Math.sqrt(49) // Java
```

```
(sqrt 49)      ;; Common Lisp
```

Такий запис застосовується не лише для функцій, аналогічних функціям в інших мовах програмування, але і для арифметичних операцій, які часто в інших мовах записуються в інфіксній формі:

```
5 * 2 + 9 / 3 // Java
```

```
(+ (* 5 2) (/ 9 3)) ;; Common Lisp
```

Тобто арифметичні операції в Lisp також реалізуються у вигляді виклику функцій.

Така форма запису хоч і дещо незвична для тих, хто почав вивчення програмування з не Lisp-подібних мов, проте є досить зручною, оскільки не потребує складного синтаксичного аналізу, а інтерпретатор Lisp вже по першому елементу списку визначає, яку функцію необхідно викликати.

Взаємодія програміста з інтерпретатором відбувається шляхом виклику програмістом функцій. Коли введення виклику завершено, інтерпретатор починає обчислення введеного рядка. Наприклад, якщо користувач ввів `(+ 2 3)`, інтерпретатор по першому елементу списку (символу) визначить, що робиться виклик функції додавання, а з решти елементів (в даному випадку чисел) отримає значення аргументів функції, виконає виклик функції, її обчислення й поверне значення 5.

Якщо ж при виклику функції є вкладені виклики, наприклад `(+ (* 5 2) (/ 9 3))`, інтерпретатор знов спочатку зліва-направо зчитає вхідну форму й визначить, що робиться виклик функції додавання. Після цього він "побачить" що серед аргументів функції є вкладені виклики і перейде до їхнього обчислення — себто визначення що це за функція (множення, ділення), обчислення її аргументів і

виклику функції. І так він робитиме до тих пір, поки всі аргументи функції (в т. ч. вкладені виклики) не будуть обчислені, після чого він обчислить значення самої функції. Іншими словами, перш ніж викликати функцію, спочатку обчислюються її аргументи — як і в інших мовах програмування. Тож можна казати, що операції у дужках в Lisp, як і в математиці, є більш пріоритетними.

Порядок обчислення, коли спочатку обчислюються аргументи, а тоді відбувається виклик функції, в Lisp справджується саме для функцій, проте працює інакше для *макросів* та *спеціальних операторів*. Макрос, якщо просто, — це функція, що виконується на етапі компіляції (а не виконання) програми. Макроси детально розглядатимуться в наступних розділах.

Спеціальний оператор (або також **спеціальна форма**) — це мовна конструкція з визначеним спеціальним чином виконанням [27-28]. Типовим прикладом спеціального оператора є `if`. Якби `if` був функцією, тоді б спочатку обчислювались всі її аргументи (умова і гілки), а тоді вже відбувався "виклик" `if`. Проте очевидно, що `if` має працювати не так. Спочатку обчислюється значення умови і в залежності від результату обчислення умови відбувається обчислення значення однієї з гілок.

```
CL-USER> (if t (print "ok") (print "not ok"))  
"ok"  
"ok"
```

1.3.5.1 Відміна обчислень

Іноді ми не хочемо, щоб інтерпретатор обчислював значення виклику. Наприклад, якщо ми хочемо отримати просто список, не обчислюючи його як виклик функції, і передати цей список в якості аргумента у виклик функції, або ж зробити його значенням якоїсь змінної. В такому випадку для блокування

обчислення інтерпретатором символьного виразу використовується спеціальний оператор `quote`, або ж `'`:

```
CL-USER> (quote (+ 2 3))
(+ 2 3)
CL-USER> '(+ 2 3)
(+ 2 3)
CL-USER> (+ 2 3)
5
```

Спеціальний оператор `quote` приймає аргумент і повертає його в якості значення. Коли інтерпретатор розпізнає `quote`, він не обчислює значення його аргумента. Ставити апостроф перед `T`, `NIL` або числами не потрібно, оскільки вони позначають самі себе. Хоча запис чисел з апострофом допускається. Приклади:

```
CL-USER> (quote 1)
1
CL-USER> (+ '2 3)
5
```

Примітка: `'` можна написати і двічі. Але в такому разі інтерпретатор "обчислить" лише перший `quote` і залишить другий. А результат буде не таким, як якби `'` був лише один:

```
CL-USER> (equal '(1 2) '(1 2))
T
CL-USER> (equal ''(1 2) '(1 2))
NIL
```

1.3.5.2 Обчислення форм з *EVAL*

На противагу спеціальній формі `quote` в Common Lisp є функція `eval`, яка дає змогу обчислити значення форми після того, як її обчислення було відмінене з `quote` або ж якщо форма була програмним чином сконструйована:

```
CL-USER> (eval '(+ 2 3))
```

```

5
CL-USER> (eval 'w)
w
CL-USER> (list 'list 1 2 3)
(LIST 1 2 3)
CL-USER> (eval (list 'list 1 2 3))
(1 2 3)
CL-USER> (eval 'w) ; ERROR

```

При спробі виконати останню форму інтерпретатор спрацює наступним чином: він зчитає всю форму, визначить, що це виклик функції `eval`, обчислить її аргумент `'w` (в результаті отримає символ `w`) і виконає виклик `eval`. Оскільки `w` не позначає якогось значення (не було жодних "присвоєнь" перед `eval`-ом), то інтерпретатор видає помилку `UNBOUND-VARIABLE`. Звісно, якби у `w` було якесь значення, то все було б добре, і `eval` повернув би його. Наприклад ось так:

```

CL-USER> (defparameter w 1)
w
CL-USER> (eval 'w)
1

```

1.3.6 Коментарі

В Lisp, як і в інших мовах, є два види коментарів: однорядкові та багаторядкові.

Однорядкові коментарі вже застосовувались в прикладах. Такі коментарі починаються з *крапки з комою* `;`. При цьому згідно з конвенцією коментування програм мовою Lisp, кількість послідовних `;` на початку коментаря відрізняється в залежності від контексту та мети застосування [29]:

```

;;; Загальний коментар до файлу або логічного відокремленого фрагменту коду.
;;; Такий коментар вирівнюється по лівому краю, зазвичай застосовується як
;;; заголовок файлів з програмами

;;; Коментар до секції коду. Цей коментар також вирівнюється по лівому краю
(defun foo (arg)
  "Документаційний опис функції (docstring)"
  (print arg) ; коментар, що пояснює особливість

```

```
; певного рядка. Зазвичай такі коментарі  
; вирівнюються редакторами по якійсь  
; конкретній колонці (наприклад, 41-й)  
; або ж через одну позицію після  
; завершення тексту у рядку, якщо довжина  
; тексту коду в рядку перевищує 40  
; символів  
;; Коментар, що ідентується разом з кодом, якому передує. Застосовується для  
;; опису стану обчислень в поточний момент часу (напр. опис етапу алгоритму)  
(+ arg 1))
```

Багаторядкові коментарі позначаються за допомогою символів початку `#|` і кінця `|#`:

```
#|Це багаторядковий  
коментар|#
```

1.3.7 Завдання на СРС

1. Встановити SBCL та/або середовище розробки, що містить SBCL.
2. Переглянути сайт Common Lisp HyperSpec [17]:
<https://www.lispworks.com/documentation/HyperSpec/Front/>.
3. Переглянути розділ «Lather, Rinse, Repeat: A Tour of the REPL» щодо налаштування REPL [50].

Додатково можна також ознайомитись зі статтею Introduction: Why Lisp? <https://gigamonkeys.com/book/introduction-why-lisp>, в якій робиться огляд мови Lisp та наводяться аргументи щодо того, чому варто вивчати цю мову.

РОЗДІЛ 2

ОСНОВИ МОВИ COMMON LISP

Тема 2.1 Базові типи даних та функції

Основними типами даних мови є *символи* (символьні структури), *числа* та *списки*. Є й інші структури даних, такі як власне структури (struct), рядки, масиви та ін. Деякі з них розглядатимуться окремо — в наступних розділах.

2.1.1 Символи

Зазвичай, коли мова заходить про **символи** (symbol) в Common Lisp, то маються на увазі символльні структури, а не рядкові символи (character). Символи позначаються ідентифікаторами, що складаються з літер, цифр та спеціальних символів. Спеціальні символи, які можуть використовуватись в іменах символів, такі:

+ - * / @ \$ % ^ & _ \ < > ~ ! ? [] .

Приклади символів:

```
a, symbol, column12, foo-bar
```

Кожен символ може позначати якусь сутність (тобто, символи використовуються як *змінні*) — число, список, складну структуру даних, тощо. Або ж він може позначати функцію чи фрагмент коду програми. Завдяки цьому, до речі, мову Lisp відносять до *символьних мов програмування* [20] — тип програмування, в якому змінні можуть позначати не лише значення, але й (під)програми. Крім Lisp до цієї парадигми також відносять мову логічного програмування Prolog. В Lisp символи можуть одночасно позначати як значення

так і функцію, а чим саме є символ в той чи інший момент часу визначається контекстом його використання. Але про це далі.

При іменуванні символів слід керуватись конвенціями іменування для розробників мовою Lisp (Додаток 1, [21, 22]). SBCL є регістро-нечутливим, тож написання символів дозволяється в будь-якому регістрі — SBCL змінить регістр всіх літер на верхній самостійно. При цьому варто зауважити, що верхній регістр в іменах символів при написанні програм застосовується досить рідко. Зазвичай всі ідентифікатори пишуться в нижньому регістрі.

Також при іменуванні символів можна використовувати й інші символи, крім тих, що перелічені вище. Наприклад, «пробіл». Проте для цього необхідно або "екранувати" їх, тобто додавати перед такими символами символ \ (back slash), або ж використати спеціальний формат запису символу — у вертикальних рисках (|, bar). Приклад:

```
foo\ bar, |foo bar|
```

Варто зауважити, що при використанні спеціального формату запису символів у вертикальних рисках регістр літер зберігається. Тому `foo\ bar` $\neq$ `|foo bar|`, оскільки інтерпретатор SBCL змінить перший символ на `F00\ BAR`.

Проте це ще не все, що можна розповісти про символи. Більш детально їхня структура розглядається в розділі 2.2. Як і "ключові" символи.

2.1.1.1 Константи

Зазвичай символи можуть позначати довільне значення, яке відрізняється в залежності від контексту використання символу. Наприклад, символ, що позначає формальний параметр деякої функції, може мати різні фактичні значення. Або у символ може бути встановлене значення за допомогою спеціального оператора `let`, функції `set`, тощо.

Проте є такі символи в Common Lisp, які завжди приймають лише одне значення. До таких символів відносяться символи T та NIL. T часто відповідає узагальненому логічному значенню true, але, все ж, не є саме булевим значенням, як в інших мовах програмування. В деяких випадках він позначає «будь-що». Аналогічно, NIL часто використовується в якості значення false, проте також не є саме булевим значенням. Крім того, NIL позначає «пустий список» (див. р. 2.1.3). Ці символи позначають самі себе і не можуть змінювати значення.

```
CL-USER> t
T
CL-USER> nil
NIL
```

Крім того є власне **константи** — наперед визначені символи, які мають фіксоване значення. Прикладом такої константи є константа, що позначає число Пі — PI:

```
CL-USER> PI
3.141592653589793d0
```

Також можна визначити користувацькі константи за допомогою макроса defconstant:

```
CL-USER> (defconstant +hero-name+ "Mad Max" "Name of a hero")
+HERO-NAME+
CL-USER> +hero-name+
"Mad Max"
```

Примітка: значення, надане константі, "Mad Max", — це *рядок*. Рядки будуть розглянуті в наступних розділах.

2.1.2 Числа

Числа в Common Lisp — це примітивні об'єкти, що позначають самі себе. Числа записуються починаючи з цифри, можуть мати префікс + чи -, містити . чи / (для дробових значень), або ж закінчуватись позначенням експоненти.

Приклади чисел [23], [24]:

```
123      ; ціле число
2/9      ; дробове число
1.0      ; число з рухомою комою
1.0e0    ; інший спосіб позначення числа з рухомою комою
1.0d0    ; число з рухомою комою з подвійною точністю
1.0e-4   ; число з рухомою комою, що відповідає значенню 0.0001
+42      ; ціле додатне значення 42
-42      ; ціле від'ємне значення 42
-1/4     ; мінус одна четверта
246/2    ; інший спосіб запису цілого значення 123
#b101    ; 5 у двійковій системі числення
#xa      ; 10 в шістнадцятковій системі
#36rX    ; 33 в системі числення з основою 36
```

Числові типи даних (в першу чергу цілі) можуть мати необмежений розмір. Тобто, якщо в якийсь момент значення числа перевищує значення, припустимо, fixnum (64 біти — аналог long в інших мовах), тоді тип даних буде "розширено" до bignum. Також варто зазначити, що пори різні варіанти запису в пам'яті числа одного типу представляються однаково. Наприклад, 1.0 і 1.0e0 матимуть один і той самий тип даних single-float. Однак, програміст зазвичай не переймається типами даних, оскільки явно зазначати тип даних у змінних не потрібно, тож це все лишається прихованим від нього.

Більше про різні типи чисел можна дізнатись в [24], [25].

Символи і числа є найпростішими об'єктами Lisp. Їх разом (а також деякі інші типи) називають *атомами*.

2.1.3 Списки

Список — це впорядкована послідовність, елементами якої є атоми, інші списки (підсписки) або інші об'єкти. Список записується у круглих дужках, всередині яких знаходяться елементи списку, розділені пробілами (пробільними символами). Списки завжди починаються відкриваючою круглою дужкою (й закінчуються закриваючою круглою дужкою). Маючи таке визначення списку, можна сказати, що список — це багаторівнева чи ієрархічна структура, в якій відкриваючі і закриваючі дужки знаходяться у строгій відповідності. Приклади списків такі:

```
(a b 3)           ; список з трьох атомів — двох символів і одного числа
(+ 2 (- 3 1))     ; список з трьох елементів — двох символів і одного підсписку,
                  ; що складається з одного символу і двох чисел
(((one) 2) three) ; список з двох елементів — підсписку ((one) 2) та символу
                  ; three. В свою чергу підсписок ((one) 2) складається з
                  ; підсписку (one) та числа 2. І так далі
```

До пробільних символів відносяться символи пробілу, табуляції, перенесення рядка та інші — загалом, як і в багатьох інших мовах програмування.

Пустий список позначається дужками без елементів () або NIL. Пустий список — це не те ж саме, що й «нічого». Тобто, (NIL) — це список, що містить один елемент — пустий список. (NIL T (NIL) ()) — це список з чотирма елементами, перший та останній — пусті списки, а третій — список з одним елементом — пустим підсписком.

В залежності від контексту використання списки можуть бути розглядатись як дані або як виклик функції. Наприклад, список (+ 1 2) може бути інтерпретований як список, що складається з трьох атомів, а може бути інтерпретований як виклик функції + з аргументами 1 та 2.

Сформовані у вигляді списків структури даних дещо нагадують дані, записані у форматах XML або JSON. Наприклад:

```
(ноутбук  
 (виробник "Apple Inc.")  
 (модель "MacBook Air 15")  
 (технічні-характеристики  
  (процесор "Apple M2")  
  (озп 8192)  
  ...))
```

2.1.4 Символьні вирази

Атоми і списки також називаються *символьними виразами* або *s-виразами* (s-expression) (рис. 2.1).

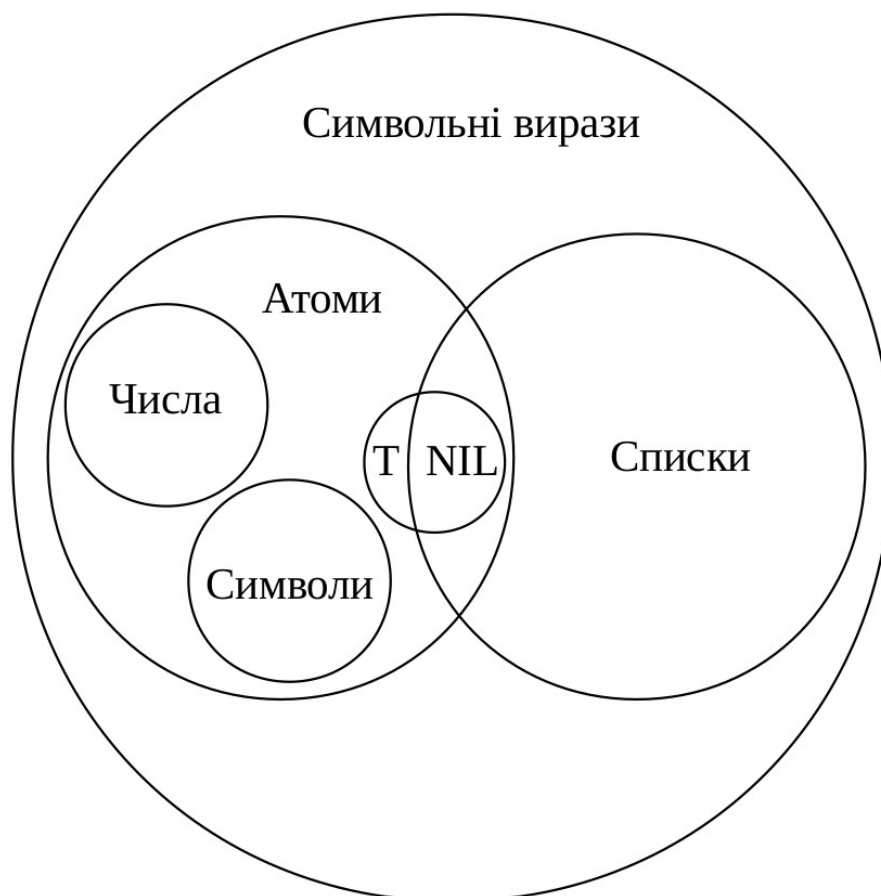


Рисунок 2.1 — Символьні вирази

Символьні вирази, які можна обчислити, також називають *формами*.

Наприклад:

```
(+ 1 2) ; можна обчислити тож це форма, але також і символний вираз  
(1 + 2) ; не можна обчислити, тому це не форма, але все ще символний вираз
```

2.1.5 Основні функції роботи зі списками

Базові функції мови Common Lisp можна поділити на кілька груп: основні базові функції, арифметичні операції, предикати, функції для роботи зі списками та ін.

До основних функцій роботи зі списками відносяться функції отримання голови (CAR) та хвоста (CDR) списку, додавання елемента до голови списку (CONS), предикати ATOM та EQ.

2.1.5.1 Отримання елементів списків

За допомогою функції CAR можна отримати "голову" списку, тобто його перший елемент. Якщо аргументом CAR буде не список, а атом (якщо це не NIL), — виникне помилка. Для порожнього списку CAR поверне NIL.

CDR дає змогу отримати "хвіст" списку, тобто список, що містить всі елементи, які розташовані за "головою" списку-аргумента CDR. Якщо список-аргумент порожній або містить лише один елемент, тоді результатом виклику CDR буде NIL. Якщо аргументом є атом не-NIL, тоді виникне помилка.

Приклади роботи CAR та CDR для різних аргументів наведені в таблиці 2.1 (адаптовано з [30]).

Загалом, маючи ці дві функції, можна отримати довільний елемент списку:

```
CL-USER> (car (cdr '(a b c)))  
B  
CL-USER> (cdr (car (cdr '(a (b c) d))))  
(C)
```

Таблиця 2.1 — Робота CAR та CDR

X	(CAR X)	(CDR X)
NIL	NIL	NIL
A	ERROR	ERROR
(A)	A	NIL
(A 2)	A	(2)
(A 2 C)	A	(2 C)
((A 2) C)	(A 2)	(C)
((A 2) (C 4))	(A 2)	((C 4))

В Common Lisp також додані функції, що є комбінацією CAR та CDR. Їхні назви починаються з "C" і закінчуються на "R", а між цими двома символами знаходяться різні комбінації "A" та "D" кількістю символів до 4 штук. Аналогічними формам, наведеним вище, є такі:

```
CL-USER> (cadr '(a b c))
B
CL-USER> (cdadr '(a (b c) d))
(C)
```

Іноді зручними можуть бути такі функції як FIRST, SECOND, THIRD, REST, NTH, NTHCDR та інші (табл. 2.2).

Таблиця 2.2 — Інші функції для отримання елементів списку

Форма	(A 2 C 4 E)
(first x)	A
(second x)	2
(third x)	C
(rest x)	(2 C 4 E)
(nth 3 x)	(A 2 C 4 E)
(nthcdr 3 x)	(4 E)

Як видно з прикладу, першим аргументом NTH є *індекс* потрібного елемента, а не *позиція* відносно початку списку.

NTHCDR повертає список, починаючи з індексу, заданого першим аргументом.

2.1.5.2 Примітивні предикати

Предикат — це функція, що виконує деяку перевірку і повертає булеве значення — "true" або "false" (T або NIL). Найпростішими предикатами в Common Lisp є функції ATOM та EQ.

Функція ATOM перевіряє, чи є її аргумент атомом — тобто символом або числом:

```
CL-USER> (atom 'a)
T
CL-USER> (atom 42)
T
CL-USER> (atom NIL)
T
CL-USER> (atom '(1))
NIL
```

Функція EQ порівнює два об'єкти і визначає чи вони є тим самим об'єктом. Функцію варто застосовувати для порівняння символів (саме символів, а не їх значень) або ж об'єктів, якщо є необхідність перевірити, чи об'єкти мають "однакове посилання". Хоча зазвичай для чисел і рядкових символів функція поверне правильний результат, застосовувати її для їх порівняння не варто, оскільки в загальному випадку числа і рядкові символи можуть бути скопійовані, через що порівняння однакових чисел з EQ поверне NIL. Аналогічна ситуація зі списками і рядками: однакові за структурою списки або за вмістом рядки можуть бути різними за EQ, якщо вони є не одним і тим самим об'єктом, а копіями. Приклади:

```

CL-USER> (eq 'a 'a)
T
CL-USER> (eq 'a 'b)
NIL
CL-USER> (eq 3 3)
T ;; або `NIL`
CL-USER> (eq 3 3.0)
NIL
CL-USER> (let ((x '(1 2 3))) (eq x x))
T
CL-USER> (eq '(1 2 3) '(1 2 3))
NIL
CL-USER> (eq "Foo" "F00")
NIL
CL-USER> (eq "Foo" "Foo")
NIL ;; або `T`
CL-USER> (let ((x "Foo")) (eq x x))
T

```

Для точного порівняння чисел, рядкових символів, списків чи рядків (за вмістом), або ж інших значень, варто застосовувати складніші предикати, які розглядаються в розділі 2.1.6.

2.1.5.3 Додавання елементів до голови списку

Найпростіший спосіб розширити список — це додати елемент до його голови за допомогою функції CONS. CONS додає заданий об'єкт (перший аргумент) до голови списку (другий аргумент), утворюючи новий список. Приклади роботи CONS наведені в таблиці 2.3.

Таблиця 2.3 — Робота CONS

X	Y	(CONS X Y)
NIL	NIL	(NIL)
1	NIL	(1)
NIL	(1)	(NIL 1)
1	(2 3)	(1 2 3)
(1 2)	(3)	((1 2) 3)
(1 2)	((3 4))	((1 2) (3 4))
1	2	(1 . 2)

В останньому записі таблиці другим аргументом CONS є число 2, а не список. Якщо другим аргументом функції CONS є не-список, тоді результатом виконання CONS буде так звана *точкова пара*. Такий результат пов'язаний із внутрішньою організацією списків — а саме з тим, що вони складаються зі *спискових комірок*. Більш детально про них йтиметься в розділі 2.3.

Як і зазначено вище, CONS, додаючи елемент, утворює *новий список*. Тому результатом порівнянь початкового списку з результатом роботи CONS завжди буде NIL:

```
CL-USER> (let ((lst '(2 3)))  
           (eq lst (cons 1 lst)))  
NIL
```

2.1.6 Предикати для порівняння двох об'єктів

Існують і інші предикати для порівняння двох об'єктів крім EQ. В залежності від того, які типи порівнюваних об'єктів, а також того, на скільки детально необхідно їх порівнювати, застосовують такі функції як EQL, EQUAL та EQUALP. Послідовність функцій за складністю перевірок така: EQ < EQL < EQUAL < EQUALP. Додатково про ці предикати можна почитати на сайті The Common Lisp Cookbook [31].

2.1.6.1 Порівняння з EQL

EQL застосовується для порівняння чисел одного типу та рядкових символів. Для символів або складніших структур даних (списки, рядки та ін.) результат буде таки самий, як і у EQ:

```
CL-USER> (eql 1 2)  
NIL
```

```
CL-USER> (eq 2 2)
T
CL-USER> (eq 2.0 2.0)
T
CL-USER> (eq 2 2.0)
NIL
CL-USER> (eq '(1 2 3) '(1 2 3))
NIL
```

Примітка: для порівняння чисел різного типу використовується функція = (розділ 2.1.7.2).

2.1.6.2 Порівняння з EQUAL

За допомогою EQUAL можна порівнювати списки та рядки. Якщо ж застосувати цю функцію для чисел чи символів, тоді порівняння відбуватиметься аналогічно EQL:

```
CL-USER> (equal '(1 2 3 (4)) '(1 2 3 (4)))
T
CL-USER> (equal '(1 2) '(1 2 3))
NIL
CL-USER> (equal 'a 'a)
T
CL-USER> (equal 3.0 3.0)
T
CL-USER> (equal 3 3.0)
NIL
CL-USER> (equal "abc" "abc")
T
CL-USER> (equal "ABC" "abc") ; equal – регістро-чутливий при порівнянні рядків
NIL
```

2.1.6.3 Порівняння з EQUALP

Найбільш загальним предикатом для порівняння двох об'єктів є EQUALP. Він може порівнювати як числа різних типів, так і будь-які складні структури даних:

```
CL-USER> (equalp 1 1.0)
T
CL-USER> (equalp '(a b c (d)) '(a b c (d)))
T
CL-USER> (equalp "ABC" "abc") ; equalp – регістро-нечутливий при порівнянні
; рядкових символів і, відповідно, рядків
T
```

Варто зауважити, що оскільки EQUALP є найбільш універсальним предикатом, час роботи більший в порівнянні з іншими предикатами. Тому його варто застосовувати лише тоді, коли в цьому дійсно є необхідність.

2.1.7 Операції з числами

Операції з числами можна розділити на арифметичні і інші операції та числові предикати.

2.1.7.1 Арифметичні і інші операції

Арифметичні операції в Common Lisp реалізовані у вигляді функцій: +, -, *, /, REM, MOD, 1+ (інкремент), 1- (декремент):

```
CL-USER> (+ 1)
1
CL-USER> (- 6 3 5)
-2
CL-USER> (* 2 3 4)
24
CL-USER> (/ 7 2)
7/2
CL-USER> (/ 8 2)
4
CL-USER> (rem 7 5)
2
CL-USER> (*) ; якщо аргументів немає, тоді результатом множення буде одиниця
1
CL-USER> (+) ; якщо аргументів немає, тоді результатом додавання буде нуль
0
CL-USER> (1+ 3)
4
```

Функції + та * приймають довільну кількість аргументів (0 і більше), в той час як функції - та / також приймають довільну кількість аргументів, але мінімум один аргумент — зменшуване і ділене відповідно.

Також в Common Lisp реалізовані тригонометричні функції (SIN, COS, TAN та ін.), функції ділення з округленням (CEILING, ROUND, FLOOR та ін.) та інші. Про ці та інші функції детально можна прочитати в HyperSpec [17].

2.1.7.2 Числові предикати

До числових предикатів відносяться такі функції:

- `=` — використовується для порівняння чисел, можливо різних типів
- `/=` — перевірка чисел на нерівність
- `<`, `>`, `<=`, `>=` — інші функції порівняння чисел
- `NUMBERP` — перевіряє, чи є аргумент числом
- `ZEROP` — перевіряє, чи є аргумент нулем
- `PLUSP`, `MINUSP` — перевіряють знак числа
- `EVENP`, `ODDP` — перевірка на парність чи непарність

```
CL-USER> (= 1.0 1)
T
CL-USER> (/= 1 2 1.0)
NIL
CL-USER> (< 1)
T
CL-USER> (<= 0 1 1 2)
T
CL-USER> (> 3 2 2)
NIL
CL-USER> (numberp 'a)
NIL
CL-USER> (numberp 10)
T
CL-USER> (evenp 7)
NIL
```

Предикати `=`, `/=`, `<`, `>`, `<=`, `>=` можуть приймати довільну кількість аргументів, але мінімум один, і послідовно попарно (крім `/=`) їх порівнюють. Якщо аргумент лише один — тоді результатом завжди буде `T`.

2.1.8 Логічні операції

До логічних операцій відносяться функція NOT та макроси AND і OR.

NOT повертає NIL, якщо аргумент не NIL, і T, якщо його аргумент NIL.

```
CL-USER> (not t)
NIL
CL-USER> (not 1)
NIL
CL-USER> (not '(1 2 3))
NIL
CL-USER> (not nil)
T
CL-USER> (not (not t))
T
```

Макроси AND та OR приймають довільну кількість аргументів (0 і більше) і реалізують логічні операції "І" та "АБО" відповідно.

Результатом виконання макроса AND є:

- результат виконання останньої форми-аргумента, якщо результат виконання всіх попередніх форм-аргументів не NIL
- NIL, якщо хоча б одна з форм-аргументів повертає NIL
- T, якщо у макроса немає аргументів

```
CL-USER> (and t t)
T
CL-USER> (and t 5)
5
CL-USER> (and nil)
NIL
CL-USER> (and t (numberp 1) (> 1 2))
NIL
CL-USER> (and)
T
```

Результатом виконання макроса OR є:

- результат виконання першої форми-аргумента, чий результат не NIL

- NIL, якщо жодна з форм-аргументів не повернула результат, відмінний від NIL, або якщо форм-аргументів не задано

```
CL-USER> (or nil (numberp 'a) (< 2 1))
NIL
CL-USER> (or (numberp 'a) (zerop 0))
T
CL-USER> (or (= 2 (1+ 1)) (print "Hey!")) ; print не виконується, оскільки
T                                         ; справджується перша умова
CL-USER> (or)
NIL
```

2.1.9 Інші функції для роботи зі списками

При роботі зі списками також часто застосовуються функції:

- NULL, що перевіряє, чи є її аргумент пустим списком (NIL)

```
CL-USER> (null nil)
T
CL-USER> (null '(1 2))
NIL
```

- LIST — приймає довільну кількість аргументів і конструює новий список з усіма своїми аргументами в якості елементів

```
CL-USER> (list)
NIL
CL-USER> (list 1 'a (+ 2 3))
(1 A 5)
```

- LIST* — приймає довільну кількість аргументів (але мінімум один) і конструює з них новий список аналогічно LIST, проте останній аргумент додається як "хвіст". Можна розглядати цю функцію як CONS, у якого може бути 0 і більше перших аргументів

```
CL-USER> (list* '(a))
(A)
CL-USER> (list* 1 '(a))
(1 A)
CL-USER> (list* 1 2 3 '(4 5))
(1 2 3 4 5)
CL-USER> (list* 'a)
A
CL-USER> (list* 1 'a)
(1 . A) ; точкова пара - див. розділ 5.1
```

- LAST — повертає *список з останнім елементом* списку-аргументу. Тобто, не сам останній елемент, а список з цим елементом. Це пов'язано з тими, що насправді LAST повертає останню спискову комірку списку (див. р. 2.3.1.1)

```
CL-USER> (last nil)
NIL
CL-USER> (last '(1))
(1)
CL-USER> (last '(1 2))
(2)
```

- APPEND — створює потенційно новий список, об'єднуючи списки-аргументи, яких може бути довільна кількість. Для цього всі списки-аргументи, крім останнього, копіюються

```
CL-USER> (append)
NIL
CL-USER> (append '(1 2))
(1 2)
CL-USER> (append '(1 2) (list 3 4))
(1 2 3 4)
CL-USER> (append '(1 2) (list 3 4) ())
(1 2 3 4)
```

- LISTP — визначає, чи є аргумент списком

```
CL-USER> (listp nil)
T
CL-USER> (listp '(1 2))
T
CL-USER> (listp t)
NIL
```

- LIST-LENGTH — повертає довжину списку-аргументу

```
CL-USER> (list-length ())
0
CL-USER> (list-length '(a 1 b 2))
4
```

2.1.10 Встановлення значення символу

Для того, щоб надати значення деякому символу використовуються функція SET та спеціальний оператор SETQ. Обидві функції мають по два аргументи: символ, якому необхідно надати значення, і власне значення, яке необхідно встановити у символ. Результатом виконання обох є значення другого аргументу. Різниця між SET та SETQ полягає у форматі передачі першого аргументу.

У SET першим аргументом є форма, яка обчислюється, і результатом обчислення якої має бути символ:

```
CL-USER> (set 'a (* 2 5))
10
CL-USER> (set (car '(a b)) 'b) ; встановити символ B значенням символу A
B
CL-USER> (set a 5)
5
CL-USER> b
5
```

Спеціальний оператор SETQ (SET Quoted) є аналогом SET з неявно кватованим першим аргументом: (SETQ a 1) == (SET 'a 1). Крім того, SETQ може приймати й більше, ніж два аргументи — це може бути довільна парна кількість аргументів, серед яких кожен непарний (1, 3, ...) має бути символом, а кожен парний (2, 4, ...) — значенням, яке необхідно надати символу, що переданий перед цим значенням. Встановлення значень відбувається по-черзі, тому попередні символи можна використовувати для визначення значень наступних. Приклади:

```
CL-USER> (setq x 2)
2
CL-USER> (setq x (1+ x))
3
CL-USER> x
3
CL-USER> (setq a 1 b (1+ a) c (+ a b))
3
CL-USER> (list a b c)
(1 2 3)
```

Примітка: при використанні SETQ SBCL може видавати попередження щодо того, що символи, яким необхідно надати значення, — не визначені. Проте при цьому SETQ відпрацьовує як і має. Попередження виникає через те, що ці символи не були попередньо визначені (наприклад, з DEFPARAMETER або DEFVAR) — тобто

вони "невідомі" в поточному контексті. (Про DEFPARAMETER та DEFVAR йдеться в розділі 2.5.2.1.)

Ці функції також називають *псевдофункціями* — функціями, що не лише повертають значення, а також виконують деяку побічну дію — наприклад, змінюють стан якогось об'єкта. У випадку SET та SETQ змінюється значення символу — їх першого аргумента.

2.1.11 Завдання на СРС

Спробуйте передбачити, що буде результатом виконання наступних форм, а тоді виконайте їх в інтерпретаторі і порівняйте очікування з фактичним результатом:

```
;; базові функції
(car (cdr '(1 2 3)))
(cons '(a) '(a b))
(cdr (car (cdr '(1 2 3))))
(atom (car '(1 2 3)))
(eq (cdr (cdr '(1 2 3))) 3)
(second (rest '(a b c)))
(rest (rest '(1 (2))))
(car (cddr '(a b c)))
(nth (second '(a 2)) '(1 2 3 4))
(nthcdr 1 (cons 1 '(2 3)))
(cadar '(((1) (2) (3)) (4 (5))))

;; порівняння об'єктів
(eq '(1 2) '(1 2))
(eq 3 (cdr '(1 3)))
(eq 3 (cdr (cons 1 3)))
(eql 2 (/ 6.0 3.0))
(eql 2 (/ 6 3))

;; логічні операції
(and (or (numberp nil) (atom 2)) (list 'geek))
(or (not (> (length '(1 2 3) 4))) (not (listp (rest '(a)))))

;; операції зі списками
(append (list '(1)) (cons 'a nil))
(cons (length (cdr '(1 2 3))) (last '(4 5)))
```

Тема 2.2 Символи та їх властивості

Символ в Common Lisp — це не просто якийсь ідентифікатор, що позначає деяке значення або функцію. Це структура, яка має певні поля (про що згадувалось в р. 2.1). Поля цієї структури містять значення, які використовуються інтерпретатором Lisp при виконанні програм неявно для програміста. І хоча в простих програмах не часто доводиться явно звертатись до значень полів символів, варто розглянути які вони є, щоб мати уявлення як реалізовані ті чи інші функції в Common Lisp (наприклад, як символ може бути одночасно змінною і функцією).

2.2.1 Структура символу

Структура, якою є символ, не має якоїсь специфічної назви, тому для простоти назовемо її **символьною структурою**. Символьна структура складається з таких полів (в дужках зазначаються функції, що дають змогу отримати значення відповідних полів):

- ім'я (symbol-name)
- пакет (symbol-package)
- значення (symbol-value)
- функція (symbol-function)
- список атрибутів (symbol-plist)

Ім'я символу — це рядок з ім'ям символу. Ім'я надається символу під час його створення, тобто визначення в деякій формі (як `let`), або ж використання (коли інтерпретатор його зчитує у формі).

```
CL-USER> (defparameter *a* 100)
*a*
CL-USER> (symbol-name '*a*)
"*a*"
```

Пакет символу — це об'єкт пакету, якому належить символ. Пакет також автоматично визначається для символу під час створення і відповідає тому пакету, в якому символ, власне, створюється інтерпретатором.

```
CL-USER> (symbol-package '*a*) ; змінна з попереднього прикладу
#<PACKAGE "COMMON-LISP-USER"> ; об'єкт пакету
```

Значення символу — це значення, яке встановлене у символ (фактичне значення формального параметра, зв'язане у формі зв'язування значення (напр. let), або ж значення, встановлене формами для встановлення значень (напр. setq))

```
CL-USER> (setq a '(1 2 3))
(1 2 3)
CL-USER> a
(1 2 3)
CL-USER> (symbol-value 'a)
(1 2 3)
```

Функція символу — це функціональний об'єкт (лямбда-вираз), пов'язаний із цим символом. Якщо у символу є значення в полі функції, можна сказати, що символ є іменованою функцією.

```
CL-USER> (defun foo () 1) ; якась проста функція
FOO
CL-USER> (symbol-function 'foo)
#<FUNCTION FOO> ; об'єкт функції
```

Список атрибутів символу — це список пар ключ-значення, що відіграють роль атрибутів або властивостей символу. Формат списку атрибутів — plist (див. р. 2.3.2.1). При визначенні символу як змінної чи константи, список його атрибутів пустий. Проте список можна редагувати під час виконання програми.

Для перевірки того, чи зв'язане із символом певне значення або функція використовуються предикати BOUNDP та FBOUNDP відповідно:

```
CL-USER> (boundp '*a*')
T
CL-USER> (boundp 'b')
NIL
CL-USER> (fboundp 'foo')
#<FUNCTION FOO>
CL-USER> (fboundp 'bar')
NIL
```

Якщо із символом не пов'язане значення або функція, тоді спроба отримати значення полів `symbol-value` і `symbol-function` призведе до викидання помилки. Тому перш ніж намагатись явно отримати значення цих полів, варто перевірити, чи вони зв'язані, використавши відповідний предикат.

2.2.1.1 Встановлення значень полів символічних структур

Деякі поля символічної структури встановлюються неявно під час виконання програми — як і було зазначено раніше. Наприклад, коли символ визначається (з `defvar`, у формі `let`, тощо), то йому одразу встановлюється значення двох полів — ім'я та пакету. Якщо це форма, яка не лише визначає символ, а ще й встановлює йому значення (`defparameter`, `let`), тоді також встановлюється значення і поля значення.

Такі форми, як `defun`, `defmacro` та деякі інші встановлюють значення полів імені, пакету та функції символічної структури.

2.2.1.2 Встановлення значень полів символічних структур з SETF

Для того, щоб встановити значення в якесь поле символічної структури, можна також використати макрос SETF (SET Field). Цей макрос працює як псевдофункція і може бути використаний для встановлення не лише полів символічної структури, але й для встановлення значень у будь-які інші структури,

змінні, спискові комірки, хеш-таблиці тощо. Детальніше його можливості розглядаються в розділі 2.3.3.1.

Наприклад, можна явно встановити якесь значення або функцію у символ:

```
CL-USER> *a*
100
CL-USER> (setf (symbol-value '*a*) 2)
2
CL-USER> *a*
2
CL-USER> (foo)
1
CL-USER> (setf (symbol-function 'foo) (lambda () 2))
#<FUNCTION (LAMBDA ()) {5365A32B}>
CL-USER> (foo)
2
```

Хоча встановлення значень таким чином виглядає складніше, ніж, наприклад, з SETQ:

```
CL-USER> (setq *a* 3)
3
CL-USER> *a*
3
```

2.2.1.3 Символи зі значенням і функцією

Маючи можливість явно або неявно визначати символи як змінні та функції, можна досягти стану, коли один символ одразу є і тим і іншим. Тобто, символ одночасно має значення в полях значення і функції:

```
CL-USER> (defun my-list (x y)
           (cons x (cons y nil)))
MY-LIST
CL-USER> (set 'my-list '(1))
(1)
CL-USER> (symbol-value 'my-list)
(1)
CL-USER> (symbol-function 'my-list)
#<FUNCTION MY-LIST>
CL-USER> (my-list my-list 2)
((1) 2)
```

Значення якого конкретно поля структури використовувати — чи то `symbol-value`, чи то `symbol-function` — визначає інтерпретатор в залежності від того, де в списку в формі знаходиться символ. Якщо на першій позиції в списку у формі — тоді інтерпретатор використовує функціональний об'єкт, встановлений в поле `symbol-function`. Якщо ж символ не є першим елементом списку у формі, або ж не є частиною списку, тоді інтерпретатор використовує значення поля `symbol-value`.

2.2.1.4 Редагування списку атрибутів символа

Для читання та/або редагування списку атрибутів символа використовується функція `GET`. Вона приймає на вхід два параметра: символ і ім'я (ключ) атрибута, значення якого необхідно отримати. Ключ може бути довільним атомом. Зазвичай використовують символи. Для встановлення значення певного атрибута, `GET` використовується разом з `SETQ`:

```
CL-USER> (defvar *x* nil)
*x*
CL-USER> *x*
NIL
CL-USER> (setf (get '*x* 'color) "red")
"red"
CL-USER> (get '*x* 'color)
"red"
CL-USER> (setf (get '*x* 'border) "bold")
"bold"
CL-USER> (symbol-plist '*x*)
(BORDER "bold" COLOR "red")
CL-USER> (setf (get '*x* 'border) nil)
NIL
CL-USER> (symbol-plist '*x*)
(BORDER NIL COLOR "red")
```

Для видалення атрибута символа використовується функція `REMPROP`, яка приймає на вхід символ і ім'я (ключ) атрибута, який необхідно видалити.

```
CL-USER> (remprop '*x*' 'border)
(BORDER NIL COLOR "red")
CL-USER> (symbol-plist '*x*')
(COLOR "red")
```

Оновлення атрибута з GET або видалення атрибута з REMPROP відбувається лише для першого знайденого входження ключа. Тож якщо в списку атрибутів є два атрибути з однаковим ім'ям, тоді зміниться або видалиться лише перший з них.

2.2.2 Ключові символи

Особливим видом символів є *ключові символи*. **Ключові символи** — це символи, що належать пакету KEYWORD. Особливими вони є тому, що такі символи записуються з двокрапкою перед іменем, завдяки чому інтерпретатор при їх зчитуванні завжди визначає їх як належні пакету KEYWORD — незалежно від того, де вони використовуються. Також їхньою особливістю є те, що вони є константами, які позначають самі себе (як T чи NIL):

```
CL-USER> :foo
:FOO
CL-USER> (symbol-package :foo)
#<PACKAGE "KEYWORD">
CL-USER> (symbol-name :foo)
"FOO"
CL-USER> (symbol-value :foo)
:FOO
```

Ключові символи використовуються для передачі ключових параметрів у функції (див. р. 2.4.2.2). А також часто як ключі в хеш-таблицях або просто для позначення унікальних значень, що точно однакові в різних частинах великих програм (що часто поділені на пакети, кожен з яких має власний простір імен символів), оскільки вони позначають самі себе і завжди належать пакету KEYWORD — тобто такі символи легко порівнювати один з одним не боячись, що вони

можуть мати однакове ім'я, але бути з різних пакетів (і, відповідно, не "одним і тим же об'єктом" по eq — тобто в пам'яті).

Тема 2.3 Списки та їх будова

Визначення списку і його базовий опис були розглянуті в другому розділі. Цей розділ має на меті пояснити внутрішню будову списків, а також їхні види.

2.3.1 Внутрішнє представлення списків

Наведені в попередньому розділі функції, що конструюють списки, завжди створюють нові списки, таким чином реалізуючи *конструктивний* підхід до створення (розширення) списків. Проте також в Common Lisp є функції, які не створюють нові списки, а змінюють наявні, реалізуючи *деструктивний* або ж *руйнівний* підхід. Щоб краще зрозуміти обидва підходи, варто розглянути внутрішню будову списків, а також те, що відбувається з ними при використанні конструктивних або деструктивних функцій [32].

2.3.1.1 Спискові комірки та точкові пари

Списки в Common Lisp є однозв'язними списками, що складаються з так званих *спискових комірок* (або *cons*) — простих структур, що містять лише два поля: CAR та CDR (рис. 2.2).

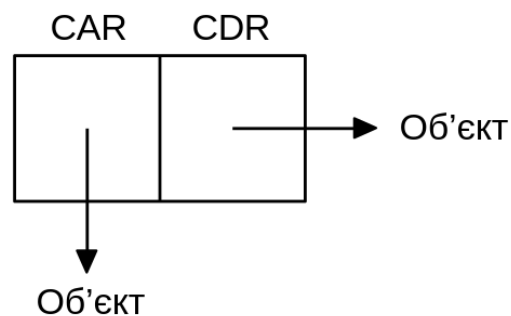


Рисунок 2.2 — Спискова комірка

Поля CAR та CDR містять посилання на деякі об'єкти, доступ до яких отримується завдяки однойменним функціям CAR та CDR. Якщо поле CDR спискової комірки містить посилання не на список (в т. ч. не на NIL), а на якийсь інший об'єкт, тоді така спискова комірка називається *точковою парою*. Інакше — списком (рис. 2.3).

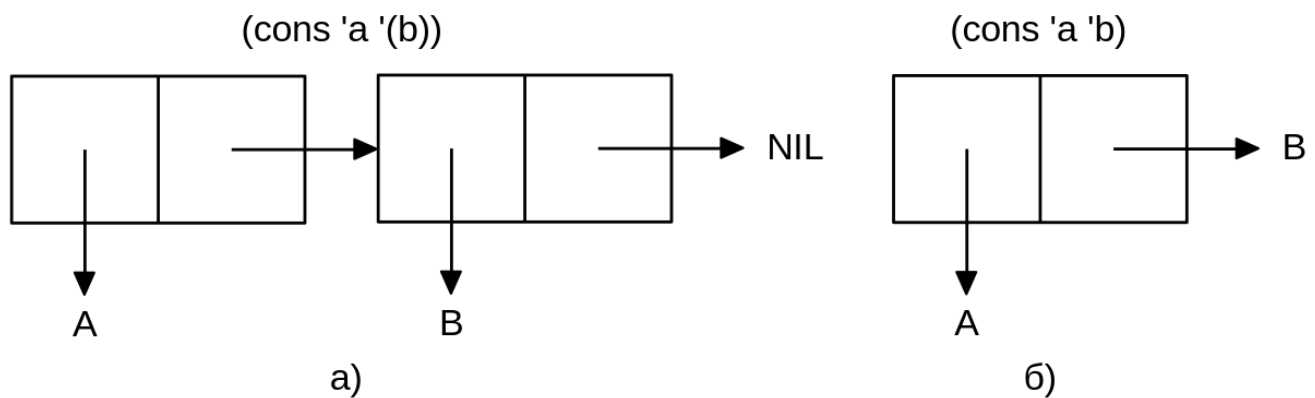


Рисунок 2.3 — Список (а) і точкова пара (б)

Ще один приклад, який може бути неочевидним: якщо отримати посилання на якусь хвостову частину списку, а тоді за допомогою CONS додати цей "хвіст" у голову, то буде створена лише одна спискова комірка, а не багато, як може здатись, якщо подивитись на вивід списку у консоль (рис. 2.4).

2.3.1.2 Конструктивний підхід

Функції CONS, LIST, APPEND та інші під час свого виконання створюють нові спискові комірки, *конструюючи* нові списки. Це забезпечує виконання другого принципу функційного програмування, наведеного в розділі 1.2, з усіма його перевагами.

```
CL-USER> (set 'lst '(a b c))
(A B C)
CL-USER> (cons (rest lst) lst)
((B C) A B C)
```

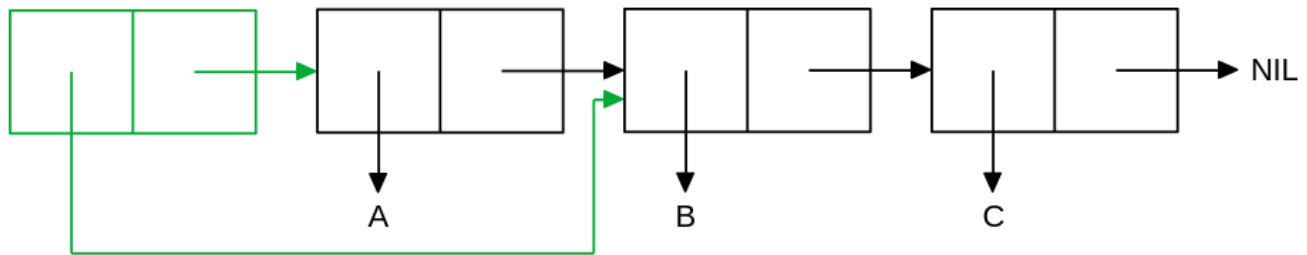


Рисунок 2.4 — Структура списку в пам'яті і при друку у консоль при додаванні "хвоста" в "голову"

В якості прикладу розглянемо роботу функції APPEND для трьох списків (рис. 2.5). APPEND копіює перших два списки повністю, створюючи нові спискові комірки, і тоді "зв'язує" копію першого списку з копією другого, а копію другого списку — з третім списком (оригіналом). При цьому змінюється значення поля CDR в останній списковій комірці скопійованих першого та другого списків. Останній список не копіюється, оскільки жодна з його спискових комірок не потребує змін.

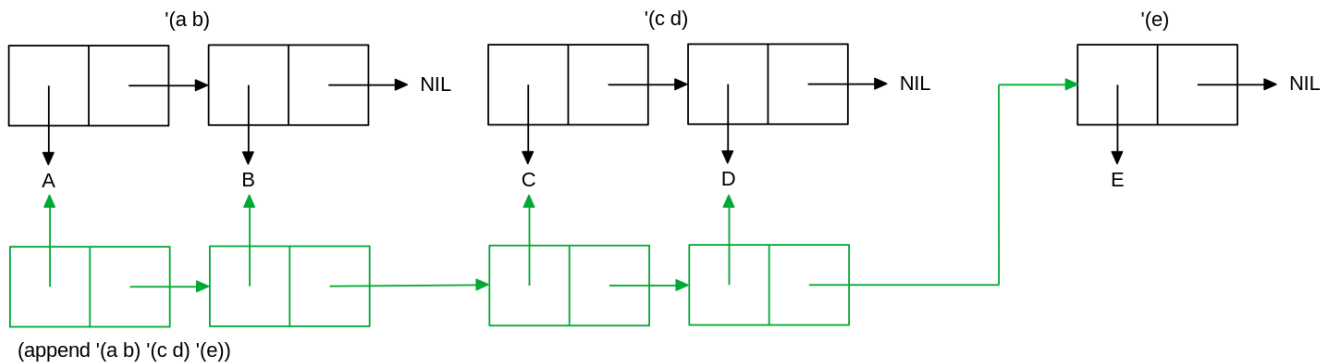


Рисунок 2.5 — Виконання APPEND для трьох списків

2.3.1.3 Деструктивний підхід

На противагу конструктивним функціям, є й функції, що замість створення нових спискових комірок змінюють вміст полів наявних спискових комірок, тобто виконують зміни списків *деструктивно*. Звісно, це порушує (або може порушити) принципи функціонального програмування, розглянуті вище. Проте очевидною перевагою такого підходу є те, що не відбувається виділення пам'яті для нових комірок, що не лише зменшує обсяг використаної пам'яті, а й забезпечує більшу швидкість виконання.

Простим прикладом руйнівних функцій є функції RPLACA, RPLACD та NCONC. Перші дві виконують встановлення значення у поля CAR та CDR спискової комірки відповідно.

```
CL-USER> (setq a '(1 2 3))
(1 2 3)
CL-USER> (rplaca a 4)
(4 2 3)
CL-USER> (rplacd a '(5))
(4 5)
CL-USER> (rplacd (rest a) '(6))
(5 6)
CL-USER> a
(4 5 6)
```

Функція `NCONC` виконує конкатенацію списків без їхнього копіювання, а за рахунок заміни значення в полі `CDR` останніх спискових комірок усіх списків-аргументів (крім останнього) на посилання на першу спискову комірку наступного списку (рис. 2.6).

Загалом, щоб не порушувати перші два принципи функціонального програмування, руйнівні функції мають використовуватись лише для зміни "внутрішніх" даних користувацьких функцій. Наприклад, якщо функція, яку написав програміст, спочатку створює якісь списки (тобто, це "нові" об'єкти), а потім змінює їх (об'єднує, фільтрує, тощо) — тоді можна використати деструктивну функцію на "нових" даних, оскільки вона не вплине на об'єкти за межами написаної функції.

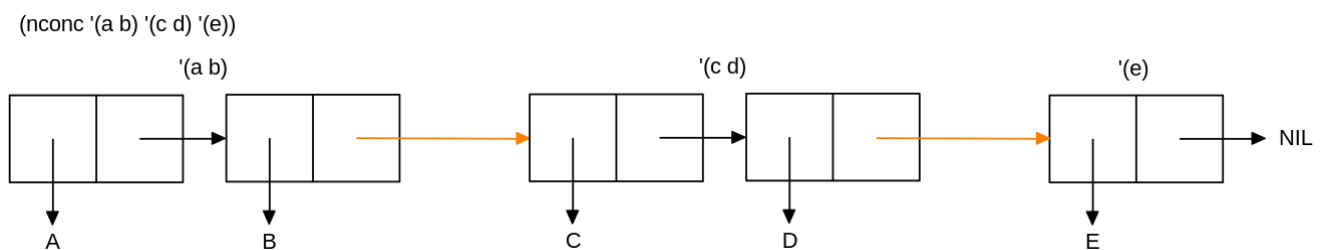


Рисунок 2.6 — Виконання `NCONC` для трьох списків

ВАЖЛИВО: надалі при виконанні завдань протягом курсу (в т. ч. лабораторних робіт) варто в першу чергу застосовувати конструктивний підхід. Варіант рішення із застосуванням деструктивного підходу може бути доданий в якості альтернативного рішення з обов'язковим поясненням того, чому деструктивний підхід в кожному окремому випадку є безпечним — тобто що деструктивна зміна не змінює стан "зовнішніх" об'єктів.

2.3.2 Види списків

Довільний список — це просто послідовність спискових комірок. Проте якщо елементи списку якимось чином впорядковані, або між ними є деяка закономірність, тоді можна говорити про певний *формат* списків.

Розрізняють два формати або види списків, що використовуються як таблиці пошуку (lookup tables) [41], тобто для пошуку значення за певним ключем:

- `plist` (property list) — список з парною кількістю елементів, де елементи на непарних позиціях є ключами, а на парних — значеннями
- `alist` (association list) — список, кожен елемент якого — це точкова пара, в комірці `CAR` якої знаходиться ключ, а в комірці `CDR` — пов'язане з ключем значення

2.3.2.1 Список атрибутів (*plist*)

Список атрибутів (`plist`, `property list`) використовується для збереження атрибутів символів, хоча такий формат можна використовувати і будь-яких списків. Формат передбачає, що список є парною послідовністю елементів, де в кожній парі перший елемент є *ключем* або *індикатором*, а другий — значенням. Наприклад, ось список, що містить дві пари індикатор-значення:

```
(a 5 b 10)
```

Для роботи з таким форматом списків використовуються функції `GET` та `GETF`:

```
(defun get (symbol indicator &optional default) ...)  
(defun getf (place indicator &optional default) ...)
```

`symbol` — це символ, зі списку атрибутів якого необхідно отримати значення, ключем якого є `indicator.place` — це "місце" (див. р. 2.3.3.1) списку атрибутів. `default` — це значення, яке поверне функція, якщо шуканого значення за ключем не знайдено. (&optional позначає «опційні» аргументи, які детальніше розглядаються в р. 2.4.2.1.)

GET може бути застосований лише до символів, в той час як GETF — до довільного списку атрибутів: `(get 'lst 'a)` може бути записаний як `(get (symbol-plist 'lst) 'a)`.

Для встановлення значення на певним ключем/індикатором у списку атрибутів можна використати функції, що працюють з "місцями", наприклад SETF.

Приклади:

```
CL-USER> (symbol-plist 'lst)
NIL
CL-USER> (setf (get 'lst 'a) 1)
1
CL-USER> (symbol-plist 'lst)
(A 1)
CL-USER> (get 'lst 'a)
1
CL-USER> (get 'lst 'b 2)
2
CL-USER> (getf (symbol-plist 'lst) 'a 2)
1
CL-USER> (set 'lst '(a 1 b 2))
(A 1 B 2)
CL-USER> (setf (getf lst 'b) 10)
10
CL-USER> lst
(A 1 B 10)
```

2.3.3 Деструктивні операції

Як і було зазначено раніше, в Common Lisp крім *чистих* функцій, які під час виконання не створюють побічних ефектів, є також і *псевдофункції* — функції, які

крім повернення результату також змінюють стан якихось об'єктів (тобто, крім основної "дії" мають і "побічну дію"). До них відносяться розглянуті раніше SET, SETQ, RPLACA, RPLACD, NCONC.

Як і було зазначено в розділі 2.3.1.3, використання таких функції порушує (або може порушити) принципи функціонального програмування, які стверджують, що функції не мають змінювати стан "зовнішніх" об'єктів або ж працювати зі змінюваними об'єктами. Проте створення щоразу нових об'єктів може вимагати значних обсягів пам'яті, що може призвести як до неефективного використання пам'яті, а так низької швидкодії реалізованих програм. Тому для оптимізації обчислень при використанні функціонального підходу іноді є сенс застосовувати і псевдофункції, хоча й з певними обмеженнями.

Можливості такої оптимізації розглядатимуться в одному з наступних розділів. А в цьому розділі увага сконцентована на огляді псевдофункцій.

2.3.3.1 Універсальний макрос SETF

Макрос SETF (SET Field — "встановити поле") є універсальним макросом для встановлення значень у поля структур. Він використовується для встановлення значень символів, функцій символів, зміни значень комірок CAR та CDR спискових комірок, зміни значень полів структур (тип даних), встановлення значень за ключем у таблиці, зміни значень полів об'єктів класів (в CLOS), тощо.

Якщо ж узагальнити, то SETF може встановлювати значення у "місцях" (places) — тобто за посиланням на місце, де зберігаються об'єкти [39]. SETF — це лише один з операторів, які можуть працювати з "місцями". "Місце" може бути символом або формою, яка отримує значення з поля деякої структури:

```
place := symbol | <getter-form> ; "місце"  
<getter-form> := form ; форма доступу до певного поля структури
```

При використанні SETF для встановлення значення символу, він працює аналогічно SETQ. І також аналогічно SETQ SETF може мати довільну парну кількість аргументів, де кожна послідовна пара визначає "куди" треба встановити і "що" треба встановити:

```
CL-USER> (setf x 1) ; те саме, що й (setq x 1) або (set 'x 1)
1
CL-USER> (setf y 2 z 3)
3
CL-USER> (list x y z)
(1 2 3)
```

Якщо ж необхідно встановити значення поля деякої структури (наприклад, явно встановити значення поля символічної структури, або спискової комірки, абощо), тоді використовується така форма запису: (SETF <getter-form> <value>), де <value> — це форма значення, а <getter-form> — це форма, за допомогою якої можна отримати те значення, яке необхідно замінити на інше (встановити нове значення). Тобто, це є вказівка (посилання) на "місце" (поле), значення в якому необхідно змінити. Наприклад:

```
CL-USER> (setf x (cons nil nil))
(NIL)
CL-USER> (setf (car x) 1) ; == (rplaca x 1)
1
CL-USER> (setf (cdr x) 2) ; == (rplacd x 2)
2
CL-USER> x
(1 . 2) ; точкова пара
```

Можна також використовувати SETF і для інших <getter-form>, що до того ж можуть бути композиціями викликів:

```
CL-USER> (setq lst '(1 2 3 4 5) lst-backup lst)
(1 2 3 4 5)
CL-USER> (setf (first lst) 6)
6
```

```
CL-USER> (setf (car (nthcdr 2 lst)) 7)
7
CL-USER> (setf (cddddr lst) '(8 9))
(8 9)
CL-USER> (setf (nth 5 lst) 10)
10
CL-USER> lst
(6 2 7 4 8 10)
CL-USER> (eq lst lst-backup)
T
```

Це лише частина можливих "місць", які можна змінити з SETF. В той же час для деяких функцій таке не спрацює. Наприклад, не можна встановити значення LAST деякого списку.

Застосування SETF для зміни значень в структурах, геш-таблицях та об'єктах розглядатиметься у відповідних розділах.

2.3.3.2 Інші оператори, що оперують "місцями"

Крім вже розглянутих в розділі 2.1.10 SET, SETQ; в розділі 2.3.1.3 RPLACA, RPLACD і NCONC; та в розділі 2.3.3.1 SETF також є й інші форми для деструктивної зміни об'єктів, зокрема такі, які так само оперують "місцями", як це робить SETF.

2.3.3.3 INCF та DECF

Для зміни значення лічильників можна використати макроси INCF та DECF. Їх формат запису такий:

```
(incf place [<delta-form>])
(decf place [<delta-form>])
<delta-form> := form ; значення, на яке необхідно змінити число
```

Приклад застосування:

```
CL-USER> (setq a 0)
0
CL-USER> (incf a)
1
CL-USER> (incf a 4)
```

```

5
CL-USER> (decf a 5)
0
CL-USER> a
0
CL-USER> (setq b '(1 2 3))
(1 2 3)
CL-USER> (incf (second b) 10)
12
CL-USER> b
(1 12 3)

```

2.3.3.4 PUSH та POP

Макроси PUSH і POP використовуються для роботи зі списком як зі стеком: вони виконують додавання елемента до голови списку (push) і викидання елемента з голови списку (pop), при цьому змінюючи посилання на список, оскільки змінюється його голова. Формат їхнього запису такий:

```

(push place <value-form>)
(pop place)
<value-form> := form ; значення, яке необхідно додати в голову стеку

```

Результатом виконання PUSH є новий список (тобто старий з доданим у голову новим елементом), а результатом POP є значення голови списку, яка була викинута з нього.

Приклад застосування:

```

CL-USER> (setq lst '(3))
(3)
CL-USER> (push 2 lst)
(2 3)
CL-USER> (push 1 lst)
(1 2 3)
CL-USER> (pop lst)
1
CL-USER> lst
(2 3)

```

2.3.3.5 ROTATEF

Макрос ROTATEF виконує зміну значень в "місцях", зазначених в якості його аргументів, виконуючи ротацію значень між ними справа-наліво. Формат його запису такий:

```
(rotatef place1 place2 ... placeN)
```

Приклад застосування:

```
CL-USER> (let ((x 1)
               (lst '(a b c d e f)))
           (rotatef x (nth 1 lst) (nth 5 lst))
           (list x lst))
(B (A F C D E 1))
```

2.3.4 Функції для роботи зі списками

Базові функції для роботи зі списками розглядаються в розділі 2.1. Більшість інших функцій, що також використовуються для роботи зі списками, визначені для *послідовностей* — типу, що є батьківським типом для списків, і розглядаються в розділі 5.1. Частина тих функцій мають відповідники для роботи саме зі списками, а не послідовностями, інші ж — не мають. Проте також є функції, які визначені лише для списків (напр. MEMBER, MEMBER-IF та MEMBER-IF-NOT).

2.3.4.1 Функції пошуку елемента MEMBER, MEMBER-IF та MEMBER-IF-NOT

MEMBER виконує пошук заданого елемента у списку і повертає спискову комірку (список), в голові якого знаходиться шуканий елемент. Якщо елемент не знайдено — повертається NIL. Синтаксис функції наступний:

```
(defun member (item list &key key test test-not) ...)
```

`item` — шуканий елемент, `list` — список, в якому елемент шукається. Ключові параметри мають таке саме призначення, як і для функцій роботи з послідовностями. Порівняння `item` з елементами `list` за замовчуванням відбувається функцією `eq1`.

Приклад використання `MEMBER` без ключових параметрів:

```
CL-USER> (member 'b '(a b c))
(B C)
CL-USER> (member 1 '(0 2 3))
NIL
```

Приклад використання з ключовими параметрами:

```
CL-USER> (member 'a '((a 1) (b 2)) :key #'car)
((A 1) (B 2))
CL-USER> (member 'b '((a 1) (b 2)) :key #'car)
((B 2))
CL-USER> (member '(b 2) '((a 1) (b 2)) :test #'equal)
((B 2))
CL-USER> (member '(b 2) '((a 1) (b 2)) :test-not #'equal)
((A 1) (B 2))
CL-USER> (member '(b 2) '((b 2) (b 2)) :test-not #'equal)
NIL
```

Для пошуку з використанням предикату замість об'єкту використовуються варіанти `MEMBER` — функції `MEMBER-IF` та `MEMBER-IF-NOT`. Ці функції не порівнюють елементи списку із шуканим, а передають елементи списку у функцію-предикат, що є їхнім першим аргументом. Якщо функція-предикат повертає не-`NIL` у `MEMBER-IF` або `NIL` у `MEMBER-IF-NOT`, тоді відповідна функція повертає список, в голові якого знаходиться "знайдений" елемент. Синтаксис їхнього запису такий:

```
(defun member-if (test list &key key) ...)
(defun member-if-not (test list &key key) ...)
```

Приклади:

```
CL-USER> (member-if #'oddp '(2 4 6))
NIL
CL-USER> (member-if #'oddp '(2 3 4))
(3 4)
CL-USER> (member-if-not #'oddp '(2 3 4))
(2 3 4)
CL-USER> (member-if-not #'oddp '(2 4 6) :key #'1+)
NIL
```

2.3.4.2 Функції конструктивного складання списків MAPCAR та MAPLIST

Функції MAPCAR та MAPLIST виконують **складання** або **зіставлення** одного або багатьох списків (mapping). Їхній синтаксис такий:

```
(defun mapcar (function list &rest more-lists) ...)
(defun maplist (function list &rest more-lists) ...)
```

`function` — це функція, яка приймає на вхід відповідні елементи вхідних списків (`list`, `more-lists` (&rest параметри розглядаються в р. 2.4.2.3.)) у випадку MAPCAR або відповідні частини/хвости вхідних списків (аналогічно тому, як працює NTHCDR) у випадку MAPLIST, виконує з ними якусь дію і повертає один елемент. З отриманих нових елементів формується *новий* список. Тобто, `function` має приймати стільки аргументів, скільки списків отримує на вхід MAPCAR або MAPLIST. Якщо кількість елементів у вхідних списках відрізняється, тоді обхід елементів списків (і формування нового списку) завершиться після досягнення останнього елементу найкоротшого з вхідних списків.

Приклади застосування:

```
CL-USER> (mapcar #'1+ '(1 2 3))
(2 3 4)
CL-USER> (mapcar #'1+ '(1 2 3) '(4 5 6) '(7 8 9))
```

```
(12 15 18)
(maplist #'identity '(1 2 3))
((1 2 3) (2 3) (3))
CL-USER> (maplist #'list '(1 2 3) '(2 3 4))
(((1 2 3) (2 3 4)) ((2 3) (3 4)) ((3) (4)))
CL-USER> (maplist #'list '(1 2 3) '(2 3))
(((1 2 3) (2 3)) ((2 3) (3)))
```

Реалізація функції COPY-LIST з MAPCAR виглядає ось так:

```
(defun my-copy-list (lst)
  (mapcar #'identity lst))
```

Функція IDENTITY просто повертає свій вхідний аргумент (див. р. 4.1.3).

2.3.4.3 Функції обходу списків MAPC та MAPL

Для того, щоб обійти елементи списку, або відповідні елементи багатьох списків, і при цьому не конструювати новий список, використовуються функції MAPC та MAPL. Їхній інтерфейс та формат застосування повністю аналогічний функціям MAPCAR і MAPLIST, а різниця полягає лише у тому, що вони не конструюють новий список, а повертають (перший) вхідний список. Такі функції краще використовувати у випадках, коли необхідно обійти елементи списку(-ів), зібрати та/або вивести якусь інформацію, але потреби конструювати новий список (наприклад, з новими значеннями) немає:

```
CL-USER> (mapc (lambda (elem) (format t "> element: ~A~%" elem)) '(1 2 3))
> element: 1
> element: 2
> element: 3
(1 2 3)
CL-USER> (mapl (lambda (numbers symbols)
  (format t "> numbers: ~A~%> symbols: ~A~%" numbers symbols))
  '(1 2 3)
  '(a b c))
> numbers: (1 2 3)
> symbols: (A B C)
> numbers: (2 3)
> symbols: (B C)
> numbers: (3)
```

```
> symbols: (C)
(1 2 3)
```

2.3.4.4 Функції деструктивного складання списків MAPCAN та MAPCON

Функції MAPCAN та MAPCON мають такий самий інтерфейс, як і функції MAPCAR та MAPLIST, та й формат їх застосування такий самий (параметри, що передаються в function). Проте замість того, щоб створити список для результатів виклику function, вони, по-перше, очікують, що function поверне списки і, по-друге, виконують деструктивне об'єднання цих списків з NCONC:

```
(mapcan #'list '(a b c))
<==>
(apply #'nconc (mapcar #'list '(a b c)))
```

Приклади застосування:

```
CL-USER> (mapcan (lambda (x) (and (numberp x) (list x)))
                '(a 1 b c 3 4 d 5))
(1 3 4 5)
CL-USER> (set 'lst '((1) (2) (3)))
((1) (2) (3))
CL-USER> (mapcan #'identity lst)
(1 2 3)
CL-USER> lst
((1 2 3) (2 3) (3))
```

Тема 2.4 Визначення функції

Визначення функції і її обчислення в Lisp базуються на лямбда-обчисленнях Алонзо Черча. Лямбда-обчислення (λ -calculus) — це формальна система математичної логіки, що розглядає такі поняття як функція, прикладення функції до аргументів, зв'язування аргументів та ін. Лямбда-обчислення є еквівалентом машини Тюрінга [33].

В Common Lisp функції реалізовані у вигляді **функціональних об'єктів**, тобто об'єктів, які можуть бути викликані — так, ніби це звичайні функції. Визначити функціональний об'єкт можна у два способи:

- визначити неіменований/анонімний функціональний об'єкт або ж *лямбда-вираз*
- визначити іменований функціональний об'єкт або ж просто функцію

Застосування функцій як об'єктів розглядається більш детально в розділі 4.1.

2.4.1 Іменовані функції

Завжди використовувати неіменовані функції для визначення поведінки програм — так, як було показано у попередньому розділі, — не зручно. До незручностей можна віднести такі випадки, коли одну і ту ж функцію необхідно викликати кілька разів з різними фактичними значеннями, в різних місцях програми (тобто, у випадку коли необхідно зробити кілька визначень одного і того ж лямбда-виразу), або ж просто зробити рекурсивний виклик. Значно зручніше визначити функцію один раз, надати їй ім'я і викликати по імені там, де це необхідно.

Для визначення іменованих функцій використовується макрос DEFUN, який створює лямбда-вираз і зв'язує його з певним ім'ям (символом) — тобто іменує його [36]. Синтаксис його такий:

```
(defun <name> (<args>) <documentation>? <body>)  
<name> := symbol ; назва функції  
<args> := s-expr* ; формальні аргументи функції  
<documentation>? := string ; рядок документування — за потреби додається як  
; перша форма тіла функції  
<body> := s-expr* ; тіло функції, що складається з форм
```

Приклади визначень функцій:

```

(defun print-hello ()
  (print "Hello!"))

(defun my-list (x y)
  "Create list out of two input arguments"
  (cons x (cons y nil)))

(defun my-second (lst)
  "Get second element of a list"
  (car (cdr lst)))

(defun my-double (n)
  "Double value of `n'"
  (* n 2))

(defun negate (n)
  "Negate input number"
  (- n))

(defun square (n)
  "Get square of a number"
  (* n n))

```

Викликається іменована функція аналогічно лямбда-виразам: виклик є списком, першим елементом якого є назва функції (символ замість лямбда-виразу), а далі знаходяться фактичні аргументи:

```

CL-USER> (negate 1)
-1
CL-USER> (my-double 2)
4
CL-USER> (square 3)
9
CL-USER> (my-second '(1 2 3))
2

```

2.4.2 Спеціальні параметри функцій

Крім обов'язкових параметрів, функція може мати *опційні*, *ключові*, *решту* та *допоміжні* параметри. Разом всі ці види параметрів називаються **спеціальними параметрами** функцій.

2.4.2.1 Опційні параметри

Опційні (або також **опціональні**) параметри функції — це параметри, які не обов'язково визначати при виклику функції. Якщо при виклику функції вони не зазначені, тоді формальний параметри отримає значення за замовчуванням, яке визначене для опційного параметра у визначенні функції.

Визначення опційних параметрів у списку параметрів в визначенні функції починається з символу `&optional`, за яким знаходяться, власне, визначення опційних параметрів, яких може бути кілька:

```
&optional <opt-param-clause>+  
<opt-param-clause> := <param> ; перший формат  
                    | (<param> <default-value>) ; другий формат  
                    | (<param> <default-value> <set-p>) ; третій формат  
<param> := symbol ; ім'я опційного параметра  
<default-value> := form ; значення за замовчуванням  
<set-p> := symbol ; індикатор того, чи було передане фактичне значення  
                ; опційного параметра
```

В першому форматі запису визначається лише ім'я опційного параметра, тому в разі, якщо фактичне значення не передається у функцію під час виклику, значенням параметру стає `NIL`. `<set-p>` встановлюється у `T` або `NIL` в залежності від того, чи передано фактичне значення для опційного параметра. Порядок зазначення передачі значень для опційних параметрів при виклику функції має збігатись з порядком їх визначення у визначенні функції.

Приклад:

```
(defun foo (arg &optional opt1 (opt2 42) (opt3 nil opt3-set-p))  
  (list arg opt1 opt2 opt3 opt3-set-p))
```

Виклики `foo` з різними аргументами:

```
CL-USER> (foo 1)
```

```

(1 NIL 42 NIL NIL)
CL-USER> (foo 2 t)
(2 T 42 NIL NIL)
CL-USER> (foo 3 t 21)
(3 T 21 NIL NIL)
CL-USER> (foo 4 t 21 '(9))
(4 T 21 (9) T)
CL-USER> (foo 5 t 21 nil)
(5 T 21 NIL T)

```

2.4.2.2 Ключові параметри

Ключові параметри передаються у функцію за ім'ям параметра, що зазначається у виклику функції ключовим символом (див. р. 2.2.2). Аналогічно до опційних параметрів, якщо фактичне значення для ключового параметра не передане у функцію під час виклику, ключовий параметр набуває *значення за замовчуванням*, визначеного у визначенні функції.

Ключові параметри визначаються після символу `&key` у визначенні функції, а формати визначення таких параметрів аналогічні форматам визначення опційних параметрів:

```

&key <key-param-clause>+
<key-param-clause> := <param> ; перший формат
                    | (<param> <default-value>) ; другий формат
                    | (<param> <default-value> <set-p>) ; третій формат
<param> := symbol ; ім'я ключового параметра
<default-value> := form ; значення за замовчуванням
<set-p> := symbol ; індикатор того, чи було передане фактичне значення
                    ; ключового параметра

```

Ще однією відмінністю ключових параметрів від опційних є те, що вони зазначаються в довільному порядку при виклику функції.

Приклад:

```

(defun foo (arg &key key1 (key2 42) (key3 nil key3-set-p))
  (list arg key1 key2 key3 key3-set-p))

```

Виклики `foo` з різними аргументами:

```
CL-USER> (foo 1)
(1 NIL 42 NIL NIL)
CL-USER> (foo 2 :key1 t)
(2 T 42 NIL NIL)
CL-USER> (foo 3 :key2 21)
(3 NIL 21 NIL NIL)
CL-USER> (foo 4 :key3 t)
(4 NIL 42 T T)
CL-USER> (foo 5 :key2 21 :key3 nil)
(5 NIL 21 NIL T)
CL-USER> (foo 6 :key3 t :key1 '(9) :key2 21)
(6 (9) 21 T T)
CL-USER> (foo 7 t t)
; Evaluation aborted on #<UNKNOWN-KEYWORD-ARGUMENT {1001BE1D93}>.
```

2.4.2.3 "Решта" параметрів

Решта — це параметр змінної довжини. В інших мовах програмування такий вид параметрів може називатись "varargs" (від "variable arguments"). Цей параметр приймає послідовність аргументів умовно довільної довжини, і створює з них один список, який і стає значенням формального параметра.

Решта визначається після символу `&rest` у визначенні функції:

```
&rest <param>
<param> := symbol ; ім'я решти
```

Приклад:

```
(defun foo (arg &rest args)
  (list arg args))
```

Виклики `foo` з різними аргументами:

```
CL-USER> (foo 1)
(1 NIL)
CL-USER> (foo 2 t)
```

```
(2 (T))
CL-USER> (foo 2 t nil 5 '(a b c))
(2 (T NIL 5 (A B C)))
```

Інший приклад: проста реалізації функції `my-list` (аналог стандартної `list`), з `&rest`:

```
CL-USER> (defun my-list (&rest args)
           args)
CL-USER> (my-list 1 2 3)
(1 2 3)
```

2.4.2.4 Допоміжні параметри

Допоміжні параметри функції — це, насправді, не зовсім "параметри функції", оскільки це є змінні, які можна визначити у списку параметрів функції, але не можна передати у функцію. Тому це скоріше "синтаксичний цукор" для визначення локальних змінних, ніж параметри функції.

Визначаються допоміжні параметри після символу `&aux` (від "auxiliary"):

```
&aux <aux-param-clause>+
<aux-param-clause> := <param> ; перший формат
                    | (<param> [<default-value>]) ; другий формат
<param> := symbol ; ім'я допоміжного параметра
<default-value> := form ; значення за замовчуванням
```

Приклад:

```
(defun foo (arg &aux aux1 (aux2 (list 1 2)))
  (list arg aux1 aux2))
```

Виклики ``foo`` з різними аргументами:

```
CL-USER> (foo 1)
(1 NIL (1 2))
CL-USER> (foo 2 2)
```

```
; Evaluation aborted on #<SB-INT:SIMPLE-PROGRAM-ERROR "invalid number of  
; arguments: ~S" {100271C793}>.
```

Таке визначення `foo` аналогічне наступному визначенню з `let*` (`let*` розглядається в р. 2.5.1):

```
(defun foo (arg)  
  (let* (aux1 (aux2 (list 1 2)))  
    (list arg aux1 aux2)))
```

2.4.2.5 Комбінування різних видів спеціальних параметрів

Маючи можливість визначати різні види спеціальних параметрів, виникає закономірне питання "а як передавати значення у випадку, коли в функції визначено кілька різних видів спеціальних параметрів?". Оскільки спеціальні параметри є необов'язковими, то не завжди може бути очевидним те, як саме відбудеться зв'язування значень з формальними параметрами, як, наприклад, у випадку, коли функція одночасно має ключові параметри і решту.

Є кілька можливих варіантів, в якому порядку можна комбінувати різні види спеціальних параметрів у визначенні функції:

1. Опційні і ключові параметри (комбінуються з попередженням):

```
CL-USER> (defun foo (&optional a &key b) (list a b))  
FOO  
CL-USER> (foo)  
(NIL NIL)  
CL-USER> (foo 1)  
(1 NIL)  
CL-USER> (foo 2 :b 3)  
(2 3)  
CL-USER> (foo :b 3)  
; Evaluation aborted on #<SB-INT:SIMPLE-PROGRAM-ERROR "odd number of &KEY ;  
arguments" {1002DC49A3}>.
```

2. Опційні параметри і решта:

```
CL-USER> (foo 1 2 3)  
(1 (2 3))  
CL-USER> (foo)  
(NIL NIL)
```

3. Решта і ключові параметри:

```
CL-USER> (defun foo (&rest args &key b) (list b args))
FOO
CL-USER> (foo 1 2)
; Evaluation aborted on #<UNKNOWN-KEYWORD-ARGUMENT {100373F5C3}>.
CL-USER> (foo :b 3)
(3 (:B 3))
CL-USER> (foo 1 2 :b 3)
; Evaluation aborted on #<UNKNOWN-KEYWORD-ARGUMENT {10038EEE43}>.
CL-USER> (foo :b 1 :b 2)
(1 (:B 1 :B 2))
```

4. Опційні, решта і ключові параметри (комбінуються з попередженням):

```
CL-USER> (defun foo (&optional a &rest args &key b) (list a b args))
FOO
CL-USER> (foo 1 2 3)
; Evaluation aborted on #<UNKNOWN-KEYWORD-ARGUMENT {1001AA3D53}>.
CL-USER> (foo :b 3)
; Evaluation aborted on #<SB-INT:SIMPLE-PROGRAM-ERROR "odd number of &KEY arguments" {1001D17E63}>.
CL-USER> (foo 1 :b 2)
(1 2 (:B 2))
```

Інший порядок визначення параметрів неможливий. При комбінуванні опційних і ключових параметрів компілятор видає попередження про таку комбінацію. Хоча сама комбінація і можлива, проте через ті вимоги, які вона створює до переданих аргументів, використовується вона рідко.

2.4.3 Локальні функції

Крім визначення глобальних іменованих функцій з DEFUN та лямбда-виразів, можна також визначити локальні іменовані функції — тобто функції, які можна використовувати лише в певному обмеженому (локальному) контексті. Для цього використовуються спеціальні оператори FLET та LABELS.

FLET можна розуміти як "Function LET". Тобто це форма для локального зв'язування функцій. Формат її запису такий:

```
(flet (<function-def>*) <body-form>*)
<function-def> := (<function-name> (<args-list>) <body-form>*)
<function-name> := symbol ; назва локальної функції
<args-list> := function args ; формальні параметри функції
<body-form> := form ; довільна форма тіла
```

Приклад використання:

```
CL-USER> (flet ((%inc (arg)
                  (1+ arg))

                (%print (arg)
                  (format t "argument> ~A~%" arg))))

(%print 1)
(%print (%inc 1)))
argument> 1
argument> 2
NIL
```

Щоб якось відрізнити локальні функції від не-локальних, варто іменувати їх з символом "%" на початку імені.

LABELS має повністю аналогічний формат запису, що і FLET. Відмінність першої форми від другої полягає у тому, що функції, визначені в LABELS, можуть викликати одна одну, в той час як функції, визначені у FLET, такої можливості не мають. Приклад використання:

```
CL-USER> (labels ((%inc (arg)
                     (1+ arg))

                   (%print (arg)
                     (format t "argument> ~A~%" arg))

                   (%print-inc (arg)
                     (%print (%inc arg))))

          (%print 1)
          (%print-inc 1))
argument> 1
argument> 2
NIL
```

2.4.4 "Багато значень" та робота з ними

Часом виникає необхідність повернути кілька результатів з функції. Наприклад, коли необхідно виконати ділення і отримати не лише частку, але й остачу. Або інший приклад — одне з типових завдань на роботу з рекурсією:

необхідно рекурсивно обчислити суму або добуток ряду на поверненні з рекурсії, коли з рекурсивного виклику необхідно повертати не лише поточний результат всього обчислення, але й значення попереднього елемента ряду, яке використовується для обчислення поточного елемента.

Типовим підходом до повернення кількох значень з функції є повернення лише одного значення і збереження інших у глобальних (для функції) змінних; створення і повернення структури, яка зберігає всі потрібні значення; або ж використання кортежів. Якщо казати про Common Lisp, то можна було б використати будь-який з цих варіантів, при цьому для третього замість кортежів застосувати списки.

Проте є й інший варіант, що не вимагає виконання деструктивних змін, або додаткових дій по створенню і розпаковуванню структури або списку результату. А саме — форми для роботи з багатьма значеннями. Наприклад, однією з функцій, що використовує такі форми, — це функція ділення `floor`, яка першим (основним) результатом повертає частку, а другим результатом — остачу:

```
CL-USER> (floor 7 3)
2 ; частка — основний результат
1 ; остача — другий результат
```

Повернути багато значень можна за допомогою функції `VALUES`, яка повертає всі свої аргументи (яких може бути довільна кількість) як багато значень:

```
CL-USER> (values 1 'a '(2 3))
1
A
(2 3)
```

При цьому, як видно з прикладу, функція VALUES не є формою керування потоком виконання і може бути використана не лише в контексті функції для повернення кількох значень з неї, але й для повернення кількох значень локально.

Для того, щоб "впіймати" всі значення, які повертає функція (або, загалом, будь-яка форма), застосовується макрос MULTIPLE-VALUE-BIND, першим аргументом якого є список символів, з якими необхідно зв'язати "багато значень", які є результатом виконання його другого аргумента. Зв'язані символи можна використовувати в "тілі" MULTIPLE-VALUE-BIND:

```
(multiple-value-bind (symbol*) <mv-form> <body-form>*)  
<mv-form> := form ; довільна форма, яка (зазвичай) повертає багато значень  
<body-form> := form ; довільна форма
```

Результатом виконання MULTIPLE-VALUE-BIND є результат виконання останньої форми його тіла.

Приклад використання:

```
CL-USER> (multiple-value-bind (f r) (floor 7 3)  
          (list f r))  
(2 1)
```

Якщо у списку символів для зв'язування в MULTIPLE-VALUE-BIND визначити менше символів, ніж повертає форма-другий аргумент, тоді всі незв'язані значення будуть втрачені. Аналогічно, якщо для збереження "багатьох значень" використати не MULTIPLE-VALUE-BIND, а, наприклад, LET, тоді збережеться лише перше значення, а решта будуть втрачені. Якщо ж навпаки визначити більше символів, ніж повертає форма-значення, тоді решта символів, яким "не вистачило" фактичних значень, будуть зв'язані зі значенням NIL.

Для трансформування списку у "багато значень" використовується функція `VALUES-LIST`, а для виконання зворотного перетворення ("багатьох значень" у список) — макрос `MULTIPLE-VALUE-LIST`:

```
CL-USER> (values-list '(1 a (2 3)))  
1  
A  
(2 3)  
CL-USER> (multiple-value-list (floor 7 3))  
(2 1)
```

2.4.5 Завдання на СРС

1. Написати функцію `split-odd-even`, яка отримує на вхід список цілих чисел, який вона розбиває на два списки — непарних і парних чисел — і повертає обидва як "багато значень". При цьому порядок елементів у списках-результатах має бути збережено:

```
CL-USER> (multiple-value-bind (odd even)  
  (split-odd-even '(1 2 3 4 5 6 7))  
  (format t "odd: ~A~%" odd)  
  (format t "even: ~A~%" even))  
odd: (1 3 5 7)  
even: (2 4 6)  
NIL
```

2. Написати функції, що реалізують "функціональні множини" (purely functional sets):

- `create-set` отримує на вхід список і повертає предикат, що приймає один аргумент, і визначає, чи належить цей елемент вхідному списку:

```
CL-USER> (let ((contains-0 (create-set nil))  
  (contains-1 (create-set (list 10)))  
  (contains-2 (create-set (list 15 13)))  
  (contains-3 (create-set (list 15 10 21))))  
  (list  
    (funcall contains-0 10)  
    (funcall contains-1 10)  
    (funcall contains-2 21)
```

```
(funcall contains-3 15)))
(NIL T NIL T)
```

- `ps-union` отримує на вхід дві "множини" (результати виконання `create-set`) і повертає предикат, який визначить множину, що є об'єднанням двох множин-аргументів:

```
CL-USER> (let* ((contains-1 (create-set (list 10)))
                 (contains-2 (create-set (list 15 13)))
                 (contains-union (ps-union contains-1 contains-2)))
            (list
             (funcall contains-union 10)
             (funcall contains-union 13)
             (funcall contains-union 14)))
(T T NIL)
```

- `ps-intersection` отримує на вхід дві "множини" (результати виконання `create-set`) і повертає предикат, який визначить множину, що є перетином двох множин-аргументів:

```
CL-USER> (let* ((contains-1 (create-set (list 10 12)))
                 (contains-2 (create-set (list 10 13)))
                 (contains-union (ps-intersection contains-1 contains-2)))
            (list
             (funcall contains-union 10)
             (funcall contains-union 12)
             (funcall contains-union 13)))
(T NIL NIL)
```

- `ps-difference` отримує на вхід дві "множини" (результати виконання `create-set`) і повертає предикат, який визначить множину, що є різницею двох множин-аргументів:

```
CL-USER> (let* ((contains-1 (create-set (list 10 12)))
                 (contains-2 (create-set (list 10 13)))
                 (contains-union (ps-difference contains-1 contains-2)))
            (list
             (funcall contains-union 10)
             (funcall contains-union 12)
             (funcall contains-union 13)))
(NIL T NIL)
```

3. Написати функцію `compose`, яка приймає на вхід довільну послідовність функцій і повертає функцію, що послідовно виконує кожну з функцій над результатом попередньої — справа-наліво. Для останньої функції аргумент(и)

передає користувач. Тобто, це функція, що працює як `call-two-fns` з прикладу в розділі 4.1.2, але для довільної кількості функцій:

```
CL-USER> (funcall (compose #'1+ #'car) '(1))
2
CL-USER> (funcall (compose #'1+ #'1- (lambda (x) (+ x 10)))) 3)
13
CL-USER> (funcall (compose #'rest #'list) 1 2)
(2)
```

4. Написати функцію `make-mapcar-if`, яка отримує на вхід предикат (`pred`) і функцію (`func`) і повертає функцію (`f`) що приймає один аргумент. `f` може бути передана в `mapcar`, який приймає лише один список, і тоді для кожного елемента вона спочатку викликає `pred` і якщо результатом буде не `NIL`, тоді також викликає `func`. В якості результату вона поверне або результат `func`, або ж вхідне значення:

```
(defun make-mapcar-if (pred func) ...)
```

Приклад використання:

```
CL-USER> (setf opt-inc-fn (make-mapcar-if #'oddp #'1+))
#<LAMBDA...>
CL-USER> (mapcar opt-inc-fn '(1 2 3))
(2 2 4) ; ((1+ 1) 2 (1+ 3))
```

5. Написати функцію `gen-sum-accumulator`, яка не приймає аргументів, але повертає функцію, що отримує на вхід число, а повертає суму цього числа з усіма числами, які вона отримувала на вхід перед цим:

```
CL-USER> (mapcar (gen-sum-accumulator) '(1 1 1 1))
(1 2 3 4)
CL-USER> (mapcar (gen-sum-accumulator) '(1 2 3 4))
(1 3 6 10)
```

Тема 2.5 Зв'язування змінних

Щоразу, коли робиться виклик функції і в неї передаються деякі фактичні аргументи, ці передані значення стають значеннями символів, які зазначені в якості формальних параметрів функції. Іншими словами, відбувається **зв'язування змінних** (в даному випадку — формальних параметрів з фактичними значенням). При цьому зв'язування означає не лише *встановлення значення* у змінну а також і *визначення змінної* (у випадку лексичного зв'язування, див. р. 2.5.2.2).

Зв'язування змінних може відбуватись не лише при виклику функції, а також у випадку використання деяких спеціальних форм та макросів. Наприклад, LET, LET*, DO та інших. Спеціальний оператор SETQ при цьому не є формою для зв'язування змінних, оскільки він не визначає змінну, а лише встановлює у неї значення (див. р. 2.5.2.1).

Крім того, а яка область дії значень у зв'язаних змінних? Типовим припущенням буде те, що зв'язане зі змінною значення буде дійсним лише в контексті форми його зв'язування. Наприклад, що значення, передане у функцію і зв'язане з формальним параметром, буде дійсне лише в контексті тіла цієї функції. Проте це припущення не справджується у випадку глобальних змінних. Детальніше контекст зв'язування розглядається в розділі 2.5.2.

2.5.1 Спеціальні оператори LET та LET*

Для зв'язування символів в тілі функції, або ж і в глобальному контексті, можна використати спеціальні оператори LET та LET*. Синтаксис їхнього використання такий:

```
(let (<var-or-var-clause>*) <form>*)  
<var-or-var-clause> := symbol | (symbol [<value-form>])  
<value-form> := s-expr ; довільна форма  
<form> := s-expr ; довільна форма
```

`<var-or-var-clause>*` — це довільна послідовність визначень зв'язувань змінних. Якщо тут зазначається лише ім'я символу, тоді цей символ зв'язується зі значенням `NIL`. Якщо ж використовується формат запису зі списком, де першим елементом є символ, тоді цей символ зв'язується зі значенням результату виконання форми `<value-form>`, якщо вона зазначена, або ж з `NIL`. `<form>*` формує "тіло" `LET`.

`LET` виконує зв'язування змінних, після чого виконує форми з тіла й в якості результату повертає результат виконання останньої з форм тіла. При цьому контекст зв'язування — тіло форми `LET`:

```
CL-USER> (let (var
               (var-2)
               (var-3 15)
               (var-4 (* 4 5)))
           (list var var-2 var-3 var-4))
(NIL NIL 15 20)
```

`LET*` визначається повністю так само. Різниця між `LET` і `LET*` полягає у тому, що зв'язування змінних в `LET` відбувається паралельно, а в `LET*` — послідовно. Тобто, в `LET*` при зв'язуванні кожної наступної змінної можна використовувати значення вже зв'язаної попередньої змінної:

```
CL-USER> (let* ((a 10)
                 (b (* 2 a))
                 (c (+ a b)))
           (list a b c))
(10 20 30)
```

Якщо зробити щось подібне в `LET`, тоді помилка не виникне лише якщо використані змінні вже зв'язані в контексті, де використано такий `LET`:

```
CL-USER> (let ((a 2)
               (b 5))
```

```
(let ((a 10)
      (b (* 2 a))
      (c (+ a b)))
  (list a b c))
(10 4 7)
```

Якби зовнішнього `let` не було — виникла б помилка, адже змінні `a` і `b` були б не визначені при спробі обчислення значень `b` і `c` у вкладеному `let`.

2.5.2 Види зв'язування змінних та область дії зв'язування

В залежності від типу визначення змінної, зв'язування цієї змінної з певним значенням має різну область дії. В Common Lisp розрізняють три види змінних (лексичні, вільні та спеціальні) та два види зв'язування (лексичне та динамічне).

2.5.2.1 Види змінних

Лексичні або **локальні** змінні визначаються за допомогою форм зв'язування (`LET`, `LET*`, `DO` та інших) або ж у визначенні функції/макроса (якщо ці змінні зазначені як формальні параметри), якщо вони не визначені як *спеціальні змінні*.

Спеціальні або **глобальні** або також **динамічні** змінні визначаються за допомогою макросів `DEFVAR` та `DEFPARAMETER`. "Глобальними" ці змінні називаються через те, що зазвичай вони визначаються на найвищому рівні (себто в глобальному контексті) і, відповідно, використовуються будь-де в коді програми. "Динамічними" — бо зв'язування таких змінних є *динамічним* (див. р. 2.5.2.3).

Формат запису `DEFVAR` та `DEFPARAMETER` такий:

```
(defvar <var> [<init-form> [<documentation>]])
(defparameter <var> <init-form> [documentation])
<var> := symbol ; назва змінної
<init-form> := s-expr ; форма початкового значення змінної
<documentation> := string ; рядок документації
```

`DEFVAR` визначає глобальну змінну і опційно встановлює в неї значення (якщо `<init-form>` зазначено). `DEFPARAMETER` визначає глобальну змінну і

обов'язково встановлює в неї значення. Інша відмінність цих двох макросів полягає у тому, що DEFVAR встановлює значення у змінну лише один раз. Тобто, повторне виконання форми (defvar ...) не призведе до заміни значення змінної, якщо значення вже було надане. Це потрібно для того, щоб при інтерактивній роботі з REPL при повторному завантаженні файлу або форми не відбувалось перезаписування значення глобальної змінної. Напротивагу DEFVAR, повторне виконання форми з DEFPARAMETER призводить до перезапису значення у визначену глобальну змінну.

Загальноприйнятим є іменування глобальних змінних з символом * на початку і в кінці імені: *foo*, *some-global-var*. Приклади:

```
(defparameter *hero-name* "Mad Max" "Name of a hero")
(defvar *cassandra-connection*)
```

Вільними називаються змінні, які не були визначені як глобальні, але яким було надане значення на глобальному рівні, наприклад за допомогою SETQ.

Примітка: саме тому, якщо в REPL виконати операцію (setq a 1), перед цим не визначивши a, то буде виведено попередження про те, що змінна A не визначена (undefined variable: COMMON-LISP-USER::A), хоча при цьому A отримає значення 1.

2.5.2.2 Лексичне зв'язування

Лексичне зв'язування відбувається тоді, коли зв'язуються *лексичні* змінні. Наприклад:

```
(defun foo (a)           ; локальна змінна `a' лексично зв'язана із певним
                          ; фактичним значенням
  (let ((b '(1 2 3)))    ; локальна змінна `b' лексично зв'язана зі списком
    ...))
```

"Лексичним" таке зв'язування (і, відповідно змінні, зв'язані таким чином) називається тому, що область його дії обмежена *текстом* тієї форми, де відбулось зв'язування. Або ж область дії *локалізована* в тій формі, де відбулось зв'язування (тому лексично зв'язані змінні також називаються "локальними"). При цьому змінна може бути повторно зв'язана у вкладеній формі зв'язування, але контекст дії вкладеного зв'язування обмежується лише вкладеною формою. Крім того, значення змінної може бути змінено в тілі форми зв'язування, якщо, наприклад, використати SETQ.

Наприклад, в наступному прикладі можна побачити те, що зв'язування символу визначеного в зовнішньому let в його тілі у вкладеному let призводить до зміни значення змінної. Але лише в контексті вкладеного let. Тобто, в наступному випадку результатом буде 2 і 5, а не 10 і 4:

```
CL-USER> (defun my-print (n a b c d)
  (format t "~d> a = ~2d, b = ~2d, c = ~3d, d = ~2d-%" n a b c d))
MY-RPRINT
CL-USER> (let ((a 2)                ; перше значення a
               (b 5)                ; перше значення b
               (d 8))
  (my-print 1 a b nil d)
  (let ((a 10)                       ; друге значення a
        (b (* 2 a))                 ; друге значення b
        (c (+ a b)))
    (my-print 2 a b c d)
    (setq a 15)                      ; третє значення a
    (my-print 3 a b c d))
  ;; after inner `let'
  (my-print 4 a b nil d))
1> a = 2, b = 5, c = NIL, d = 8
2> a = 10, b = 4, c = 7, d = 8
3> a = 15, b = 4, c = 7, d = 8
4> a = 2, b = 5, c = NIL, d = 8
NIL
```

Інший приклад, який демонструє використання лексичних і вільних змінних:

```

CL-USER> (setq x 100) ; встановлення значення у змінну `x' – створення
                               ; вільної змінної
100
CL-USER> (defun f1 (x)
           (f2 x))      ; локальна змінна `x'
F1
CL-USER> (defun f2 (y)
           (list x y))   ; вільна змінна `x' і локальна змінна `y'
F2
CL-USER> (f1 2)
(100 2)

```

Як видно з прикладу, лексичне зв'язування *x* в функції *f1* діє лише в контексті функції *f1*, а вже в функції *f2* *x* є вільною змінною і має значення 100.

2.5.2.3 Динамічне зв'язування

Динамічне зв'язування відбувається щоразу, коли в формі зв'язування/у визначенні функції в якості змінної використовується змінна, що визначена як спеціальна (глобальна). Область дії такого зв'язування не обмежується лише текстовим "тілом" форми зв'язування, а діє весь час, поки виконання форми зв'язування не буде завершено, або ж поки не відбудеться повторне (вкладене) зв'язування. (У випадку вкладеного зв'язування область дії нового зв'язування також обмежене лише часом виконання вкладеної форми зв'язування.) Тобто, навіть якщо з тіла форми зв'язування викликається функція, в якій використовується зв'язана глобальна змінна:

```

CL-USER> (defvar *x* 100) ; `*x*' визначено як глобальну змінну
*x*
CL-USER> (defun f1 (*x*)   ; динамічне зв'язування глобальної змінної `*x*'
           (f2 *x*))
F1
CL-USER> (defun f2 (y)
           (list *x* y))   ; лексично зв'язана `y' та глобальна змінна `*x*'
F2
CL-USER> (f1 2)
(2 2)
CL-USER> *x*
100

```

Тобто, після динамічного зв'язування змінної $*x*$ в функції $f1$, використання цієї змінної за межами $f1$, але до того, як потік виконання завершив виконання форми зв'язування в $f1$, призведе до отримання значення, динамічно зв'язаного з $*x*$, а не значення, що було визначене для змінної глобально.

Динамічне зв'язування застосовується для встановлення значення у змінні, що використовуються як глобальні в програмі. Наприклад, для таких об'єктів як об'єкти зв'язку з базами даних (connection) чи сесії (session). Або також може використовуватись для тимчасової зміни поведінки деяких функцій, поведінка яких залежить від глобальних змінних. (Хоча це, звісно, і не є частиною *чистого* функціонального підходу.)

2.5.2.4 Особливості зв'язування змінних для замикань

У випадку, якщо зв'язані *лексично* змінні використовуються як вільні змінні у замиканні, тоді після завершення виконання форми зв'язування, зв'язані у ній значення нікуди не зникають — допоки існує функція із замикання, що їх використовує (див. р. 4.3).

Тема 2.6 Форми керування потоком виконання

Для керування потоком виконання в Common Lisp використовуються різні форми гілкування, послідовного виконання, циклів та інші форми.

2.6.1 Послідовне виконання

Для забезпечення послідовного виконання довільної кількості форм використовуються макроси PROG1, PROG2 та спеціальний оператор PROGN. Усі вони мають однакову форму запису:

```
(prog<1/2/n> <form>*)  
<form> := s-expr ; довільна форма, яку необхідно виконати
```

Відрізняються вони лише тим, результат виконання якої з форм-аргументів буде повернено. Це визначається номером аргумента в назві відповідної функції:

- PROG1 повертає результат виконання першої форми
- PROG2 повертає результат виконання другої форми
- PROGN повертає результат виконання останньої форми

Приклади:

```
CL-USER> (prog1 (list 1)
                (print "print this")
                (cons 2 3)
                (list 4))

"print this"
(1)
CL-USER> (prog2 (list 1)
                (print "print this")
                (cons 2 3)
                (list 4))

"print this"
"print this"
CL-USER> (progn
                (list 1)
                (print "print this")
                (cons 2 3)
                (list 4))

"print this"
(4)
```

Для роботи з "багатьма значеннями" (р. 2.4.4), існує спеціальний оператор MULTIPLE-VALUE-PROG1 який працює як PROG1, проте дає змогу повернути багато значень:

```
CL-USER> (multiple-value-prog1 (values 1 2)
                (list 3))

1
2
```

При цьому PROGN і сам може повертати "багато значень".

2.6.2 Розгалуження обчислень

В Common Lisp є різні макроси та спеціальні оператори, що дають змогу розгалужувати потік виконання. Крім звичних з інших мов IF та CASE (є аналогом оператора SWITCH в інших мовах програмування), також є такі макроси як WHEN, UNLESS, COND, тощо.

2.6.2.1 IF, WHEN та UNLESS

Спеціальний оператор IF має наступний синтаксис:

```
(if <condition> <t-branch> [<nil-branch>])  
<condition> := s-expr    ; форма умови, яку необхідно перевірити  
<t-branch> := s-expr     ; форма, що виконується в разі справдження умови  
<nil-branch> := s-expr   ; форма, що виконується в разі несправдження умови
```

Обов'язковими аргументами IF є умови та гілка T (або ж true-гілка). Гілка NIL (else-гілка) є опційною. Якщо умова справджується — потік виконання переходить до виконання T-гілки. Умова вважається такою, що справдилась, якщо результатом виконання форми умови є будь-що, відмінне від NIL. Відповідно, якщо умова не справджується (тобто результатом виконання форми умови є NIL), потік виконання переходить до NIL-гілки. Якщо NIL-гілка не визначена, тоді в разі несправдження умови IF поверне NIL.

Приклади:

```
CL-USER> (if (evenp 0) "ok")  
"ok"  
CL-USER> (if (evenp 1) "ok")  
NIL  
CL-USER> (if (evenp 1) "ok" "not ok")  
"not ok"  
CL-USER> (if (and (atom 'a) (listp nil))  
              (print "ok")  
              (print "not ok"))
```

```

"ok"
"ok"
CL-USER>
(defun first-number-p (lst)
  (if (listp lst)
      (numberp (first lst))))
FIRST-NUMBER-P
CL-USER> (first-number-p '(a 2 3))
NIL

```

У випадках, коли одна з гілок IF явно чи неявно повертає NIL (тобто є лише одна гілка, яка може повернути щось, відмінне від NIL), можна використати макроси WHEN або UNLESS. Вони мають такий формат використання:

```

(when <condition> <body>)
(unless <condition> <body>)
<condition> := s-expr ; умова, яку необхідно перевірити
<body> := s-expr* ; форма, що виконується в разі справдження/несправдження умови

```

Тіло WHEN виконується в разі, якщо умова справджується, а тіло UNLESS виконується в разі, якщо умов не справджується. Варто зазначити, що на відміну від IF, тіло WHEN та UNLESS може містити довільну кількість форм, в той час як гілки IF складаються лише з однієї форми (яка, тим не менш, може бути PROGN).

Приклади використання WHEN та UNLESS:

```

CL-USER> (when (evenp 0) "ok")
"ok"
CL-USER> (when (evenp 1) "ok")
NIL
CL-USER> (unless (evenp 0) "ok")
NIL
CL-USER> (unless (evenp 1) "ok")
"ok"
CL-USER>
(defun first-number-p (lst)
  (when (listp lst)
    (numberp (first lst))))
WARNING: redefining COMMON-LISP-USER::FIRST-NUMBER-P in DEFUN
FIRST-NUMBER-P
CL-USER> (first-number-p '(a 2 3))
NIL
CL-USER> (first-number-p '(1 2 3))

```

2.6.2.2 COND

Більш загальною формою розгалуження є макрос COND. Його аргументами є послідовність списків, кожен з яких має умову (перший елемент списку) та опційно тіло (решта елементів списку):

```
(cond <test-clause>*)
<test-clause> := (<condition> <branch-body>) ; гілка COND
<condition>   := s-expr                     ; умова гілки
<branch-body> := s-expr*                    ; тіло гілки (може бути пусте)
```

COND виконує умови з його "гілок" (тобто умови зі списків-аргументів) у порядку їх визначення. Якщо умова справджується, COND припиняє обхід умов і відбувається перехід до виконання "тіла" гілки, умова якої справдилась. Результатом COND стає результат виконання останньої форми в "тілі" гілки, адже форми, що формують "тіло", неявно загорнуті у PROG. Якщо "тіло" не визначене — повертається результат виконання умови. Для позначення гілки за замовчуванням використовується список з умовою T.

Нехай визначено функцію:

```
(defun match (arg)
  (cond
    ((and (numberp arg) (eql arg 1)) "match 1")
    (arg)
    (t "default branch")))
```

Кілька викликів, які тестують її роботу:

```
CL-USER> (match 1)
"match 1"
CL-USER> (match 2)
2
CL-USER> (match "match 2")
```

```
"match 2"  
CL-USER> (match nil)  
"default branch"
```

2.6.2.3 CASE

Ще однією конструкцією розгалуження є макрос CASE. Його аргументами є ключова форма (перший аргумент) та гілки CASE. "Ключовою" форма називається тому, що значення, яке вона повертає, використовується як "ключ" до певної гілки. Гілки — це послідовність списків, першим елементом яких є значення, або список значень, з яким(и) порівнюється ключ. Якщо ключ збігається зі значенням (або одним зі значень) в гілці, тоді виконується "тіло" гілки, яке складається з довільної кількості форм:

```
(case <key-form> <normal-clause>* [<otherwise-clause>])  
<key-form> := s-expr ; ключова форма  
<normal-clause> := (<value-or-values> <branch-body>) ; гілка CASE  
<value-or-values> := atom | (atom*) ; значення (одне або список) для ключа  
<branch-body> := s-expr* ; тіло гілки (може бути пусте)  
<otherwise-clause> := ((otherwise | t) <branch-body>) ; гілка за замовчуванням
```

Значення в <value-or-values> не виконуються інтерпретатором. Тому якщо записати список значень ключа (напр. ((a b c) <branch-body>)), тоді це не буде викликом функції a, а буде трьома можливими значеннями ключа.

Також варто зауважити, що порівняння ключа зі значеннями відбувається за допомогою EQL. Тому використовувати списки в якості значень в CASE не вийде.

Результатом виконання CASE є результат виконання останньої форми гілки, значення в якій збіглось з ключем. Якщо ж тіло такої гілки відсутнє — результатом буде NIL. Гілка за замовчуванням (otherwise) виконується в разі, якщо ключ не збігся зі значеннями в жодній з решти гілок. Для того, щоб порівняти ключ з символами t або otherwise, але не потрапити в гілку за замовчуванням, ці символи варто явно записати у списку.

Нехай визначено функцію:

```
(defun match-with-case (arg)
  (case arg
    (a "match 1")
    ((1 2 (1+ 3)) (format t "Match 2!") "ok 2")
    (5)
    ((t otherwise) "symbol branch")
    (otherwise "default branch")))
```

Приклади роботи CASE:

```
CL-USER> (match-with-case 'a)
"match 1"
CL-USER> (match-with-case 2)
Match 2!
"ok 2"
CL-USER> (match-with-case '(1+ 3))
"default branch"
CL-USER> (match-with-case 4)
"default branch"
CL-USER> (match-with-case 5)
NIL
CL-USER> (match-with-case t)
"symbol branch"
```

2.6.3 Цикли

В Common Lisp є кілька циклічних конструкцій: DO, DO*, DOLIST, DOTIMES та інші.

2.6.3.1 DO та DO*

Макроси DO та DO* є загальними циклічними формами. Вони мають наступний формат запису:

```
(do (<var-or-var-clause>*)
    (<end-test-form> <result-form>*)
  <body-form>*)
<var-or-var-clause> := symbol | (symbol [<init-form> [<step-form>]])
<end-test-form> := s-expr ; умова завершення циклу
<result-form> := s-expr* ; форма, що виконується після справдження умови виходу
<body-form> := s-expr ; довільна форма
```

```
<init-form> := s-expr      ; форма ініціалізації змінної (напр. лічильника)
<step-form> := s-expr      ; форма зміни змінної наприкінці ітерації
```

Алгоритм виконання DO і DO* такий:

1. Відбувається паралельне зв'язування всіх визначених змінних (<var-or-var-clause>*). Змінні, для яких визначена форма ініціалізації (<init-form>), зв'язуються з результатом виконання цієї форми, а решта — з NIL. Зв'язування в DO* відбувається послідовно в порядку визначення змінних — подібно до LET* в порівнянні з LET.
2. Перевіряється умова завершення циклу (<end-test-form>).
3. Якщо умова справджується, тоді потік виконання переходить до виконання форм формування результату (<result-form>*). Якщо форми зазначені — в якості результату DO повертається результат виконання останньої з форм. Якщо ж форми не зазначені, то результатом DO буде NIL.
4. Якщо умова завершення циклу не справджується, відбувається виконання форм тіла циклу (<body-form>*).
5. Після завершення виконання тіла циклу, відбувається паралельне (послідовне для DO*) виконання форм зміни значення змінних (<step-form>).
6. Відбувається перехід до перевірки умови (крок 2).

Приклад використання DO:

```
CL-USER> (do (id
              (val 0)
              (idx 0 (1+ idx)))
            ((>= idx 5) (format t "Finished!") idx)
            (format t "> ~A, ~A, ~A~%" id val idx))
> NIL, 0, 0
> NIL, 0, 1
> NIL, 0, 2
> NIL, 0, 3
```

```
> NIL, 0, 4
Finished!
5
```

Приклад використання DO*:

```
CL-USER> (do* ((a 1 (1+ a))
               (b 0 (* a 2))
               (c 0 (+ a b)))
             ((>= c 10) (format t "Finished!") (list a b c))
             (format t "> ~A, ~A, ~A~%" a b c))
> 1, 0, 0
> 2, 4, 6
> 3, 6, 9
Finished!
(4 8 12)
```

Для повернення з циклу до завершення обходу всіх елементів списку можна використати макрос RETURN (див. Розділ 2.6.4.1), оскільки DO та DO* неявно обгортаються в BLOCK з іменем nil. Наприклад:

```
CL-USER> (do* ((lst '(1 2 3 4) (cdr lst))
               (elem (car lst) (car lst)))
             ((null lst))
             (if (> elem 2)
                 (return t)
                 (format t "> ~A~%" elem)))
> 1
> 2
T
```

Це досягається завдяки тому, що DO та DO* неявно додають реалізацію циклу в якості тіла форми (block nil ...). В цьому можна пересвідчитись, використавши функцію MACROEXPAND-1 для виконання макророзширення деякого макровиклику:

```
CL-USER> (macroexpand-1
          '(do* ((lst '(1 2 3 4) (cdr lst))
                 (elem (car lst) (car lst)))
                ((null lst))
                (if (> elem 2)
```

```

                (return t)
                (format t "> ~A~%" elem))))
(BLOCK NIL
  (LET* ((LST '(1 2 3 4)) (ELEM (CAR LST)))
    (DECLARE (IGNORABLE LST ELEM))
    (TAGBODY
      (GO #:G244)
      #:G243
      (TAGBODY
        (IF (> ELEM 2)
          (RETURN T)
          (FORMAT T "> ~A~%" ELEM)))
      (SETQ LST (CDR LST)
            ELEM (CAR LST))
      #:G244
      (UNLESS (NULL LST) (GO #:G243))
      (RETURN-FROM NIL (PROGN))))
T

```

2.6.3.2 DOLIST

Макрос циклу DOLIST забезпечує ітерування по елементах списку. Формат його запису такий:

```

(dolist (<var> <list-form> [<result-form>]
  <body-form>*)
<var> := symbol ; змінна для зв'язування поточного елемента списку
<list-form> := s-expr ; форма отримання списку, по якому необхідно ітеруватись
<result-form> := s-expr ; форма результату `dolist'
<body-form> := s-expr ; довільна форма, що буде виконана на кожній ітерації

```

На кожній ітерації змінна <var> циклу зв'язується зі значенням елемента списку, після чого виконуються форми "тіла" циклу. Після виконання всіх форм тіла відбувається перехід на наступну ітерацію. Коли тіло виконається для всіх елементів списку, робота циклу завершується і повертається значення виконання <result-form>, якщо вона зазначена, або NIL:

```

CL-USER> (dolist (elem (list 1 'a 2 'b))
  (format t "Element: ~A~%" elem))
Element: 1
Element: A
Element: 2
Element: B

```

NIL

Реалізація функції MY-REVERSE із використанням DOLIST і псевдофункції SETQ:

```
CL-USER>
(defun my-reverse (list)
  "Reverse input `list' using `dolist' macro"
  (let (reversed)
    (dolist (elem list reversed)
      (setq reversed (cons elem reversed)))))
MY-REVERSE
CL-USER> (my-reverse '(1 2 3 4 5))
(5 4 3 2 1)
CL-USER> (my-reverse '(1 a (2 3 4) t))
(T (2 3 4) A 1)
```

Як і у випадку з DO, для повернення з циклу до завершення обходу всіх елементів списку можна використати макрос RETURN.

2.6.3.3 DOTIMES

Макрос циклу DOLIST забезпечує ітерування визначену кількість разів. Формат його запису такий:

```
(dotimes (<var> <count-form> [<result-form>])
  <body-form>*)
<var> := symbol ; змінна для зв'язування номера ітерації
<count-form> := s-expr ; форма отримання числа ітерацій
<result-form> := s-expr ; форма результату `dolist'
<body-form> := s-expr ; довільна форма, що буде виконана на кожній ітерації
```

На кожній ітерації змінна <var> циклу зв'язується зі значенням номера поточної ітерації, починаючи з 0. Після зв'язування виконуються форми "тіла" циклу. Після виконання всіх форм тіла відбувається перехід на наступну ітерацію. Коли тіло виконається визначену кількість разів, робота циклу завершується і повертається значення виконання <result-form>, якщо вона зазначена, або NIL:

```
CL-USER> (dotimes (n 5)
           (format t "n = ~A~%" n))
n = 0
n = 1
n = 2
n = 3
n = 4
NIL
```

Як і у випадку з DO, для повернення з циклу до завершення обходу визначену кількість разів можна використати макрос RETURN.

2.6.3.4 Макрос LOOP

Макрос LOOP — це також один із видів циклічних конструкцій. Він є значено універсальнішим за інші циклічні конструкції, оскільки може реалізовувати різні види циклів. Фактично, макрос реалізує свою мову для визначення циклів. Детально про нього можна прочитати в документації [37], а також в статті "LOOP for Black Belts" [39].

2.6.4 Інші форми керування потоком виконання

Є й інші форми для керування потоком виконання, крім форм послідовності, розгалуження та циклів. Це такі форми як BLOCK, UNWIND-PROTECT, THROW, CATCH та інші.

2.6.4.1 BLOCK, RETURN, RETURN-FROM

Спеціальний оператор BLOCK створює іменовану точку виходу (шляхом лексичного зв'язування символу). Він записується наступним чином:

```
(block <tag-name> <form>*)
<tag-name> := symbol ; ім'я блоку
<form> := s-expr ; довільні форми, з яких формується "тіло" `block'
```

Після визначення імені блоку, виконуються форми тіла. В тілі блоку можуть бути використані спеціальний оператор RETURN-FROM та макрос RETURN для

виконання локального переходу. Можна сказати, що це структурований перехід на мітку, де міткою є ім'я блоку. Якщо ж для виходу з BLOCK не будуть використані RETURN-FROM або RETURN, тоді результатом виконання стане результат виконання останньої форми тіла.

Формат запису RETURN-FROM та RETURN такий:

```
(return-from <tag-name> [<result-form>])  
(return [<result-form>])  
<tag-name> := symbol      ; ім'я блоку, з якого необхідно повернутись  
<result-form> := s-expr   ; результат, який необхідно повернути
```

Ім'я блоку для RETURN визначене неявно як NIL. Цей макрос зазвичай використовується в формах, які неявно визначають блок з іменем NIL, наприклад DO, DO*, DOLIST та інші. Іншими словами, наступні записи ідентичні:

```
(return t) == (return-from nil t)
```

Якщо результат для повернення не зазначений в RETURN-FROM або RETURN, тоді значенням, повернутим з BLOCK буде NIL.

Приклад використання BLOCK:

```
CL-USER> (block stop  
           (format t "start~%")  
           (return-from stop)  
           (format t "end~%"))  
start  
NIL  
CL-USER> (block outer  
           (block inner  
             (return-from outer 1))  
           2)  
1
```

РОЗДІЛ 3

РЕКУРСІЯ І ЇЇ ВИКОРИСТАННЯ У ФУНКЦІОНАЛЬНОМУ ПРОГРАМУВАННІ

У чистому функціональному програмуванні майже не використовуються операції присвоєння, передачі керування (структуровані переходи) та цикли. Натомість використовуються рекурсивні виклики. *Рекурсія* — це метод визначення об'єкта, за якого об'єкт визначається через самого себе [35]. Функція є *рекурсивною*, якщо в її визначенні є виклик(и) самої себе. В Common Lisp рекурсія застосовується при роботі зі списками, оскільки списки самі мають рекурсивну будову.

Тема 3.1 Проста рекурсія

Найпростішим видом рекурсії є *проста рекурсія*, про яку можна говорити, якщо в функції є лише один виклик самої себе в одній або кількох гілках. Типовим прикладом простої рекурсії є обчислення факторіалу числа (рис. 3.1).

$$n! = \begin{cases} 1 & n = 0, \\ n \cdot (n - 1)! & n > 0. \end{cases}$$

Рисунок 3.1 — Рекурентна формула обчислення факторіалу

Реалізація обчислення факторіалу мовою Common Lisp має наступний вигляд:

```
CL-USER>
(defun factorial (n)
  "Calculate factorial of n"
  (if (<= n 1)
      1
      (* n (factorial (1- n)))))
```

```

FACTORIAL
CL-USER> (factorial 0)
1
CL-USER> (factorial 1)
1
CL-USER> (factorial 3)
6
CL-USER> (factorial 5)
120

```

Інші приклади: функції MY-NTH, MY-NTHCDR та MY-LAST, що є аналогами функцій NTH, NTHCDR та LAST.

MY-NTH — для спрощення вважатимемо, що індекс шуканого елемента (n) завжди більший або рівний нулю (якби було треба, можна було б додати перевірку першого n):

```

CL-USER>
(defun my-nth (n lst)
  "Get `n`-th element of list `lst`"
  (if (= n 0)
      (car lst)
      (my-nth (1- n) (cdr lst))))
MY-NTH
CL-USER> (my-nth 0 '(1 2 3))
1
CL-USER> (my-nth 2 '(1 2 3))
3
CL-USER> (my-nth 5 '(1 2 3))
NIL

```

В наведеній реалізації умовою зупину рекурсії є рівність n нулю. Це означає, що рекурсивні виклики припиняться лише після того, як виконаються n разів — навіть якщо список lst має меншу довжину. При цьому функція не "впаде", адже car та cdr для пустого списку (nil) повертають nil (пустий список). Щоб такого не відбувалось, варто додати перевірку списку на пустоту:

```

(defun my-nth (n lst)
  "Get `n`-th element of list `lst`"
  (if (or (null lst) (= n 0))
      (car lst)
      (my-nth (1- n) (rest lst))))

```

Або ж, якщо повернення результату (car lst) у випадку пустого lst виглядає недоречно (хоча, знов таки, результатом буде той самий nil, можна переписати функцію з cond:

```
(defun my-nth (n lst)
  "Get `n`-th element of list `lst`"
  (cond
    ((null lst) nil)
    ((= n 0) (car lst))
    (t (my-nth (1- n) (rest lst)))))
```

MY-NTHCDR має таку ж реалізацію, як у MY-NTH, але відрізняється друга гілка cond:

```
CL-USER>
(defun my-nthcdr (n lst)
  "Get `n`-th element of list `lst`"
  (cond
    ((null lst) nil)
    ((= n 0) lst)
    (t (my-nthcdr (1- n) (rest lst)))))
CL-USER> (my-nthcdr 0 '(1 2 3))
(1 2 3)
CL-USER> (my-nthcdr 1 '(1 2 3))
(2 3)
CL-USER> (my-nthcdr 4 '(1 2 3))
NIL
```

Або ж інший варіант без cond, з неявним поверненням nil у випадку пустого списку (аналогічним чином можна переписати і MY-NTH) завдяки when:

```
(defun my-nthcdr (n lst)
  "Get `n`-th tail of list `lst`"
  (when lst
    (if (= n 0)
        lst
        (my-nthcdr (1- n) (rest lst)))))
```

Реалізація MY-LAST така:

```
CL-USER>
(defun my-last (lst)
  "Get list with last element of input list"
  (if (rest lst)
      (my-last (rest lst))
      lst))
MY-LAST
CL-USER> (my-last (list 1 2 3))
(3)
CL-USER> (my-last ())
NIL
```

І ще один приклад простої рекурсії — функція IS-IN-LIST яка визначає, чи є заданий елемент у заданому списку (порівнюючи з EQL). Реалізація подібна до MY-NTH, в якій змінено умову зупину пошуку і результат:

```
CL-USER>
(defun is-in-list (elem lst)
  "Define whether `elem' is in `lst'"
  (cond
   ((null lst) nil)
   ((eql (car lst) elem) t)
   (t (is-in-list elem (cdr lst)))))
IS-IN-LIST
CL-USER> (is-in-list 'a '(b c d))
NIL
CL-USER> (is-in-list 'c '(b c d))
T
```

Функція REMOVE-FIRST, що видаляє перше входження заданого елемента зі списку, конструюючи новий список, має такий вигляд:

```
CL-USER>
(defun remove-first (elem lst)
  "Remove first occurrence of `elem' in `lst'"
  (cond
   ((null lst) nil)
   ((eql (car lst) elem) (cdr lst))
   (t (cons (car lst) (remove-first elem (cdr lst)))))
REMOVE-FIRST
CL-USER> (remove-first 1 '(0 2 4))
(0 2 4)
CL-USER> (remove-first 1 '(1 2 3))
(2 3)
```

```
CL-USER> (remove-first 3 '(1 2 3))
(1 2)
CL-USER> (remove-first 3 '(1 2 3 4 5))
(1 2 4 5)
```

Функція INSERT-BEFORE, додає заданий елемент-атом (перший аргумент) перед іншим елементом (другий аргумент) у списку атомів (третій аргумент). Якщо елемент, перед яким необхідно додати заданий елемент, відсутній у списку, додавання не відбувається:

```
CL-USER>
(defun insert-before (item pivot lst)
  "Insert atom `item' before `pivot' in `lst'"
  (when lst
    (if (eq (car lst) pivot)
        (cons item lst)
        (cons (car lst) (insert-before item pivot (cdr lst))))))
WARNING: redefining COMMON-LISP-USER::INSERT-BEFORE in DEFUN
INSERT-BEFORE
CL-USER> (insert-before 'x 'a '(a b c))
(X A B C)
CL-USER> (insert-before 'x 'b '(a b c))
(A X B C)
CL-USER> (insert-before 'x 'd '(a b c))
(A B C)
```

Для того, щоб уникнути копіювання списку функцією INSERT-BEFORE в разі, якщо шуканий елемент (pivot) у списку відсутній, можна додати найпростіше порівняння результату рекурсивного виклику з вхідним списком:

```
(defun insert-before (item pivot lst)
  "Insert atom `item' before `pivot' in `lst'"
  (when lst
    (if (eq (car lst) pivot)
        (cons item lst)
        (let ((res (insert-before item pivot (cdr lst))))
          (if (eq res (cdr lst))
              lst
              (cons (car lst) res)))))))
```

Тоді проста перевірка покаже, що вхідний і вихідний списки — це один і той самий список:

```
CL-USER> (let ((in-lst '(a b c)))  
           (eq in-lst (insert-before 'x 'd in-lst)))  
T  
CL-USER> (let ((in-lst '(a b c)))  
           (eq in-lst (insert-before 'x 'c in-lst)))  
NIL
```

Функція MY-LENGTH, що визначає довжину списку-аргумента:

```
CL-USER>  
(defun my-length (lst)  
  "Get length of `lst'"  
  (if lst  
      (1+ (my-length (rest lst)))  
      0))  
MY-LENGTH  
CL-USER> (my-length nil)  
0  
CL-USER> (my-length '(a b))  
2  
CL-USER> (my-length '(a b 3 4 e))  
5
```

3.1.1 Хвостова та не хвостова рекурсія

Проста рекурсія може бути хвостовою і не хвостовою. Рекурсія є *хвостовою*, якщо рекурсивний виклик є останнім в потоці виконання поточного виклику функції. Якщо ж після повернення потоку керування з рекурсивного виклику виконуються ще якісь операції, тоді рекурсія є *не хвостовою*.

Прикладами хвостової рекурсії є реалізації функцій MY-NTN, MY-NTHCDR, MY-LAST та IS-IN-LIST, що були розглянуті раніше. Прикладами не хвостової рекурсії є реалізації FACTORIAL, REMOVE-FIRST та MY-LENGTH.

Виокремлення хвостової рекурсії є важливим тому, що функції з таким видом рекурсії можуть бути перетворені на цикл, що в свою чергу може значно

пришвидшити виконання обчислень, оскільки цикли, на відміну від рекурсивних викликів, не вимагають використання стеку викликів, збереження контексту і т. і. Тому часто функціональні мови програмування явно (у вигляді мовних конструкцій) або ж неявно (за рахунок суто роботи компілятора) оптимізують роботу програм з хвостовою рекурсією.

Наприклад, реалізації функцій MY-NTH і IS-IN-LIST за допомогою циклів мають наступний вигляд:

```
CL-USER>
(defun my-nth-cyclic (n list)
  "Cyclic implementation of `my-nth'"
  (do ((i n (1- i))
      (lst list (cdr lst)))
      ((or (null lst) (= i 0)) (car lst))))
MY-NTH-CYCLIC
CL-USER> (my-nth-cyclic 0 '(1 2 3))
1
CL-USER> (my-nth-cyclic 2 '(1 2 3))
3
CL-USER> (my-nth-cyclic 5 '(1 2 3))
NIL
```

```
CL-USER>
(defun is-in-list-cyclic (elem list)
  "Cyclic implementation of `is-in-list'"
  (do ((lst list (rest lst))
      ((or (null lst)
          (eql (car lst) elem))
       (and lst t))))
IS-IN-LIST-CYCLIC
CL-USER> (is-in-list-cyclic 'a '(b c d))
NIL
CL-USER> (is-in-list-cyclic 'c '(b c d))
T
CL-USER> (is-in-list-cyclic nil '(b c nil))
T
```

Наведений вище приклад реалізації функції FACTORIAL є реалізацією не хвостової рекурсії. Якщо ж трохи його переписати, тоді можна отримати реалізацію з хвостовою рекурсією:

```

(defun factorial-2-inner (n res)
  (if (<= n 1)
      res
      (factorial-2-inner (1- n) (* n res))))

(defun factorial-2 (n)
  "Calculate factorial of n"
  (if (< n 0)
      (error "N should not be negative!")
      (factorial-2-inner n 1)))

```

Прикладом не хвостової рекурсії є також наступна реалізація функції MY-COPY-LIST, що виконує копіювання списку-аргументу:

```

CL-USER>
(defun my-copy-list (lst)
  (when lst
    (cons (first lst) (my-copy-list (rest lst)))))
MY-COPY-LIST
CL-USER> (my-copy-list '(1 2 3))
(1 2 3)
CL-USER> (let ((lst '(1 2 3)))
           (eq lst (my-copy-list lst)))
NIL

```

Примітка: в Common Lisp вже є реалізована функція COPY-LIST (див. HyperSpec [17]).

Варіант функції MY-COPY-LIST з хвостовою рекурсією такий:

```

CL-USER>
(defun my-copy-list-2-inner (lst res)
  (if lst
      (my-copy-list-2-inner
        (rest lst)
        (nconc res (list (first lst)))) ; nconc змінює новостворений список
      res))

(defun my-copy-list (lst)
  (when (listp lst)
    (when lst
      (my-copy-list-2-inner (rest lst)
                            (cons (first lst) nil))))))
MY-COPY-LIST
CL-USER> (let ((lst '(1 2 3)))
           (eq lst (my-copy-list lst)))
NIL

```

```
CL-USER> (my-copy-list '(1 2 3))  
(1 2 3)
```

Як можна побачити з прикладів вище, в деяких випадках використання додаткової функції, яка крім вхідних параметрів основної функції також отримує на вхід значення проміжного результату, а в кінці своєї роботи повертає його, дає змогу отримати інший вид рекурсії. Це також може призвести до зменшення витрат пам'яті та/або часу виконання, наприклад за рахунок того, що відбуватиметься менше копіювань частин проміжного результату — у випадках, коли результатом є якийсь новий сконструйований список.

Тема 3.2 Інші види рекурсії

Крім простої рекурсії вирізняють ще такі види рекурсії, як *паралельна*, *взаємна* та *рекурсія вищого порядку*.

3.2.1 Паралельна рекурсія

Паралельна рекурсія — це такий вид рекурсії, за якого в кодї функції є кілька рекурсивних викликів, що знаходяться в одній і тій же гілці розгалуженого алгоритму. Тобто, функція має умовно такий вигляд:

```
(defun foo (...)  
  ...  
  (bar ... (foo ...) ... (foo ...) ...)  
  ...)
```

Класичним прикладом паралельної рекурсії є функція визначення чисел Фібоначчі (рис. 3.2).

$$F_n = \begin{cases} 1, & n=0 \\ 1, & n=1 \\ F_{n-1} + F_{n-2}, & n \geq 2 \end{cases}$$

Рисунок 3.2 — Рекурентна формула обчислення чисел Фібоначчі

Реалізація функції обчислення чисел Фібоначчі мовою Common Lisp має наступний вигляд:

```
CL-USER>
(defun fibonacci (n)
  "Calculate Fibonacci number for `n'"
  (if (<= n 1)
      1
      (+ (fibonacci (- n 1))
         (fibonacci (- n 2)))))
FIBONACCI
CL-USER> (fibonacci 1)
1
CL-USER> (fibonacci 2)
2
CL-USER> (fibonacci 6)
13
```

Інший приклад — функція MY-EQUAL, що є аналогом стандартної функції EQUAL:

```
CL-USER>
(defun my-equal (obj other-obj)
  "Compare two objects as `equal' does"
  (cond
   ((null obj) (null other-obj))
   ((atom obj) (when (atom other-obj) (eql obj other-obj)))
   ((atom other-obj) nil)
   (t (and (my-equal (car obj) (car other-obj))
            (my-equal (cdr obj) (cdr other-obj)))))
MY-EQUAL
CL-USER> (my-equal 'a 'a)
T
CL-USER> (my-equal '(1 2 (c) d) '(1 2 (c) d))
T
CL-USER> (my-equal '(1 2 (c) d) '(1 b (c) d))
NIL
```

Ще один приклад: функція COUNT-ATOMS, яка рекурсивно рахує кількість атомів у вхідному аргументі. Якщо аргумент — атом, то результат буде 1. Якщо ж список — тоді відбувається рекурсивний обхід списку і підрахунок:

```
CL-USER>
(defun count-atoms (obj)
  "Count atoms in `obj'"
  (if (atom obj)
      1
      (+ (count-atoms (car obj))
         (if (cdr obj)
             (count-atoms (cdr obj))
             0)))))
COUNT-ATOMS
CL-USER> (count-atoms 1)
1
CL-USER> (count-atoms '(1 nil 3))
3
CL-USER> (count-atoms '(1 (2 nil t) 3))
5
```

3.2.2 Взаємна рекурсія

Взаємна рекурсія — це такий вид рекурсії, за якого рекурсивний виклик функції робиться не напряду зсередини себе, а в іншій функції, що викликається з неї. Умовно такий вид рекурсії виглядає ось так:

```
(defun bar (... )
  ...
  (foo ...)
  ...)

(defun foo (... )
  ...
  (bar ...)
  ...)
```

При цьому також можна сказати, що рекурсія є *непрямою*, оскільки рекурсивний виклик знаходиться в іншій функції (виклик ``foo`` знаходиться в ``bar``

і навпаки). *Прямою ж* рекурсія є тоді, коли рекурсивний виклик знаходиться в тій самій функції, що викликається.

Прикладом взаємної рекурсії є такий: якщо розбити обхід списку і перевірку на атомарність в функції COUNT-ATOMS, розглянутої раніше, на дві функції, то можна отримати реалізацію із взаємною рекурсією:

```
CL-USER>
(defun count-atoms-in-list (lst)
  (if lst
      (+ (count-atoms-2 (car lst))
         (count-atoms-in-list (cdr lst)))
      0))

(defun count-atoms-2 (item)
  "Count atoms in input `item'"
  (if (atom item)
      1
      (count-atoms-in-list item)))
COUNT-ATOMS-2
CL-USER> (count-atoms-2 1)
1
CL-USER> (count-atoms-2 '(1 nil 3))
3
CL-USER> (count-atoms-2 '(1 (2 nil t) 3))
5
```

Іншим прикладом взаємної рекурсії може бути реалізація розбору рекурсивних правил деякої граматики, наприклад методом рекурсивного спуску. Нехай такий парсер має виконати аналіз послідовності "тверджень" (statement), одним з яких може бути блоковий оператор. Фрагмент граматики має наступний вигляд:

```
statement-list : statement * ;
statement : block-statement | if-statement | expr-statement | ... ;
block-statement : '{' statement-list '}' ;
if-statement : ... ;
expr-statement : ... ;
```

Ідентифікатори в даному прикладі — це нетермінальні символи, в той час як кватований текст ('{', '}') — термінальні символи. ':' означає "є" або "складається з", '|' — "або", '*' — "нуль і більше", ';' — завершення правила граматики; '...' позначає "щось", що не має значення для даного прикладу.

Як можна побачити, нетермінал `statement-list` міститься всередині `block-statement`, а `block-statement` опосередковано може знаходитись в `statement-list`. Тож при реалізації парсера за такою граматикою можна отримати реалізацію з приблизно такими функціями:

```
(defun parse-statement-list (...)  
  ...  
  (statement ...)  
  ...)  
  
(defun parse-statement (...)  
  ...  
  (or (block-statement ...)  
       (if-statement ...)  
       (expr-statement ...)  
       ...)  
  ...)  
  
(defun parse-block-statement (...)  
  ...  
  (parse-statement-list ...)  
  ...)
```

3.2.3 Рекурсія вищого порядку

Рекурсія вищого порядку — це такий вид рекурсії, за якого аргументом рекурсивного виклику є результат іншого рекурсивного виклику. Умовно такий вид рекурсії виглядає наступним чином:

```
(defun foo (...)  
  ...  
  (foo ... (foo ...) ...)  
  ...)
```

Класичним прикладом рекурсії вищого порядку є функція Акермана (рис. 3.3).

$$A(m, n) = \begin{cases} n + 1, & m = 0; \\ A(m - 1, 1), & m > 0, n = 0; \\ A(m - 1, A(m, n - 1)), & m > 0, n > 0. \end{cases}$$

Рисунок 3.3 — Рекурентна формула обчислення функції Акермана

Реалізація функції Акермана мовою Common Lisp така:

```
CL-USER>
(defun akerman (m n)
  (cond
    ((= m 0) (+ n 1))
    ((= n 0) (akerman (- m 1) 1))
    (t (akerman (- m 1) (akerman m (- n 1))))))
AKERMAN
CL-USER> (akerman 1 1)
3
CL-USER> (akerman 3 1)
13
CL-USER> (akerman 1 3)
5
```

3.2.4 Реалізація циклів за допомогою рекурсії

Закономірним є те, що якщо рекурсію можна перетворити у цикл (як зазначено в р. 3.1.1), то і цикл можна реалізувати за допомогою рекурсії. Для цього цикл необхідно переписати як рекурсивну функцію. Рекурсивний виклик функції буде аналогом переходу на нову ітерацію циклу.

У випадку вкладених циклів необхідно реалізувати рекурсивну функцію для кожного циклу. Тоді кілька рекурсивних функцій викликатимуть одна одну, реалізуючи просту або взаємну рекурсію (або й інші види).

3.2.4.1 Дворівневі цикли

В якості прикладу перетворення циклів, що мають один вкладений цикл, розглянемо наступну задачу: необхідно визначити скільки списків-елементів вхідного списку містять заданий елемент.

Реалізація за допомогою циклів мовою C така (замість списку зі списками тут використовується матриця):

```
#include <stdio.h>

#define N 5
#define M 5

int main()
{
    // test data
    int data[N][M] = {
        {1, 2, 3, 4, 5},
        {2, 4, 6, 8, 10},
        {1, 3, 5, 7, 9},
        {0, 0, 1, 2, 2},
        {0, 5, 5, 9, 10}
    };

    int element = 5;

    // implementation
    int counter = 0;

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) {
            if (data[i][j] == element) {
                counter++;
                break;
            }
        }
    }

    printf("Number of arrays with %d is %d\n", element, counter);

    return 0;
}
```

Аналогічна їй реалізація мовою Common Lisp (на базі списків):

```

(defparameter data
  '((1 2 3 4 5)
    (2 4 6 8 10)
    (1 3 5 7 9)
    (0 0 1 1 2)
    (0 5 5 9 10)))

(defun count-lists-with-element (element lists)
  "Count lists in `lists' that have `element'"
  (let ((counter 0))
    ;; замість використання індексів в цій реалізації вже відбувається робота
    ;; з головою і хвостом списку
    (do* ((lists-aux lists (rest lists-aux))
          (list (first lists-aux) (first lists-aux)))
          ((null lists-aux))
      ;; знайти `element' в `list'
      (do ((lst list (rest lst)))
          ((null lst))
        (when (eql (first lst) element)
          (incf counter)
          (return))))))
    ;; повернути лічильник
  counter))

```

Перш ніж реалізувати цю ж логіку за допомогою рекурсії, розділимо цикли по окремим функціям:

```

(defun count-lists-with-element (element lists)
  "Count lists in `lists' that have `element'"
  (let ((counter 0))
    ;; instead of working with index, walk over input `lists'
    (do* ((lists-aux lists (rest lists-aux))
          (list (first lists-aux) (first lists-aux)))
          ((null lists-aux))
      (when (is-in-list-cyclic element list)
        (incf counter)))
    ;; return result
  counter))

(defun is-in-list-cyclic (element list)
  "Check whether `element' is in `list'"
  (do ((lst list (rest lst)))
      ((null lst))
    (when (eql (first lst) element)
      (return t))))

```

Тепер окремі функції можна переписати з використанням рекурсії:

- `is-in-list-cyclic` рекурсивно реалізується як `is-in-list` (трохи змінена версія того `is-in-list`, що розглядався в р. 3.1): функція перевіряє, чи міститься `element` в `list`;
- `count-lists-with-element` переписана за допомогою хвостової рекурсії: функція визначає лічильник як опційний параметр і збільшує його щоразу, коли `is-in-list` повертає `T`, передаючи результат у рекурсивний виклик.

```
(defun count-lists-with-element (element lists &optional (counter 0))
  "Count lists in `lists' that have `element'"
  (if lists
      (count-lists-with-element
        element (rest lists) (if (is-in-list element (first lists))
                                (1+ counter)
                                counter))
      counter))

(defun is-in-list (element list)
  "Check whether `element' is in `list'"
  (when list
    (or (eql (car list) element)
        (is-in-list element (rest list)))))
```

Результат тестування:

```
CL-USER> (count-lists-with-element 5 data)
3
```

Така реалізація — це реалізація з використанням простої рекурсії. Проте також можливий варіант з взаємною рекурсією. В такому разі вхідний список списків і результат потрібно прокидати крізь обидві функції:

```
(defun count-lists-with-element (element lists &optional (counter 0))
  "Count lists in `lists' that have `element'"
  (if lists
      (is-in-list element (first lists) (rest lists) counter)
      counter))
```

```
(defun is-in-list (element list lists counter)
  "Check whether `element' is in `list'"
  (if list
      (if (eql (car list) element)
          (count-lists-with-element element lists (1+ counter))
          (is-in-list element (rest list) lists counter))
      (count-lists-with-element element lists counter)))
```

Проте варіант з взаємною рекурсією для такого завдання буде менш ефективним, оскільки максимальний рівень рекурсії який може знадобитись для виконання буде рівним $N * AvgM$, де N — початкова довжина вхідного списку `lists`, а $AvgM$ — середня довжина `list` (списків-елементів `lists`). У випадку ж простої рекурсії, максимальний рівень рекурсії буде $N + MaxM$, де $MaxM$ — максимальна довжина `list`. Крім значного збільшення глибини рекурсії, в другому варіанті також в кожен з викликів передається більше аргументів, що також негативно впливає на швидкодію.

3.2.4.2 Трирівневі цикли

Якщо додати ще один цикл (рівень) в попереднього дворівневого циклу, тоді до рекурсивної реалізації також треба додати ще одну рекурсивну функцію. Наприклад, якщо треба визначити яка максимальна кількість входжень списків із заданим елементом у "рядки" "матриці" списків (тобто, тут вже є три рівні списків), тоді це можна зробити ось так:

```
(defparameter m-data
  '(((1 2 3 4 5)
      (2 4 6 8 10)
      (1 3 5 7 9)
      (0 0 1 1 2)
      (0 5 5 9 10))
    ((5)
      (5)
      (5)
      (5)
      (0))))
```

```

(defun max-count-lists-with-element (element m-lists &optional (max-count 0))
  "Define max count of lists with `element' in rows (lists) of `m-lists'"
  (if m-lists
      (max-count-lists-with-element
       element
       (rest m-lists)
       (max max-count
            (count-lists-with-element element (first m-lists))))
      max-count))

(defun count-lists-with-element (element lists &optional (counter 0))
  "Count lists in `lists' that have `element'"
  (if lists
      (count-lists-with-element
       element (rest lists) (if (is-in-list element (first lists))
                                (1+ counter)
                                counter))
      counter))

(defun is-in-list (element list)
  "Check whether `element' is in `list'"
  (when list
    (or (eql (car list) element)
        (is-in-list element (rest list)))))

```

Результат тестування:

```

(max-count-lists-with-element 5 m-data)
4

```

3.2.5 Завдання на СРС

Реалізувати функції, використовуючи рекурсію і конструктивний підхід:

1. Написати функцію `sum`, яка рахує суму чисел зі списку-аргумента. Якщо вхідний список пустий, функція має повертати 0.

```

CL-USER> (sum '(1 2 3))
6
CL-USER> (sum ())
0

```

2. Написати функцію `pow`, яка підносить число (перший аргумент) у заданий степінь (другий аргумент):

```
CL-USER> (pow 2 2) ; 2^2
4
CL-USER> (pow 3 4) ; 3^4
81
```

3. Написати функцію `inc-list`, яка приймає на вхід список чисел і повертає новий список з числами, більшими на 1.

```
CL-USER> (inc-list '(1 2 3))
(2 3 4)
```

4. Написати функцію `remove-all`, яка видаляє всі входження заданого елемента (по `eq`) у заданий список — аналогічну `remove-first`, але для всіх входжень.

```
CL-USER> (remove-all 'a '(1 a 2 a 3 a))
(1 2 3)
```

5. Написати функцію `remove-nth`, яка видаляє зі списку елемент за певним індексом, конструюючи новий список.

```
CL-USER> (remove-nth 1 '(1 a 2 b))
(1 2 b)
CL-USER> (remove-nth 4 '(1 a 2 b))
(1 a 2 b)
```

6. Написати функцію `my-reverse`, яка обертає список-аргумент (розміщує його елементи у зворотному порядку).

```
CL-USER> (my-reverse '(1 (2 3) 4))
(4 (2 3) 1)
```

7. Написати функцію `sum-two-lists`, яка отримує на вхід два списки з числами і повертає новий список, що містить суми відповідних елементів списків-аргументів.

```
CL-USER> (sum-two-lists '(1 2 3) '(1 2 3))  
(2 4 6)  
CL-USER> (sum-two-lists '(1 2) '(1 2 3))  
(2 4 3)
```

8. Написати функцію `count-lists`, що є аналогічною розглянутій функції `count-atoms`, але яка рахує кількість непустих списків всередині свого аргументу.

```
CL-USER> (count-lists '(1 2 3))  
1  
CL-USER> (count-lists '(1 nil (2 (3) nil)))  
3
```

9. Написати функцію `my-reverse-2`, що змінює порядок розміщення елементів на протилежний не лише у вхідному списку, але і у його підсписах. (Може бути кілька варіантів реалізації — з різними видами рекурсії.)

```
CL-USER> (my-reverse-2 '(1 (2 (3 4))))  
(((4 3) 2) 1)
```

10. Написати функцію `my-copy-tree`, яка виконує копіювання списку і його підсписків.

```
CL-USER> (my-copy-tree '(1 (2 (3 4))))  
(1 (2 (3 4))) ; НОВИЙ СПИСОК З НОВИМИ ПІДСПИСКАМИ
```

11. Написати функцію `flatten`, яка "розгортає" список-аргумент із вкладеними списками у "плаский" (однорівневий) список.

```
CL-USER> (flatten '(1 (2) (3 (4)) 5))  
(1 2 3 4 5)
```

РОЗДІЛ 4

ВИДИ ФУНКЦІЙ ТА ЇХ ЗАСТОСУВАННЯ

Тема 4.1 Функції вищого порядку

Як зазначено в р. 4.2, лямбда-вирази створюють функціональні об'єкти, які описують певну поведінку, і які, якщо їх записати в якості першого елемента списку, будуть "викликані" з деякими фактичними значеннями. В р. 2.4.1 показано, як створити іменовану функцію, тобто лямбда-вираз, що "прив'язаний" до певного символу в якості функції (р. 2.2.1). Іменовані функції викликаються інтерпретатором, коли назва функції (символ) зазначений в якості першого елемента списку (замість лямбда-виразу).

Проте функціональні об'єкти можуть бути використані не лише як функція, але і як значення. Тобто, створивши лямбда-вираз, не обов'язково його одразу "виконати" або робити функцією деякого символу (як це робить DEFUN). Створений функціональний об'єкт може бути зв'язаний із символом як *значення*:

```
CL-USER> (let ((foo (lambda (x) x)))
            (format t "value: ~A~%" foo)
            (format t "call: ~A~%" (foo 1)))
; in: LET ((FOO (LAMBDA (X) X)))
; (FOO 1)
;
; caught STYLE-WARNING:
;   undefined function: COMMON-LISP-USER::FOO
;
; compilation unit finished
;   Undefined function:
;     FOO
;   caught 1 STYLE-WARNING condition
value: #<FUNCTION (LAMBDA (X)) {536598EB}> ; вивід значення
; Evaluation aborted on #<UNDEFINED-FUNCTION FOO {10019A7763}>.
```

З прикладу видно, що функціональний об'єкт (`#<FUNCTION...>`) є саме значенням символу `foo`. Тому викликати `foo` як звичайну іменовану функцію, записавши `foo` першим елементом списку, — не можна.

Оскільки функціональний об'єкт може бути значенням, то і працювати з ним можна як зі значенням. Наприклад, зв'язати його з локальною змінною (як в прикладі вище), або передати його в якості фактичного параметра в іншу функцію. Функції, що приймають інші функції в якості параметрів, називаються **функціями вищого порядку** або **функціоналами** [36].

4.1.1 Отримання функціонального об'єкта

Для того, щоб отримати функціональний об'єкт, можна його створити за допомогою лямбда-виразу або ж, якщо мова йде про іменовані функції, використавши спеціальний оператор `FUNCTION`, який також записується як `#'` перед формою (аналогічно `QUOTE` і `'`). При цьому `#'` можна застосовувати не лише до символів, але й до деяких форм, як той самий лямбда-вираз:

```
CL-USER> (function list)
#<FUNCTION LIST>
CL-USER> #'list
#<FUNCTION LIST>
CL-USER> #'(lambda (x) x)
#<FUNCTION (LAMBDA (X)) {5365A22B}>
CL-USER> (lambda (x) x)
#<FUNCTION (LAMBDA (X)) {5365A2BB}>
```

Як видно з останньої команди, `lambda` повертає функціональний об'єкт і без `#'`. Це пов'язано з тим, що в SBCL `lambda` — це макрос, який макро-розширюється у виклик `FUNCTION` для форми з символом `lambda`:

```
CL-USER> (macroexpand-1 '(lambda (x) x))
#'(LAMBDA (X) X)
T
```

Тому при передачі лямбда-виразу в іншу функцію як фактичного значення не потрібно писати `#'`.

Може здатись, що `FUNCTION` повертає значення поля `SYMBOL-FUNCTION` символу (р. 2.2.1), і що замість `FUNCTION` можна застосовувати `SYMBOL-FUNCTION`. Проте це не завжди так. `SYMBOL-FUNCTION` можна використати для отримання функціонального об'єкта з символу, якщо функція визначена глобально. Для локальних функцій (р. 2.4.3) це не спрацює. Як і для чогось, крім символів (наприклад, для лямбда-виразу).

4.1.2 Виклик функціонального об'єкта

Якщо функціональний об'єкт є значенням (а не функцією) деякого символу, то як же тоді його використовувати як функцію? Себто, як його "викликати"? Для цього існують різні функції: `FUNCALL`, `APPLY` та `MULTIPLE-VALUE-CALL` (для багатьох значень — р. 2.4.4).

`FUNCALL` приймає на вхід функціональний об'єкт та довільну послідовність (0 і більше) фактичних параметрів, які необхідно передати у виклик функціонального об'єкта:

```
CL-USER> (funcall #'list 1 2 3)
(1 2 3)
CL-USER> (funcall (lambda (a b) (+ a b)) 1 2)
3
```

`APPLY` так само приймає на вхід функціональний об'єкт і довільну послідовність фактичних параметрів, які мають бути передані у виклик функціонального об'єкта. Проте фактичних параметрів має бути мінімум один. Крім цього, останній фактичний параметр має бути списком, який при виклику функціонального об'єкта буде "розгорнутий" у послідовність фактичних значень:

```
CL-USER> (apply #'min '(1 2 3 4 5)) ; (min 1 2 3 4 5)
1
CL-USER> (apply #'max 1 3 5 '(7 9)) ; (max 1 3 5 7 9)
9
CL-USER> (apply #'list 2 4 nil) ; (list 2 4)
(2 4)
```

Те, що частина (або навіть і всі) фактичних аргументів задаються списком, створює можливості для того, щоб сформувати набір фактичних параметрів для виклику програмним чином, а не визначати явно при написанні APPLY:

```
CL-USER> (defun sum (&rest args)
  (apply #' + args))
SUM
CL-USER> (sum 1 2 3 4)
10
```

Невеликий приклад: необхідно реалізувати дві функції:

- call-fn, яка приймає на вхід один функціональний об'єкт і повертає функцію, що викликає цей функціональний об'єкт для довільного набору вхідних аргументів:

```
(defun call-fn (f)
  (lambda (&rest args)
    (apply f args)))
```

- call-two-fns, що приймає на вхід два функціональних об'єкти і повертає функцію, яка, як і call-fn, викликає *останню* функцію для довільного набору вхідних аргументів, а тоді викликає першу функцію з єдиним результатом виклику другого функціонального об'єкта:

```
(defun call-two-fns (f1 f2)
  (lambda (&rest args)
    (funcall f1 (apply f2 args))))
```

Приклад застосування:

```
CL-USER> (set 'sum (call-fn #' +))
#<FUNCTION (LAMBDA (&REST ARGS) :IN CALL-FN) {10024AFD2B}>
CL-USER> (funcall sum 1 2 3 4)
```

```

10
CL-USER> (set 'double-sum (call-two-fns (lambda (s) (* s 2)) #'+))
#<FUNCTION (LAMBDA (&REST ARGS) :IN CALL-TWO-FNS) {10026CB54B}>
CL-USER> (funcall double-sum 1 2 3 4)
20

```

Ще однією формою для виклику функціонального об'єкта є спеціальний оператор MULTIPLE-VALUE-CALL, який працює аналогічно до FUNCALL, але в якості фактичних аргументів для передачі у виклик функціонального об'єкта він може отримати "багато-значні" параметри (див. р. 2.4.4), які будуть "розгорнуті" у послідовність фактичних параметрів аналогічно до того, як це робить APPLY зі своїм останнім аргументом:

```

CL-USER> (multiple-value-call #'list 1 (values) 2 (values 3 4 5) 6)
(1 2 3 4 5 6)

```

4.1.3 Стандартні функції вищого порядку

В Common Lisp є дуже багато функцій, що є функціями вищого порядку, оскільки використання таких функцій нерозривно пов'язане з функціональним програмуванням. Крім наведених в р. 4.1.2 функцій для виклику функціональних об'єктів, також функціями вищого порядку є більшість функцій для роботи з послідовностями (р. 5.1), багато функцій для роботи зі списками (р. 2.3.4) та інші.

Також до числа функціоналів можна віднести такі функції:

- CONSTANTLY приймає один аргумент `arg` на вхід і повертає функцію, що при виклику з довільним набором аргументів завжди (`constantly`) повертатиме значення `arg`;
- COMPLEMENT приймає на вхід функцію `f` і повертає функцію (`complement-f`), результатом якої буде доповнення (з теорії множин) до результату виклику `f`. Тобто, спочатку викликається `f`, а тоді `complement-f` повертає доповнення до цього результату;

- IDENTITY приймає на вхід один аргумент і повертає його ж.

Приклади їх роботи:

```
CL-USER> (mapcar #'identity '(a b c)) ; аналог (copy-list '(a b c))
(A B C)
CL-USER> (mapcar (constantly 2) '(1 2 3 4))
(2 2 2 2)
CL-USER> (mapcar #'oddp '(1 2 3))
(T NIL T)
CL-USER> (mapcar (complement #'oddp) '(1 2 3))
(NIL T NIL)
```

Тема 4.2 Лямбда-вирази

Лямбда-вираз (lambda-вираз, lambda-expression) в Lisp — це запозичення з лямбда-обчислень Черча. Лямбда-вираз має вигляд форми, яка може бути використана замість імені функції в певних місцях коду і дає змогу прямо описати поведінку замість непрямого посилання на деяку функцію за ім'ям [34]. Тобто, можна сказати, що лямбда-вираз — це неіменована або ж анонімна функція. Лямбда-вираз може бути записаний за допомогою макроса LAMBDA і має наступний синтаксис:

```
<lambda-expression> := (lambda (<args>) <body>)
<args> := s-expr* ; формальні параметри лямбда-виразу
<body> := s-expr* ; тіло лямбда-виразу, що складається з Lisp-форм
```

Зазвичай формальні параметри мають вигляд послідовності символів. Крім звичайних або ж *обов'язкових* параметрів, функція може мати *спеціальні* формальні параметри, які розглядаються в розділі 2.4.2.

Наприклад, ось визначення lambda-виразу, що приймає на вхід два аргументи, з яких створює список:

```
(lambda (x y) (list x y))
```

Застосування lambda-виразу має наступний вигляд:

```
(<lambda-expression> <actual-args>)  
<actual-args> := s-expr* ; фактичні параметри
```

Фактичні параметри обчислюються інтерпретатором і отримані значення зв'язуються з відповідними формальними параметрами лямбда-виразу, який до них "прикладається" (applied). Після зв'язування відбувається так зване *лямбда-перетворення*, тобто виконання форм, які складають тіло лямбда-виразу. Результатом лямбда-перетворення буде результат виконання останньої форми з тіла.

Наприклад:

```
CL-USER> ((lambda (x y) (list x y)) 1 2)  
(1 2)
```

Що також еквівалентно цьому:

```
CL-USER> ((lambda (x)  
            ((lambda (y)  
              (list x y))  
              2))  
          1)  
(1 2)
```

Тема 4.3 Замикання

Замиканням в програмуванні називається функція і контекст її визначення. Під "контекстом" маються на увазі вільні змінні, які визначені за межами функції, але використовуються в ній. Простий приклад визначення замикання такий:

```
CL-USER> (let ((b 5)
               (c 10))
           (defun adder (arg)
             (+ arg b c)))
ADDER
CL-USER> (adder 5)
20
```

Функція `adder` визначена в контексті, де є лексично зв'язані змінні `b` та `c`. Оскільки змінні визначені за межами функції, але при цьому використовуються в ній, в тілі функції вони є *вільними змінними*. Після створення функції і завершення виконання форми `let`, пам'ять для цих лексично зв'язаних змінних не звільняється, оскільки вони є частиною *контексту* функції, яка разом з цим контекстом і утворює *замикання*.

Зазвичай замикання асоціюють саме з функціональними мовами програмування. Вони можуть використовуватись у *локальних функціях* (р. 2.4.3) для того, щоб реалізувати якісь повторювані фрагменти алгоритму і при цьому мати доступ до локальних змінних *зовнішньої* функції. Інший приклад застосування замикань — це імітація об'єктів, коли визначаються "методи", що працюють з "полями" класу:

```
CL-USER> (defun create-linear-function (k b)
           (lambda (x)
             (+ (* k x) b)))
CREATE-LINEAR-FUNCTION
CL-USER> (setf (symbol-function 'foo) (create-linear-function 2 5))
#<FUNCTION (LAMBDA (X) :IN CREATE-LINEAR-FUNCTION) {10029EAE2B}>
CL-USER> (foo 4)
13
```

У наведеному прикладі `k` і `b` можна вважати константними "полями", значення яких використовуються щоразу, коли викликається створений `lambda`-вираз ("метод").

Інший приклад: необхідно реалізувати функцію, що створить функції, що імітують `set/get` методи для поля об'єкта (класу). Реалізація така:

```
(defun construct-person (name)
  (values
    (lambda (new-name)
      "Set `new-name' to `name'"
      (setq name new-name))

    (lambda ()
      "Get `name'"
      name)))
```

Приклад застосування:

```
CL-USER> (multiple-value-bind (set-name-fn get-name-fn)
           (construct-person "Paul")
           (format t "Name: ~A~%" (funcall get-name-fn))
           (funcall set-name-fn "Ted")
           (format t "New name: ~A~%" (funcall get-name-fn)))
Name: Paul
New name: Ted
NIL
```

РОЗДІЛ 5

ІНШІ МОЖЛИВОСТІ МОВИ COMMON LISP

Тема 5.1 Послідовності, масиви

Послідовностями в Common Lisp називаються впорядковані колекції елементів, до яких відносяться списки, масиви та рядки (оскільки рядки реалізовані на базі масивів). Крім специфічних для певного типу функцій (як от функції для роботи виключно зі списками, що розглядаються в розділах 2.1.9 та 2.3.4) в стандартній бібліотеці Common Lisp визначені функції, що можуть працювати з будь-якими послідовностями [40].

Функції для роботи з послідовностями можна умовно поділити на 8 груп:

1. Функції отримання елементів з послідовностей.
2. Функція копіювання послідовності.
3. Функції об'єднання послідовностей.
4. Функції аналізу послідовностей.
5. Функції пошуку елементів у послідовностях.
6. Конструктивні функції зміни послідовностей.
7. Деструктивні функції зміни послідовностей.
8. Функція трансформації послідовності.

Деякі з функцій для роботи з послідовностей мають своїх "дублерів" для роботи з конкретним типом послідовності. При роботі лише з одним типом послідовності варто застосовувати функції для роботи саме з цим типом — якщо такі функції існують. Наприклад, при роботі виключно зі списками для отримання елемента за індексом варто використовувати `NTN`, а не `ELT`.

Багато функцій, що працюють із послідовностями, крім основних аргументів мають також спеціальні аргументи. Часто у них є ключові параметри `key`, `test`, `start` та інші, визначені в таблиці 5.1, і які реалізують однакову логіку в усіх функціях, в яких вони визначені.

Таблиця 5.1 — Ключові параметри функцій для роботи з послідовностями

Назва параметра	Призначення	Значення за замовчуванням
<code>:key</code>	Функція з одним аргументом, яка застосовується на елементі послідовності, перш ніж елемент буде порівняно з шуканим чи передано у предикат для виконання перевірки. Значення <code>NIL</code> вказує на те, що з елементом не потрібно щось робити перед виконанням порівнянь	<code>NIL</code>
<code>:test</code>	Функція з двома аргументами, яка виконує їхнє порівняння	<code>EQL</code>
<code>:test-not</code>	Функція з двома аргументами, яка виконує їхнє порівняння. Результат цієї функції інтерпретується як обернений ("збігається, якщо <code>NIL</code> "). Значення <code>NIL</code> вказує на те, що таку перевірку не потрібно застосовувати	<code>NIL</code>
<code>:start</code>	Індекс, з якого необхідно починати пошук в послідовності	<code>0</code>
<code>:end</code>	Індекс, перед яким необхідно припинити пошук в послідовності. <code>NIL</code> означає, що послідовність перевіряється до кінця	<code>NIL</code>
<code>:from-end</code>	Прапорець, що визначає, чи потрібно обходити послідовність з кінця	<code>NIL</code>
<code>:count</code>	Кількість елементів, які необхідно видалити чи замінити. Значення <code>NIL</code> означає, що кількість не обмежена	<code>NIL</code>

Деякі функції для роботи з послідовностями мають варіації, що замість елемента, який необхідно знайти/видалити/тощо у послідовності, отримують на вхід предикат (функцію), який викликається на елементах послідовності і визначає чи поточний елемент відповідає деякій умові — для виконання відповідної дії (повернення елемента, його видалення, тощо). Такі функції закінчуються суфіксами -IF та -IF-NOT (для перевірки справдження умови в предикаті, або ж несправдження відповідно). Прикладом функцій, які мають такі варіації є COUNT, FIND, POSITION, REMOVE та інші. (Див. HyperSpec [17].)

5.1.1 Функції отримання елементів з послідовностей

До цієї групи належать дві функції: ELT та SUBSEQ. ELT дає змогу отримати елемент послідовності за індексом, в той час як з SUBSEQ можна отримати підпослідовність елементів у заданому проміжку. Формат їхнього використання такий:

```
(elt sequence index)
(subseq sequence start &optional end)
```

sequence — послідовність; index, start, end — індекси елемента у послідовності, початку і кінця (не включно) підпослідовності відповідно.

```
CL-USER> (elt '(1 2 3 4 5) 1)
2
CL-USER> (subseq '(0 1 2 3) 1)
(1 2 3)
CL-USER> (subseq '(0 1 2 3) 1 3)
(1 2)
CL-USER> (subseq "Big bang theory" 4 8)
"bang"
```

5.1.2 Функція копіювання послідовності

Для того, щоб отримати копію послідовності, можна скористатись функцією COPY-SEQ. Вона приймає на вхід один аргумент (послідовність) і повертає його копію.

```
CL-USER> (set 'lst '(1 2 3))  
(1 2 3)  
CL-USER> (set 'lst-copy (copy-seq lst))  
(1 2 3)  
CL-USER> (eq lst lst-copy)  
NIL
```

5.1.3 Функції об'єднання послідовностей

Для об'єднання послідовностей можна використати функції:

- **CONCATENATE** об'єднує кілька послідовностей-аргументів за рахунок їх повного або часткового копіювання і "з'єднання" в одну послідовність. Аналогічною функцією для об'єднання списків є **APPEND**;
- **MERGE** поєднує дві (скоріше за все впорядковані) послідовності по чергово порівнюючи елементи з обох і формуючи нову (впорядковану) послідовність;
- **MAP** виконує складання багатьох послідовностей в одну, перетворюючи елементи з багатьох послідовностей, що розміщені на однакових позиціях, в один. Це є більш загальна версія функції для роботи зі списками **MAPCAR**.

Послідовності-аргументи цих функцій можуть бути різних підтипів послідовностей (списки, рядки, вектори), а тип результату цих функцій визначається явно першим аргументом. Синтаксис їхнього використання такий:

```
(concatenate result-type &rest sequences)  
(merge result-type sequence1 sequence2 &key key)  
(map result-type function first-sequence &rest more-sequences)
```

`result-type` — символ, що позначає тип результату (`list`, `vector`, `string`);
`sequences` — послідовності, які необхідно об'єднати (можуть бути різних типів);
`function` — функція, що приймає стільки аргументів, скільки послідовностей є аргументами `map`.

Приклад застосування:

```
CL-USER> (concatenate 'list "abc" '(1 2 3))
(#\a #\b #\c 1 2 3)
CL-USER> (concatenate 'vector "abc" '(1 2 3))
#(#\a #\b #\c 1 2 3)
CL-USER> (concatenate 'string "abc" '(\1 \2 \3))
"abc123"
CL-USER> (merge 'list '(1 2 3) '(0 2 4 6) #'<)
(0 1 2 2 3 4 6)
CL-USER> (merge 'list '(-3 -1 2 3) '(0 2 4 6) #'> :key #'abs)
(-3 -1 2 3 0 2 4 6)
CL-USER> (merge 'list '("a" "bar" "car") '("bucket" "pie") #'string>)
("bucket" "pie" "a" "bar" "car")
CL-USER> (merge 'list '("a" "bar" "car") '("bucket" "pie") #'< :key #'length)
("a" "bar" "car" "bucket" "pie")
CL-USER> (map 'list #'list '(1 2 3) "abcd")
((1 #\a) (2 #\b) (3 #\c))
CL-USER> (map 'list (lambda (k b) (+ (* k 2) b)) '(1 2 3) '(3 4 5))
(5 8 11)
CL-USER> (map 'list (lambda (k x b) (+ (* k x) b)) '(1 2 3) '(0 2 3) '(3 4 5))
(3 8 14)
```

5.1.4 Функції аналізу послідовностей

До функцій, за допомогою яких можна отримати інформацію про послідовність загалом, за рахунок її аналізу, відносяться такі функції, як `LENGTH`, `COUNT`, а також предикати `EVERY`, `NOTEVERY` та `NOTANY`.

5.1.4.1 *LENGTH*

`LENGTH` визначає довжину свого єдиного аргумента — послідовності.

```
CL-USER> (length nil)
0
```

```
CL-USER> (length '(a b c d))
4
CL-USER> (length "Hello!")
6
```

5.1.4.2 COUNT, COUNT-IF та COUNT-IF-NOT

Функція COUNT рахує кількість входжень заданого елемента у послідовність. В стандартній бібліотеці є її варіації, що працюють з предикатами замість елемента. Їх синтаксис такий:

```
(count item sequence
  &rest args
  &key from-end (start 0) end key (test #'eql) test-not)
(count-if predicate sequence
  &rest args
  &key from-end (start 0) end key)
(count-if-not predicate sequence
  &rest args
  &key from-end (start 0) end key)
```

`item` — елемент, кількість входжень якого треба порахувати;

`predicate` — предикат, за допомогою якого перевіряється елемент і робиться висновок щодо того, чи треба його рахувати.

Приклади використання функцій COUNT*:

```
CL-USER> (count 1 '(1 2 3 4 1.0))
1
CL-USER> (count 1 '(1 2 3 4 1.0 1e0) :test #'=)
3
CL-USER> (count-if #'zerop '(1 2 3 4 5 6 7)
              :key (lambda (x) (truncate x 5))
              :start 2)
2
CL-USER> (count-if-not #'zerop '(1 2 3 4 5 6 7)
              :key (lambda (x) (truncate x 5))
              :start 2)
3
```

5.1.4.3 Предикати *EVERY*, *NOTEVERY* та *NOTANY*

Предикати *EVERY*, *NOTEVERY* та *NOTANY* дають отримати відповідь на питання:

- *EVERY*: чи всі елементи послідовності відповідають умові?
- *NOTEVERY*: чи є в послідовності такий елемент, що не відповідає умові?
- *NOTANY*: чи для всіх елементів послідовності не справджується умова?

Синтаксис їхнього запису такий:

```
(every pred first-seq &rest more-seqs)
(notevery pred first-seq &rest more-seqs)
(notany pred first-seq &rest more-seqs)
```

pred — предикат, який послідовно викликається на кожному елементів послідовності;

first-seq — перша послідовність (одна має бути обов'язково);

more-seqs — додаткові послідовності.

Предикат має бути функцією, що приймати на вхід стільки аргументів, скільки всього послідовностей передано у *every*/*notevery*/*notany*.

Приклад використання:

```
CL-USER> (every #'evenp '(2 4 6))
T
CL-USER> (every #'evenp '(1 2 3))
NIL
CL-USER> (notevery #'< '(1 2 3) '(2 3 4))
NIL
CL-USER> (notevery #'< '(1 2 3) '(2 2 4))
T
CL-USER> (notany (lambda (&rest args)
                    (>= (apply #'+ args) 10))
           '(1 2 3)
           '(2 2 4)
           '(0 2 0))
T
```

```
CL-USER> (notany (lambda (&rest args)
                  (>= (apply #' + args) 10))
          '(1 2 3)
          '(2 2 4)
          '(0 2 5))
NIL
```

5.1.5 Функції пошуку елементів у послідовностях

До цієї групи відносяться такі функції: FIND, POSITION, SOME, SEARCH, MISMATCH.

5.1.5.1 FIND, FIND-IF та FIND-IF-NOT

Функція FIND (та її варіації FIND-IF і FIND-IF-NOT) працює подібно того, як працює функція MEMBER (р. 2.3.4.1). Відмінністю є те, що в якості результату FIND повертає знайдений елемент, а також має більший набір ключових аргументів. Інтерфейс функцій FIND такий:

```
(find item sequence
      &rest args
      &key from-end (start 0) end key (test #'eql) test-not)
(find-if predicate sequence
          &rest args
          &key from-end (start 0) end key)
(find-if-not predicate sequence
              &rest args
              &key from-end (start 0) end key)
```

`item` — це шуканий елемент;

`sequence` — послідовність, в якій відбувається пошук;

`predicate` — предикат, за допомогою якого перевіряється елемент і робиться висновок щодо того, чи це шуканий елемент.

Приклади використання:

```
CL-USER> (find 'a '(1 a 2))
A
CL-USER> (find '(1) '((1) 2 3))
```

```

NIL
CL-USER> (find 5 '(1 2 3 4 5) :from-end t :end 4 :test #'<=)
NIL
CL-USER> (find 5 '(1 2 3 4 5) :from-end t :end 4 :test #'>=)
4
CL-USER> (find-if (lambda (elem) (and (listp elem) (>= (list-length elem) 2)))
              '(1 nil (2) (3 4) (5 6 7))
              :from-end t)
(5 6 7)
CL-USER> (find-if (lambda (elem) (and (listp elem) (>= (list-length elem) 2)))
              '(1 nil (2) (3 4) (5 6 7))
              :from-end nil)
(3 4)
CL-USER> (find-if-not #'oddp '(0 2 4 6))
0
CL-USER> (find-if-not #'evenp '(0 2 4 6))
NIL

```

Оскільки FIND повертає сам шуканий елемент або NIL, якщо елемент не знайдено, може виникнути проблема, якщо шуканим елементом є сам NIL:

```

CL-USER> (find nil '(1 2 3))
NIL
CL-USER> (find nil '(nil t))
NIL

```

Щоб такого не сталося, у випадку, якщо шуканим елементом є NIL, варто застосувати функцію MEMBER або інші функції для пошуку елемента в послідовності, що не повертають сам шуканий елемент (напр. POSITION).

5.1.5.2 POSITION, POSITION-IF, POSITION-IF-NOT

Функція POSITION та її варіації за -IF та -IF-NOT працюють аналогічно FIND, проте замість самого шуканого елемента повертають його позицію. Синтаксис їх наступний:

```

(position item sequence
          &rest args
          &key from-end start end key test test-not)
(position-if predicate sequence
            &rest args
            &key from-end start end key)
(position-if-not predicate sequence
                &rest args
                &key from-end start end key)

```

```
&rest args  
&key from-end start end key)
```

Приклади використання:

```
CL-USER> (position 'a '(1 a 2))  
1  
CL-USER> (position '(1) '((1) 2 3))  
NIL
```

Оскільки результатом є позиція, проблем із пошуком NIL не виникає (я у FIND):

```
CL-USER> (position nil '(1 nil 2))  
1
```

5.1.5.3 SOME

Функцію SOME можна віднести як до функцій, що шукають елемент, так і до функцій, що аналізують послідовність: вона шукає перший елемент, який задовольняє деякому предикату (тобто визначає, чи є в послідовності елемент, що задовольняє предикату) і повертає результат роботи предиката (на відміну від того ж FIND). Її інтерфейс такий:

```
(some pred first-seq &rest more-seqs)
```

pred — це предикат, який має приймає на вхід стільки аргументів, скільки послідовностей передано у some, і повертає значення, яке, якщо воно відмінне від NIL, визначає, що знайдено той самий "деякий" ("some") елемент.

Приклад використання:

```
CL-USER> (some #'first '(() (nil) ()))
```

```
NIL ; це індикатор того, що елемент не знайдено, а не `NIL` з підписку  
CL-USER> (some #'first '(() (nil) (1 2) ()))  
1
```

5.1.5.4 SEARCH

Функція SEARCH шукає входження однієї послідовності в іншу і повертає позицію такого входження.

5.1.5.5 MISMATCH

Функція MISMATCH є протилежністю SEARCH: вона шукає місце невідповідності однієї послідовності іншій.

5.1.6 Конструктивні функції зміни послідовностей

До цієї групи відносяться такі функції: REVERSE, REMOVE, SUBSTITUTE, REMOVE-DUPPLICATES.

5.1.6.1 REVERSE

Функція REVERSE приймає на вхід одну послідовність і обертає її у зворотному порядку, створюючи при цьому нову послідовність того ж типу.

```
CL-USER> (reverse '(a b c d e))  
(E D C B A)  
CL-USER> (reverse "abcde")  
"edcba"
```

5.1.6.2 REMOVE, REMOVE-IF та REMOVE-IF-NOT

Функція REMOVE видаляє заданий об'єкт з послідовності, створюючи нову послідовність. Функції REMOVE-IF та REMOVE-IF-NOT видаляють елемент, якщо справджується умова, задана предикатом. Їх інтерфейс такий:

```
(remove item sequence
```

```

    &rest args
    &key from-end (test #'eql) test-not (start 0) end count key)
(remove-if predicate sequence
  &rest args
  &key from-end (start 0) end count key)
(remove-if-not predicate sequence
  &rest args
  &key from-end (start 0) end count key)

```

Приклад застосування:

```

CL-USER> (remove 'a '(1 a 2))
(1 2)
CL-USER> (remove '(1) '((1) 2 3))
((1) 2 3)
CL-USER> (remove '(1) '((1) 2 3) :test #'equal)
(2 3)
CL-USER> (remove '(1) '((1) 2 (1) 3 (1)) :test #'equal :count 2)
(2 3 (1))

```

5.1.6.3 SUBSTITUTE, SUBSTITUTE-IF та SUBSTITUTE-IF-NOT

SUBSTITUTE і її варіації виконують заміну заданого об'єкта у послідовності на інший, конструюючи (в разі заміни) нову послідовність того ж типу.

5.1.6.4 REMOVE-DUPPLICATES

Функція REMOVE-DUPPLICATES видаляє всі дублікати елементів з послідовності.

5.1.7 Деструктивні функції зміни послідовностей

До цієї групи відносяться такі функції: FILL, NREVERSE, DELETE, NSUBSTITUTE, SORT, REPLACE.

Функція FILL виконує деструктивну заміну елементів послідовності на заданий елемент у визначених межах.

Функції NREVERSE, DELETE (а також DELETE-IF, DELETE-IF-NOT), NSUBSTITUTE (та її варіації) є деструктивними реалізаціями функцій REVERSE, REMOVE та SUBSTITUTE.

Деструктивна функція SORT виконує сортування елементів послідовності "на місці" (тобто без створення нової послідовності). Її інтерфейс такий:

```
(sort sequence predicate &rest args &key key)
```

sequence — послідовність, елементи якої треба відсортувати;

predicate — предикат для порівняння двох елементів.

REPLACE виконує деструктивну заміну підпослідовності елементів однієї послідовності на підпослідовність елементів іншої підпослідовності.

5.1.8 Функція перетворення послідовності

Функцією перетворення послідовності є функція REDUCE. Назва функції походить від "reduce" — "зменшувати" — адже функція зазвичай використовується для зменшення або ж згортання послідовностей. Проте, насправді, її можна застосовувати не лише для згортання, а ще й для інших перетворень, зокрема й для розширення послідовності. Інтерфейс функції такий:

```
(reduce function sequence  
      &rest args  
      &key key from-end (start 0) end initial-value)
```

REDUCE почергово викликає функцію function для елементів послідовності sequence, і передає у function два значення: поточний елемент списку і *акумулятор*. **Акумулятор** — це накопичене або ж акумульоване значення, що є результатом попереднього виклику function. Тобто, акумулятор — це значення, що передається між викликами function і таким чином "пов'язує" всі елементи

послідовності. Результатом виконання REDUCE буде останнє значення акумулятора. Тобто, REDUCE за допомогою function може зменшити/згорнути послідовність у якусь одне значення, або змінити на іншу послідовність (відфільтрувати, розширити і т. і.).

Порядок передачі поточного елемента списку і акумулятора у function залежить від значення ключового параметра from-end: якщо воно NIL, тоді першим параметром в function передається акумулятор, а другим — елемент послідовності; якщо не NIL — тоді навпаки.

Початковим значенням акумулятора стає значення, передане з ключовим параметром initial-value — якщо воно передане. Якщо значення не передане, тоді початковим значенням акумулятора стає перший або останній (в залежності від значення from-end) елемент послідовності.

Приклади застосування:

```
CL-USER> (reduce #'* '(1 2 3 4 5))
120
CL-USER> (reduce #'append
                 '((1) (2))
                 :initial-value '(i n i t))
(I N I T 1 2)
CL-USER> (reduce #'append
                 '((1) (2))
                 :initial-value '(i n i t)
                 :from-end t)
(1 2 I N I T)
CL-USER> (reduce #'cons
                 '((1) (2))
                 :key #'car
                 :initial-value '(i n i t)
                 :from-end t)
(1 2 I N I T)
CL-USER> (reduce (lambda (elem acc)
                  (if (evenp elem)
                      (cons (1+ elem) acc)
                      acc))
              '(1 3 6 10 15)
              :initial-value nil
              :from-end t)
(7 11)
```

5.1.9 Масиви

В Common Lisp крім списків є також і інші типи послідовностей або колекцій. Одним з таких типів є **масив**. Масив — це послідовність значень одного типу, що розміщуються у пам'яті послідовно, за рахунок чого до його елементів можна звертатись за індексом. В Common Lisp розрізняють два види масивів:

- загальні — можуть містити елементи будь-якого типу (тип `t`)
- спеціалізовані — можуть містити елементи одного визначеного типу

Часто для забезпечення швидкодії доступу до елементів колекції/послідовності саме використання масивів може бути кращим рішенням, ніж використання списків. Хоча при цьому зміна вмісту комірок масиву вимагає використання псевдо-функцій, а розширення масиву — його копіювання.

Масиви можуть бути 0-, одно- та багатовимірні. 0-вимірні масиви — це просто один об'єкт. Одновимірні масиви називаються **векторами** (`vector`).

5.1.9.1 Створення масиву

Для того, щоб створити масив, можна використати декілька підходів:

1. Використати функцію `MAKE-SEQUENCE` з вказанням типу результату `vector`.
2. Використати функцію `MAKE-ARRAY`.
3. Використати *рідер-макро*: `#( ... )`, `#2A( ... )` та ін.
4. За допомогою функції `VECTOR` та ін.

Функція `MAKE-ARRAY` має наступний синтаксис:

```
(make-array dimensions &key element-type initial-element ...)  
dimensions := number | (number*) ; "вимірність" масиву  
element-type := symbol ; тип елементів масиву  
initial-element := form ; значення, яким необхідно заповнити створений масив
```

Функція MAKE-ARRAY є найбільш універсальною функцією для створення масивів. Крім визначення розмірності масиву, за допомогою неї можна визначити тип елементів масиву (параметр `element-type`), початкове значення, яким необхідно заповнити створений масив (параметр `initial-element`) та інше (див. HyperSpec [17]). Якщо тип елементів масиву не зазначений, тоді за замовчуванням використовується тип `t` — тобто "будь-який" (тобто, за замовчуванням створюється загальний масив).

Вимірність масиву (`dimensions`) задається числом, що вказує на те, що це має бути одновимірний масив, або списком чисел, що вказує на вимірність та величину кожного виміру масиву. Якщо ж в якості першого параметру в MAKE-ARRAY передати `nil`, тоді створений масив буде 0-вимірним — тобто це буде просто один об'єкт.

Приклади застосування MAKE-ARRAY:

```
CL-USER> (make-array 1)
#(0)
CL-USER> (make-array 0)
#()
CL-USER> (make-array '())
#0A0
CL-USER> (make-array '(2 2))
#2A((0 0) (0 0))
CL-USER> (make-array '(2 2) :initial-element 5)
#2A((5 5) (5 5))
CL-USER> (make-array '(2) :element-type 'character :initial-element #\a)
"aa"
```

Для створення масиву можна також використати спеціальний *рідер-макро*:

```
#<n-dimensions>A<object>
<n-dimensions> := number ; вимірність масиву
<object> := object | list ; об'єкт 0-вимірного масиву або список, що
                        ; визначає значення елементів масиву
```

В залежності від вимірності створюваного масиву (якщо вона більша за 0), на місці `<object>` має бути список з вкладеними списками (в разі, якщо розмірність більша з 1), який задає вміст створюваного масиву. При цьому

кількість елементів у підписах на відповідних рівнях (у відповідних вимірах) має бути однаковою, інакше буде видано повідомлення про помилку.

Якщо масив одновимірний — тоді це просто список, якщо двовимірний — тоді список з підсписками, і т. д. Одновимірні масиви (*вектори*) можна записувати у спрощеній формі:

```
#(...) <--> #1A(...) ; аналогічні записи для створення вектора
```

Приклади створення масиву за допомогою рідер-макро:

```
CL-USER> #0A5
#0A5
CL-USER> #1A(1 2 3)
#(1 2 3)
CL-USER> #(1 2 3)
#(1 2 3)
CL-USER> #2A((1 a) (2 b))
#2A((1 A) (2 B))
CL-USER> #3A(((5 4) (7 6)) ((1 2) (3 4)))
#3A(((5 4) (7 6)) ((1 2) (3 4)))
CL-USER> #3A(((5 4) (3)) ((1 2)))
; malformed :INITIAL-CONTENTS: Dimension of axis 2 is 2, but (3) is 1 long.
; Evaluation aborted on #<SIMPLE-ERROR "malformed :INITIAL-CONTENTS:
; Dimension of axis ~W is ~W, but ~S is ~W long." {1002C1F5E3}>.
```

Функція VECTOR створює масив загального виду з усіх "решти" елементів, що отримує на вхід (тобто, працює аналогічно LIST):

```
CL-USER> (vector)
#()
CL-USER> (vector 1 2 'a 'b)
#(1 2 A B)
```

5.1.9.2 Доступ до елементів з масиву

Для того, щоб отримати елемент з масиву, можна використати функцію для роботи з послідовностями ELT або ж специфічну саме для масивів функцію AREF.

Вона приймає на вхід два аргументи — масив та послідовність індексів, за якими треба досягнути елементу масиву. При цьому досягнути необхідно саме до елемента, а не підмасиву, інакше виникне помилка.

```
CL-USER> (aref #0A1)
1
CL-USER> (aref #(1 2 3 4) 2)
3
CL-USER> (aref #2A((1 2) (3 4)) 0 1)
2
CL-USER> (aref #2A((1 2) (3 4)) 0)
; ERROR: Wrong number of subscripts, 1, for array of rank 2.
```

5.1.9.3 Встановлення значень у масив

Для встановлення значення у масив, можна скористатись комбінацією SETF + AREF:

```
CL-USER> (set 'arr #(0 1 2 3 4))
#(0 1 2 3 4)
CL-USER> (setf (aref arr 1) 5)
5
CL-USER> arr
#(0 5 2 3 4)
CL-USER> (set 'arr2 #2A((0 1 2) (3 4 5)))
#2A((0 1 2) (3 4 5))
CL-USER> (setf (aref arr2 1 0) 7)
7
CL-USER> arr2
#2A((0 1 2) (7 4 5))
```

5.1.10 Завдання на СРС

1. Написати функцію `my-find` для списків, яка працює аналогічно стандартній функції `find`, проте має менший набір ключових параметрів. Інтерфейс функції наступний:

```
(defun my-find (item list &key (start 0) end from-end) ...)
```

2. Написати функцію `my-remove` для списків, яка працює аналогічно стандартній функції `remove`, проте має менший набір ключових параметрів. Інтерфейс функції наступний:

```
(defun my-remove (item list &key (start 0) end from-end) ...)
```

3. Написати функцію `find-best-element`, яка визначає "найкращий" елемент послідовності згідно заданому предикату `pred`, який порівнює два елементи:

```
(defun find-best-element (list pred &key key) ...)
```

Приклад:

```
CL-USER> (find-best-element '((1 2 3) (1 2) () (1 2 3 4)) #'> :key #'length)
(1 2 3 4)
```

Тема 5.2 Рядки, рядкові символи

Рядками в Common Lisp є спеціалізовані масиви, елементами яких є *рядкові символи*. Тому всі функції, які працюють з масивами, також працюють і з рядками. Рядки записуються як послідовність рядкових символів у *лапках* (double quote):

```
"abc123"  
"Hello world!"
```

Рядкові символи (characters) — це об'єкти, що представляють символи (унітарні лексеми — літери, цифри, спеціальні символи), що використовуються у тексті. Рядкові символи записуються з префіксом `#\`, за яким знаходиться сам

рядковий символ (якщо він має графічне представлення), або його назва (якщо він не має графічного представлення та/або якщо він має назву).

```
#\r
#\2
#\ ; пробіл
#\Space ; також пробіл
#\NewLine
```

5.2.1 Робота з рядковими символами

Для того, щоб отримати рядковий символ, можна записати його у вигляді літерала (з префіксом #\). А якщо необхідно отримати символ з рядка, тоді можна скористатись функцією CHAR (що є аналогом AREF, ELT для рядків). Наприклад:

```
CL-USER> #\f
#\f
CL-USER> #\F
#\F
CL-USER> #\;
#\;
CL-USER> (char "Break a leg!" 3)
#\a
CL-USER> (char "Break a leg!" 4)
#\k
```

5.2.1.1 Створення і перетворення рядкових символів

Кодування рядкових символів залежить від реалізації інтерпретатору Lisp. Тим не менш, типовим є використання UNICODE. Отримати код рядкового символу можна використавши функцію CHAR-CODE. І навпаки, щоб отримати рядковий символ по коду, використовується функція CODE-CHAR. Обидві функції приймають лише по одному аргументу — рядковий символ і код (число) відповідно:

```
(char-code character)
```

(code-char code)

Приклади використання:

```
CL-USER> (char-code #\a)
97
CL-USER> (char-code #\Space)
32
CL-USER> (char-code #\ )
32
CL-USER> (char-code #\e)
1108
CL-USER> (code-char 97)
#\a
CL-USER> (code-char 32)
#\
CL-USER> (code-char 1028)
#\CYRILLIC_CAPITAL_LETTER_UKRAINIAN_IE
CL-USER> (char-code #\i)
1111
CL-USER> (code-char 1111)
#\CYRILLIC_SMALL_LETTER_YI
```

5.2.1.2 Порівняння рядкових символів

Для порівняння рядкових символів можна використовувати загальні предикати EQL, EQUAL та EQUALP (див. р. 2.1.6.3). Перші два порівнюють рядкові символи зважаючи на регістр літер, а EQUALP порівнює без врахування регістру літер.

Крім загальних предикатів, що порівнюють два об'єкти на рівність, є також функції, що порівнюють виключно символи та функції, що виконують інші види порівнянь, аналогічні функціям порівняння чисел. Для регістро-чутливого порівняння і регістро-нечутливого порівняння є окремі набори функції. Весь перелік цих функції наведено в таблиці 5.2.

Таблиця 5.2 — Функції для порівняння рядкових символів

Числа	Регустро-чутливе порівняння рядкових символів	Регістро-нечутливе порівняння рядкових символів
=	CHAR=	CHAR-EQUAL
/=	CHAR/=	CHAR-NOT-EQUAL
>	CHAR>	CHAR-GREATERP
<	CHAR<	CHAR-LESSP
>=	CHAR>=	CHAR-NOT-LESSP
<=	CHAR<=	CHAR-NOT-GREATERP

Всі функції для порівняння рядкових символів приймають довільну кількість аргументів (один і більше) й порівнюють їх за таким самим принципом, що й аналогічні функції для порівняння чисел (див. р. 2.1.7.2).

5.2.1.3 Інші предикати для рядкових символів

Крім порівняння двох рядкових символів символів між собою, є також і інші предикати, що дають змогу перевірити ту чи іншу особливість символу. До таких предикатів відносяться:

- CHARACTERP перевіряє, чи є аргумент рядковим символом;
- ALPHA-CHAR-P перевіряє, чи є рядковий символ літерою;
- DIGIT-CHAR-P перевіряє, чи є рядковий символ цифрою. Другим аргументом опційно приймає основу числа (radix), що за замовчуванням рівна 10. Результатом функції є "вага" — індекс цифри в залежності від основи;
- ALPHANUMERICP поєднує ALPHA-CHAR-P та DIGIT-CHAR-P;
- GRAPHIC-CHAR-P перевіряє, чи є рядковий символ графічним — тобто чи має він графічне представлення;
- та ін.

Деякі приклади:

```
CL-USER> (characterp #\X)
T
CL-USER> (alpha-char-p #\t)
T
CL-USER> (alpha-char-p #\;)
NIL
CL-USER> (digit-char-p #\5)
5
CL-USER> (digit-char-p #\B 16)
11
CL-USER> (alphanumericp #\Space)
NIL
CL-USER> (alphanumericp #\t)
T
CL-USER> (graphic-char-p #\t)
T
CL-USER> (graphic-char-p #\Newline)
NIL
```

5.2.2 Робота з рядками

Як і було зазначено, з рядками можна працювати так само, як і з масивами та послідовностями. Наприклад, щоб отримати довжину рядка, можна скористатись функцією `LENGTH`, а щоб доступитись його елемента — `ELT`, `AREF` або, найкраще, `CHAR`.

```
CL-USER> (length "12345")
5
CL-USER> (elt "12345" 3)
#\4
CL-USER> (aref "12345" 2)
#\3
CL-USER> (char "12345" 1)
#\2
```

Також можна використовувати й інші функції для роботи з послідовностями, наприклад, `CONCATENATE`, `SUSEQ`, `REVERSE`, `COUNT`, `REMOVE-IF`, `SEARCH` та інші:

```
CL-USER> (reverse "abc")
"cba"
```

```
CL-USER> (count #\t "This is a test")
2
CL-USER> (count #\t "This is a test" :test #'char=)
2
CL-USER> (count #\t "This is a test" :test #'char-equal)
3
CL-USER> (search "is" "This is a test")
2
CL-USER> (search "is" "This is a test" :start2 3)
5
```

Замінити рядкові символи у рядку можна аналогічно до того, як це робиться для масивів та послідовностей — використавши макрос SETF:

```
CL-USER> (set 'str "Handboll")
"Handboll"
CL-USER> (setf (char str 5) #\a)
#\a
CL-USER> str
"Handball"
CL-USER> (setf (subseq str 0 4) "Foot")
"Foot"
CL-USER> str
"Football"
```

5.2.2.1 Порівняння рядків

Для порівняння рядків можна використовувати функції предикати EQUAL та EQUALP (див. р. 2.1.6.3). Перший порівнює рядки зважаючи на регістр літер, а другий — не зважаючи на нього:

```
CL-USER> (equal "123" "123")
T
CL-USER> (equal "abc" "ABC")
NIL
CL-USER> (equalp "abc" "ABC")
T
```

Крім загальних предикатів, також є функції для порівняння саме рядків. Аналогічно з функціями для рядкових символів, є два набори: для регістро-

чутливого порівняння і регістро-нечутливого, які аналогічні наборам функцій для чисел (таблиця 5.3).

Таблиця 5.3 — Функції для порівняння рядків

Числа	Регустро-чутливе порівняння рядків	Регістро-нечутливе порівняння рядків
=	STRING=	STRING-EQUAL
/=	STRING/=	STRING-NOT-EQUAL
>	STRING>	STRING-GREATERP
<	STRING<	STRING-LESSP
>=	STRING>=	STRING-NOT-LESSP
<=	STRING<=	STRING-NOT-GREATERP

Всі функції для порівняння рядків порівнюють лише два рядки (основні аргументи), а також можуть приймати на вхід деякі ключові параметри, що визначають індекси початку і кінця підрядків в обох рядках, які необхідно порівняти:

- `start1` і `start2` — індекси початку в першому та другому рядках відповідно (за замовчуванням 0)
- `end1` і `end2` — індекси кінця в першому та другому рядках відповідно (за замовчуванням `nil` — тобто, "до кінця рядка")

Результатом функцій, що виконують порівняння на рівність двох рядків, є значення `T` або `NIL`. Інші функції повертають `NIL`, в разі, якщо умова порівняння не справджується, або ж індекс невідповідності у першому рядку — якщо умова справджується.

Деякі приклади:

```
CL-USER> (string= "test call" "call test")
NIL
CL-USER> (string= "test call" "call test" :start1 0 :end1 4 :start2 5 :end2 9)
T
CL-USER> (string< "1234" "1254")
2
CL-USER> (string> "1234" "1254")
NIL
```

5.2.2.2 Інші функції для роботи з рядками

Також є й інші функції, які можна застосовувати для об'єднання, розділення, перетворення з та у рядок, перетворення рядків та ін.

Функція WRITE-TO-STRING перетворює довільний об'єкт у рядок:

```
CL-USER> (write-to-string 42)
"42"
CL-USER> (write-to-string 42.15)
"42.15"
CL-USER> (write-to-string (/ 1 5))
"1/5"
CL-USER> (write-to-string (list 'a 'b 'c))
"(A B C)"
CL-USER> (write-to-string "Test write")
"\Test write\"
```

Також для перетворення об'єкта у рядок можна використати функцію форматного виводу FORMAT, передавши в якості першого аргументу nil а в якості другого — рядок з "~S":

```
CL-USER> (format nil "~S" "Test write")
"\Test write\\""
CL-USER> (format nil "~S" 42.15)
"42.15"
```

Як і в інших мовах, типовою є задача отримати число з рядка. Для цього в Common Lisp є функція PARSE-INTEGER (для розбору цілих чисел):

```
(parse-integer string &key (start 0) end (radix 10) junk-allowed)
```

start і end визначають індекси початку і кінця підрядка, який необхідно розібрати;

radix визначає основу числа (за замовчуванням 10);

junk-allowed вказує на те, чи допустимі в рядку символи, що не є цифрами (залежить також від основи числа).

PARSE-INTEGER повертає два значення: розібране число та індекс, перед яким завершився розбір числа.

Приклади:

```
CL-USER> (parse-integer "100")
100
3
CL-USER> (parse-integer "100 ")
100
4
CL-USER> (parse-integer "100s")
; Evaluation aborted on #<SB-INT:SIMPLE-PARSE-ERROR "junk in string ~S"
{10028622A3}>.
CL-USER> (parse-integer "100s" :junk-allowed t)
100
3
CL-USER> (parse-integer "s100s" :junk-allowed t)
NIL
0
CL-USER> (parse-integer "s100s" :start 1 :end 3)
10
3
CL-USER> (parse-integer "1AF" :radix 16)
431
3
```

Корисною також може бути функція для об'єднання рядків. Загалом, можна використати функцію CONCATENATE, але тоді їй треба передати ще й тип результату. А можна визначити функцію, що сама робитиме виклик CONCATENATE зі 'string в якості першого параметра:

```
(defun concat-strings (&rest strings)
  (apply #'concatenate 'string strings))

CL-USER> (concat-strings "Created" " from " "parts")
"Created from parts"
```

Інша функція, що виконує протилежну дію — розділяє рядок — відсутня в стандартній бібліотеці (принаймні, в SBCL), проте можна скористатись функцією з пакету UIOP [42], який є частиною SBCL, — `uiop:split-string`:

```
(split-string string &key max (separator '(&#\ &#\Tab)))
```

Приклади:

```
CL-USER> (uiop:split-string "Created from parts")
("Created" "from" "parts")
CL-USER> (uiop:split-string "AAA:1234:BBB:" :separator '(&#\:))
("AAA" "1234" "BBB" "")
```

Функція `COERCE` виконує перетворення об'єкта у об'єкт заданого типу. Вона може перетворювати не лише рядки. Проте може бути зручною при роботі саме з рядками, якщо є необхідність якимось чином проаналізувати рядок по-символьно (наприклад, щоб перетворити рядок у список, який потім буде використану в рекурсії).

```
(coerce object output-type-spec)
```

Приклади застосування:

```
CL-USER> (coerce "Test string" 'list)
(&#\T &#\e &#\s &#\t &#\ &#\s &#\t &#\r &#\i &#\n &#\g)
CL-USER> (coerce '(&#\H &#\e &#\l &#\l &#\o &#\!) 'string)
"Hello!"
```

Є в стандартній бібліотеці також функції для інших типових змін рядків. Як, наприклад, переведення рядка у верхній регістр (функція `STRING-UPCASE`), нижній регістр (`STRING-DOWNCASE`), у текст із великими літерами на початку слів (`STRING-CAPITALIZE`) та функція, що видаляє визначені символи з набору, якщо вони є крайніми символами у рядку (`STRING-TRIM`). Більше подібних функцій можна знайти в документації [43].

5.2.3 Завдання на СРС

1. Написати функцію, що визначить найбільший паліндром у рядку:

```
CL-USER> (greatest-palindrome "1221122")
"221122"
CL-USER> (greatest-palindrome "124424413341122")
"44244"
```

2. Написати функцію, що визначить найбільший спільний підрядок у двох рядках:

```
CL-USER> (greatest-common-substring "12cde67" "abcdefg")
"cde"
CL-USER> (greatest-common-substring "123" "abc")
""
CL-USER> (greatest-common-substring "ABABC" "ABC")
"ABC"
```

Тема 5.3 Асоціативні списки

Список асоціацій (`alist`, `association list`) або **асоціативний список** — це список, що складається зі спискових комірок (часто — точкових пар). В полях `CAR` цих комірок зберігаються *ключі*, а в полях `CDR` — значення. Якщо записати раніше розглянутий список атрибутів у вигляді списку асоціацій, то він виглядатиме ось так:

```
((a . 5) (b . 10))
```

Створити такий список можна і за допомогою базових функцій для роботи зі списками, проте в Common Lisp є й спеціалізовані функції ACONS та PAIRLIS:

```
(defun acons (key datum alist) ...)  
(defun pairlis (keys data &optional alist) ...)
```

key — це ключ, а datum — це значення, що разом у вигляді спискової комірки будуть додані до списку alist. Тобто, ACONS працює схожим чином до CONS — додає до списку один елемент. При цьому доданий елемент є списковою коміркою.

PAIRLIS попарно поєднує у спискові комірки відповідні елементи з keys та data. Якщо у цю функцію не передавати опційний параметр, то вона працює аналогічно LIST (створюючи список списків/точкових пар), а якщо передати — тоді як LIST* (додаючи створені спискові комірки перед елементами списку-аргумента). Порядок доданих спискових комірок при цьому може відрізнитись від порядку відповідних ключів і значень в keys та data.

```
CL-USER> (acons 'a 1 ())  
((A . 1))  
CL-USER> (pairlis '(a b c) '(1 2 3))  
((C . 3) (B . 2) (A . 1))  
CL-USER> (pairlis '(a b c) '(1 2 3) '((d . 4)))  
((C . 3) (B . 2) (A . 1) (D . 4))
```

Для отримання значень за ключем зі списку асоціацій використовується функція ASSOC та її варіації ASSOC-IF і ASSOC-IF-NOT (-IF та -IF-NOT працюють аналогічно іншим функціям див. р. 2.3.4.1 або р. 5.1). Ці функції повертають першу спискову комірку, поле CAR якої (ключ) відповідає заданому значенню. Формат їхнього запису такий:

```
(defun assoc (item alist &key key test test-not) ...)
(defun assoc-if (predicate alist &key key) ...)
(defun assoc-if-not (predicate alist &key key) ...)
```

item — шуканий ключ; alist — список асоціацій, в якому відбувається пошук; predicate — предикат для ключів; ключові параметри працюють так само, як і параметри, описані для послідовностей (р. 5.1).

Ці ж функції можуть відігравати роль "місць" для деструктивного встановлення значення за певним ключем.

Приклади використання:

```
CL-USER> (set 'alst '((x . 100) (y . 200) (z . 50)))
((X . 100) (Y . 200) (Z . 50))
CL-USER> (assoc 'y alst)
(Y . 200)
CL-USER> (rplacd (assoc 'y alst) 201)
(Y . 201)
CL-USER> (assoc 'y alst)
(Y . 201)
CL-USER> (assoc '(a b) '(((a b) 1) ((a) 2)))
NIL
CL-USER> (assoc '(a b) '(((a b) 1) ((a) 2)) :test #'equal)
((A B) 1)
CL-USER> (assoc-if #'numberp '((a . 1) (b . 2) (3 . c)))
(3 . C)
```

Може здатись, що значенням, асоційованим з ключем, може бути лише не-список. Проте це не так. Тобто, список асоціацій може складатись і не з точкових пар, а просто зі списків. В такому разі кожен елемент такого списку все одно міститиме ключ в полі CAR спискової комірки та значення в полі CDR спискової комірки, просто значенням буде інша спискова комірка:

```
CL-USER> (assoc 1 '((2 3 4 5) (1 6) (2 . 7)))
(1 6) ; (1 . (6))
```

Тема 5.4 Геш-таблиці

Геш-таблиця (також *словник* в інших мовах програмування) — це структура даних, що зберігає асоціативні пари ключ-значення. Геш-таблиці часто використовуються у випадках, коли необхідно швидко отримати значення за певним ключем. Для вирішення задачі пошуку у великих наборах даних геш-таблиці є більш ефективними, ніж асоціативні списки (див. р. 5.3), проте для невеликої кількості ключів можуть бути й менш ефективним [46]. Стандартна реалізація геш-таблиць в Common Lisp є змінюваною (*mutable*), тобто зміна вмісту таблиці не призводить до створення нового об'єкта таблиці.

Для роботи з геш-таблицями визначені три базові операції, специфічні для цієї структури даних — додавання значення за ключем, пошук значення за ключем та видалення значення за ключем — та операції отримання інформації про деталі реалізації цієї структури в Common Lisp.

5.4.1 Створення геш-таблиці

Для створення геш-таблиці використовується функція `MAKE-HASH-TABLE`:

```
(make-hash-table &key (test 'eql) (size 7) hash-function ...)
```

`test` визначає функцію, що використовується для порівняння ключів. Стандартними для цього параметра є значення `eq`, `eql` (використовується за замовчуванням), `equal`, `equalp`. Проте в SBCL також можна створити й інші функції порівняння за допомогою функції `sb-ext:define-hash-table-test`.

`size` визначає початкову максимальну кількість пар ключ-значення в геш-таблиці. Якщо поточна кількість пар ключ-значення в геш-таблиці перевищить `size`, тоді `size` буде автоматично збільшено.

hash-function визначає функцію, що виконує обчислення геша для ключа. Для стандартних значень параметра test передавати hash-function не потрібно.

Приклад:

```
CL-USER> (make-hash-table)
#<HASH-TABLE :TEST EQL :COUNT 0 {1001EE9503}>
CL-USER> (make-hash-table :test 'eq :size 10)
#<HASH-TABLE :TEST EQ :COUNT 0 {10020A4973}>
```

Для того, щоб отримати значення деяких параметрів геш-таблиці, можна використати функції:

- HASH-TABLE-TEST повертає значення test`;
- HASH-TABLE-SIZE повертає поточне значення size;
- HASH-TABLE-COUNT повертає поточну фактичну (не максимальну) кількість пар ключ-значення в геш-таблиці;
- та інші.

Крім того, для того, щоб перевірити, чи є об'єкт геш-таблицею, можна скористатись предикатом HASH-TABLE-P.

5.4.2 Додавання значень та пошук за ключем

Для пошуку значення за ключем у геш-таблиці використовується функція GETHASH:

```
(gethash key hash-table &optional default)
```

GETHASH шукає значення за ключем key в геш-таблиці hash-table, і якщо такого ключа у таблиці немає, тоді повертає значення default. Крім основного результату GETHASH в якості другого значення повертає прапорець, що вказує, чи

було знайдено шуканий ключ. Це необхідно для випадку, коли основним значенням повертається значення default і необхідно чітко визначити, чи було знайдено шуканий ключ і повернулось таке ж значення, що й default, чи ключ не було знайдено і повернулось значення default.

Для того, щоб додати елемент до геш-таблиці, використовується комбінація макроса SETF та GETHASH:

```
CL-USER> (set 'ht (make-hash-table :test 'eq))
#<HASH-TABLE :TEST EQ :COUNT 0 {100213F943}>
CL-USER> (setf (gethash :name ht) "Marco")
"Marco"
CL-USER> (setf (gethash :surname ht) "Polo")
"Polo"
CL-USER> (gethash :name ht)
"Marco"
T
CL-USER> (gethash :birth-date ht)
NIL
NIL
CL-USER> (gethash :known-as ht "trader and traveller")
"trader and traveller"
NIL
CL-USER> (hash-table-size ht) ; поточний максимальний розмір геш-таблиці
7
CL-USER> (hash-table-count ht) ; фактична кількість пар ключ-значення
2
```

При додаванні значень до геш-таблиці варто пам'ятати про те, що ключі порівнюються функцією, заданою параметром test, а значення можуть бути довільного типу.

Примітка: GETHASH — це функція, тому всі її аргументи обчислюються перед викликом самої функції. Через це в якості default значення варто передавати вже значення, яке не вимагає складних обчислень, бо воно взагалі може не знадобитись, а обчислюватиметься в будь-якому разі. Якщо ж значення за замовчуванням таки має обчислюватись складно, тоді можна або використати OR, або додати його до геш-таблиці в разі якщо відповідний ключ не знайдено і якщо це можливо за логікою програми:

```

CL-USER> (gethash :name ht (progn (format t "evaluate default!~%" ) (log 1000 2)))
evaluate default!
"Marco"
T

CL-USER> (or (gethash :name ht)
              (progn (format t "evaluate default!~%" ) (log 1000 2)))
"Marco" ; друге значення `getash' втрачається через `OR'

CL-USER> (defun custom-gethash (key hash-table)
          (multiple-value-bind (res found-p) (gethash key hash-table)
            (values
              (if found-p
                  res
                  (progn
                     (format t "evaluate default!~%" )
                     (setf (gethash key ht) (log 1000 2))))
              found-p)))
CUSTOM-GETHASH
CL-USER> (custom-gethash :some-key ht)
evaluate default!
9.965784
NIL
CL-USER> (custom-gethash :some-key ht)
9.965784
T

```

5.4.3 Обхід геш-таблиці

Для обходу пар ключ-значення геш-таблиці можна використати функцію MAPHASH:

```
(maphash function-designator hash-table)
```

function-designator — це функція, що приймає два аргументи — ключ і значення.

Приклад: обхід геш-таблиці, створеної і зміненої в попередньому підрозділі:

```

CL-USER> (maphash (lambda (key value)
                    (format t "-> key: ~a, ~s~%" key value))
            ht)
-> key: NAME, "Marco"
-> key: SURNAME, "Polo"

```

```
-> key: SOME-KEY, 9.965784  
NIL
```

5.4.4 Видалення значень та очищення геш-таблиці

Для того, щоб видалити елемент з геш-таблиці, використовується функція `REMHASH`. Вона видаляє пару ключ-значення за заданим ключем і повертає значення `T`, якщо видалення відбулось (тобто, якщо ключ було знайдено), або ж `NIL` — якщо не відбулось.

Продовження попередніх прикладів:

```
CL-USER> (remhash :some-key ht)  
T  
CL-USER> (maphash (lambda (key value)  
                    (format t "-> key: ~a, ~s~%" key value))  
                ht)  
-> key: NAME, "Marco"  
-> key: SURNAME, "Polo"  
NIL  
CL-USER> (remhash :some-key ht)  
NIL
```

Для того, щоб видалити всі пари ключ-значення з геш-таблиці, використовується функція `CLRHASH`:

```
CL-USER> (clrhash ht)  
#<HASH-TABLE :TEST EQ :COUNT 0 {100213F943}>
```

5.4.5 Завдання на СРС

1. Написати функцію `string-to-hash-table`, яка отримує на вхід рядок і повертає геш-таблицю, в якій ключами є рядкові символи з вхідного рядка, а значеннями — списки позицій цих рядкових символів:

```
CL-USER> (string-to-hash-table "Buy it, use it, break it, fix it, trash it..")  
#<HASH-TABLE ...>
```

2. Написати функцію `hash-table-to-string`, яка приймає на вхід геш-таблицю у форматі, який повертає функція `string-to-hash-table`, і виконує створення рядка по даним з неї (тобто, виконує зворотне перетворення):

```
CL-USER> (hash-table-to-string
           (string-to-hash-table "Buy it, use it, break it, fix it, trash it.."))
"Buy it, use it, break it, fix it, trash it.."
```

Тема 5.5 Структури

Структури в Common Lisp дають змогу визначати об'єкти, що складаються з кількох (одного і більше) іменованих компонентів. Загалом, вони подібні до структур в мові C та інших мовах програмування.

5.5.1 Визначення структур

Визначення структури відбувається за допомогою макроса `DEFSTRUCT`:

```
(defstruct <name-and-options> [documentation] &rest <slot-descriptions>)
<name-and-options> := symbol | (symbol <option>*) ; назва структури і,
                                                    ; можливо, опції
<option> := (:include symbol) ; "батьківська" структура
            | (:conc-name symbol) ; префікс до імені утиліт структури
            | (:type symbol) ; базовий тип структури
            | (:constructor symbol) ; назва конструктора
            | ...
<slot-descriptions> := <slot-description>*
;; визначення поля/компоненти структури
<slot-description> := symbol | (symbol [slot-initform <slot-option>*])
<slot-option> := :type symbol ; тип значення поля структури
                | :read-only t-or-nil ; прапорець "можна лише читати"
```

`DEFSTRUCT` визначає новий тип структури з визначеними полями, що в Common Lisp називаються **слотами** (slots), а також функції для роботи з цим новим типом. До цих функцій відносяться:

- конструктор, який має назву MAKE-<struct-name>, де <struct-name> — це назва структури. Назву конструктора можна також задати явним чином при визначенні структури за допомогою опції :constructor (див. HyperSpec [17]). Конструктор в якості ключових аргументів може отримати на вхід значення всіх слотів структури;
- функції для доступу до слотів структури з назвами <struct-name>-<slot-name>;
- предикат, який перевіряє, чи є вхідний об'єкт екземпляром структури визначеного типу, і який має назву <struct-name>-P;
- функцію для копіювання екземпляра структури з назвою COPY-<struct-name>.

Наприклад, якщо визначити структуру з назвою RECORD і двома полями id та value, тоді DEFSTRUCT автоматично згенерує не лише тип RECORD, але й функції MAKE-RECORD, RECORD-ID, RECORD-VALUE, RECORD-P та COPY-RECORD:

```
CL-USER> (defstruct record
           id
           value)
RECORD
CL-USER> (set 'rec (make-record :id 1 :value "New Year"))
#S(RECORD :ID 1 :VALUE "New Year")
CL-USER> (record-p rec)
T
CL-USER> (record-id rec)
1
CL-USER> (record-value rec)
"New Year"
CL-USER> (set 'rec2 (copy-record rec))
#S(RECORD :ID 1 :VALUE "New Year")
CL-USER> (eq rec rec2)
NIL
```

В <name-and-options> може бути зазначене лише символ назви структури, або назва і певні опції визначення. До таких опцій відносяться:

- (`:include <included-struct-name>`) вказує, що ця структура базується на іншій структурі, ім'я якої задається символом `<included-struct-name>`. При цьому всі поля (або лише визначені) "батьківської" структури стають частиною нововизначеної структури. Це є аналогом спадкування класів в об'єктно-орієнтованих мовах. Можливо "успадкувати" лише одну батьківську структуру;
- (`:conc-name <conc-name>`) визначає префікс, який необхідно додавати до імен функцій для доступу до слотів. За замовчуванням (якщо опція не вказана) цей префікс має значення `<struct-name>-;`;
- (`:type <type>`) визначає представлення структури у пам'яті. За замовчування це власне структура (окремий тип). Проте за допомогою `<type>` представлення структури можна задати як вектор (`vector`) або список (`list`);
- та інші (див. HyperSpec [17]).

`<slot-option>` може бути лише двох видів:

- `:type <type-symbol>` визначає тип значення слота;
- `:read-only <t-or-nil>` вказує, що слот може лише мати те значення, що надане йому при створенні екземпляру структури. Тобто, його можна лише лише "прочитати".

5.5.2 Зміна значень слотів структур

Для зміни значень слотів структур використовується SETF разом з функцією доступу до значення слота. Наприклад:

```
CL-USER> (setf (record-id rec) 2)
2
```

```
CL-USER> rec
#S(RECORD :ID 2 :VALUE "New Year")
```

Або інший приклад:

```
CL-USER> (defstruct (project (:conc-name proj-))
  (id 0 :type integer)
  (name "" :type string)
  (primary-language "" :type string)
  (developers 0 :type integer))
PROJECT
CL-USER> (defparameter *projects* nil)
*PROJECTS*
CL-USER> (push (make-project :id 2
                             :name "Translator"
                             :primary-language "Common Lisp"
                             :developers 2)
             *projects*)
(#S(PROJECT
  :ID 2
  :NAME "Translator"
  :PRIMARY-LANGUAGE "Common Lisp"
  :DEVELOPERS 2))
CL-USER> (push (make-project :id 1
                             :name "ML Toolkit"
                             :primary-language "Python"
                             :developers 3)
             *projects*)
(#S(PROJECT :ID 1 :NAME "ML Toolkit" :PRIMARY-LANGUAGE "Python" :DEVELOPERS 3)
 #S(PROJECT
  :ID 2
  :NAME "Translator"
  :PRIMARY-LANGUAGE "Common Lisp"
  :DEVELOPERS 2))
CL-USER> (setf (proj-developers (first *projects*)) 5)
5
CL-USER> projects
(#S(PROJECT :ID 1 :NAME "ML Toolkit" :PRIMARY-LANGUAGE "Python" :DEVELOPERS 5)
 #S(PROJECT
  :ID 2
  :NAME "Translator"
  :PRIMARY-LANGUAGE "Common Lisp"
  :DEVELOPERS 2))
```

5.5.3 Спадкування структур

Як і було зазначено в р. 5.5.1, для того, щоб визначити структуру, що базується на іншій структурі (включає іншу структуру), можна скористатись

опцією визначення структури (`:include ...`). При цьому всі або частина полів базової структури стають частиною нововизначеної структури, а екземпляри "розширеної" структури будуть водночас екземплярами як нової, так і базової структур — згідно зі згенерованими предикатами.

Наприклад, структура, що "спадкує" структуру `RECORD`, визначену раніше:

```
CL-USER> (defstruct (project-record (:include record) (:conc-name pr-))
           (primary-language "" :type string)
           (developers 0 :type integer))
PROJECT
CL-USER> (set 'proj (make-project-record :id 2
                                         :primary-language "Common Lisp"
                                         :developers 2))
#S(PROJECT-RECORD :ID 2 :VALUE NIL :PLANGUAGE "Common Lisp" :DEVELOPERS 2)
CL-USER> (pr-primary-language proj)
"Common Lisp"
CL-USER> (pr-id proj)
2
CL-USER> (record-id proj)
2
CL-USER> (project-record-p proj)
T
CL-USER> (record-p proj)
T
```

Тема 5.6 Файли

Для роботи з файлами в Common lisp є багато різних функцій. Умовно їх можна поділити на функції для роботи зі шляхами (`pathnames`), функції для роботи з файлами та функції для роботи з потоками вводу/виводу (`streams`), що використовуються для читання/запису з/у файл.

До функцій роботи зі шляхами відносять такі функції, як:

- (`file-namestring pathname`), яка повертає ім'я файлу з розширенням зі шляху (аргумент функції);
- (`pathname-name pathname &key (case :local)`) повертає просто ім'я файлу;

- `(pathname-type pathname &key (case :local))` повертає розширення файлу (у верхньому регістрі);
- та інші.

До функцій для роботи з файлами відносяться:

- макрос `with-open-file` (розглядається в р. 5.6.2);
- `(probe-file pathspec)` перевіряє чи файл існує і повертає його дійсний шлях, якщо так;
- `(uiop:read-file-string file &rest keys)` зчитує файл і повертає рядок з його вмістом;
- та багато інших.

До функцій для роботи з потоками вводу/виводу відносяться функції:

- `format` (див. р. 5.6.1);
- `read`, `read-char` читають символний вираз, рядковий символ з вхідного потоку (за замовчуванням зі стандартного потоку вводу, який зберігається у глобальній змінній `*standard-input-stream*`);
- `(read-line &optional (stream *standard-output-stream*) (eof-error-p t) eof-value ...)` зчитує один рядок з потоку (послідовність символів до символа нового рядка);
- `write`, `write-char` записують об'єкт, рядковий символ у вихідний потік (за замовчуванням у стандартний потік вводу, який зберігається у глобальній змінній `*standard-input-stream*`);
- `(write-line string &optional (stream *standard-output-stream*) &key (start 0) end)` записує рядок у потік;

- `princ`, `prin1` та ін. записують об'єкт у потік (формат виводу відрізняється від того що робить `write`);
- та ін.

5.6.1 Принтер

Common Lisp забезпечує представлення більшості об'єктів у формі друкованого тексту, яке називається **друкованим** або **текстовим представленням**. Такі функції, як `PRINT`, `WRITE`, `FORMAT` та інші приймають об'єкт і надсилають символи його друкованого представлення в певний потік. Набір функцій, які забезпечують друкований вивід об'єктів, відомий як (Common Lisp) **принтер** [44].

5.6.1.1 Форматований вивід

Для форматованого виводу в Common Lisp використовується функція `FORMAT`:

```
(format destination control-string &rest format-arguments)
```

`destination` — потік виводу. Цей параметр може бути одним із трьох об'єктів:

- об'єкт потоку — в такому разі вивід відбуватиметься у цей об'єкт;
- символ `T`, який вказує на те, що вивід необхідно виконати у стандартний потік виводу, який в Common Lisp зберігається у глобальній змінній `*standard-output*`;
- символ `NIL`, який вказує на те, що вивід необхідно виконати у рядок (який поверне `FORMAT`).

Якщо значення `destination` не `NIL`, тоді результатом функції `FORMAT` буде `NIL`.

`control-string` — форматований рядок (або ж функція, наприклад, сформована макросом `FORMATTER`). Цей рядок складається зі звичайного тексту і вбудованих директив [45]. Директиви складаються з символу `~` (тильда), опційних параметрів та модифікаторів, та одного рядкового символу, що вказує на те, яка саме це директива.

`format-arguments` — об'єкти, які будуть використані для заповнення форматowanego рядка.

Деякі директиви для виводу різних об'єктів:

- `c/C` — рядковий символ
- `%` — символ нового рядка
- `~` — символ "тильда"
- `d/D` — число цілого типу
- `f/F` — число з рухомою комою фіксованого типу
- `s/S` — текстове представлення об'єкта у формі, яку повертає функція `PRIN1`
- `a/A` — текстове представлення об'єкта у формі, яку повертає функція `PRINC`
- та інші.

Більш детально про роботу функції `FORMAT` можна прочитати в документації [45].

Приклади роботи функції `FORMAT`:

```
CL-USER> (format t "~d" 1305)
1305
NIL
CL-USER> (format t "~7d" 1305)
      1305
NIL
CL-USER> (format nil "~3,2f" 14.235)
"14.23"
CL-USER> (format nil "~4,2f" 14.235)
"14.23"
CL-USER> (format nil "~d ~A bottles standing on the ~A" 10 "green" 'wall)
```

```
"10 green bottles standing on the WALL"
```

5.6.2 Макрос WITH-OPEN-FILE

Універсальною формою для читання і запису файлів є макрос WITH-OPEN-FILE:

```
(with-open-file (stream filespec  
                 &rest options  
                 &key (direction :input) if-exists if-does-not-exist ...)  
  <body>)
```

WITH-OPEN-FILE створює потік вводу або виводу (в залежності від значення ключового параметра `direction`) для файлу, заданого шляхом `filespec`, і зв'язує його з символом `stream`.

Параметр `direction` задає "напрямок" в якому відбувається робота з файлом:

- `:input` (за замовчуванням) вказує на те, що файл необхідно прочитати;
- `:output` — що файл необхідно записати;
- інші можливі значення можна знайти на HyperSpec [17].

`if-exists` задає поведінку щодо файлів, які вже існують, при створенні потоку виводу:

- `:new-version` — створити новий файл з більшою "версією";
- `:error` — повідомити про помилку;
- `:supersede` — переписати наявний файл;
- `:append` — дописати в кінець файлу;
- та ін.

`if-does-not-exist` задає поведінку щодо файлів, які ще не існують:

- `:error` — повідомити про помилку;
- `:create` — створити файл;
- `nil` — повернути `nil`, що сигналізує про помилку.

Після відкриття потоку, в тілі `WITH-OPEN-FILE` (`<body>`) виконуються форми, що працюють зі створеним потоком.

Крім створення потоку, `WITH-OPEN-FILE` також виконує закриття об'єкту потоку після того, як потік виконання виходить з тіла макроса (як при коректному завершенні, так і в разі помилки).

Приклади запису і читання файлу:

```
CL-USER> (with-open-file (stream "~/tmp/io-test.txt" :direction :output)
           (format stream "LISP stands for LISt Processing.))
NIL
CL-USER> (with-open-file (stream "~/tmp/io-test.txt")
           (read-line stream))
"LISP stands for LISt Processing."
T
```

Приклад читання файлу по рядкам:

```
CL-USER> (with-open-file (stream "~/tmp/io-test.txt"
                           :direction :output
                           :if-exists :supersede)
           (format stream "A: Hi!~%")
           (format stream "B: Hey!~%")
           (format stream "A: What's up?~%"))
NIL
CL-USER> (with-open-file (s "~/tmp/io-test.txt")
           (do ((line (read-line s) (read-line s nil 'eof)))
               ((eq line 'eof) (format t "Reached end of file.") t)
               (format t "----> ~A~%" line)))
----> A: Hi!
----> B: Hey!
----> A: What's up?
Reached end of file.
T
```

Тема 5.7 Макроси

Макроси в Common Lisp — це потужний інструмент, що дає змогу програмісту розширювати мову, створюючи нові синтаксичні конструкції. На відміну від функцій, які обчислюють значення, макроси виконуються на етапі компіляції і перетворюють один код (*макровиклик*) на інший (*макророзширення*).

5.7.1 Визначення макросів

В Common Lisp макроси визначаються за допомогою спеціальної форми DEFMACRO. Її синтаксис подібний до синтаксису DEFUN, але з однією ключовою відмінністю: значенням, яке повертає тіло макросу, є новий код, який потім буде розширений (expanded) на етапі компіляції, скомпільований і виконається на етапі виконання програми.

Синтаксис DEFMACRO такий:

```
(defmacro macro-name (<arg>*)  
  <body-form>*)
```

macro-name — це символ, що є назвою макросу;

<arg>* — це список аргументів макросу, які, на відміну від аргументів функцій, не обчислюються, а обробляються у тому вигляді, в якому вони визначаються програмістом у макровиклику — тобто, у вигляді форм;

<body-form>* — набір форм, що формують тіло макросу.

5.7.2 Прості макроси

Прикладом простого макросу є стандартний макрос WHEN, який є аналогом IF без гілки else, який в гілці true може містити довільну кількість форм, в той час як IF може містити лише одну форму.

Форма з IF в якій гілка true містить кілька форм реалізується за допомогою PROGN:

```
(if (> a b)
    (progn
      (format t "a is greater than b")
      (setq a b)))
```

Аналогічний код з використанням макросу WHEN є коротшим, проте працює повністю так само, як і код з IF:

```
(when (> a b)
  (format t "a is greater than b")
  (setq a b))
```

В стандартній бібліотеці Common Lisp є й багато інших простих макросів [17].

5.7.3 Створення складних макросів

Найбільша проблема при написанні складних макросів — це конфлікт імен змінних (назв символів). Це відбувається, коли символи, визначені у згенерованому макросом коді, випадково збігається з іменами змінних, визначених у зовнішньому контексті.

Щоб уникнути таких ситуацій, за допомогою функції **gensym** створюються символи з унікальними іменами — такими, що відрізняються від усіх імен у зовнішньому контексті. Ця функція генерує унікальний символ щоразу, коли її викликають.

Наприклад, якщо створити макрос, що визначає тимчасовий файловий кеш, то код визначення такого макросу може мати наступний вигляд:

```
(defmacro with-cleanup (cleanup-form &body body)
```

```
`(let ((cache (make-temp-file)))
  (unwind-protect
    (progn ,@body)
    (delete-file cache))))
```

Якщо програміст, що використовує такий макрос, визначить змінну `cache` в контексті макровиклику `with-cleanup` (напр. `(let ((temp 10)) (with-cleanup ...))`), змінна `cache` буде зв'язана в коді макророзширення і її використання десь в тілі макросу призведе до використання тимчасового файлу, а не значення `10`.

Реалізація того самого макросу з `gensym` має такий вигляд:

```
(defmacro with-cleanup (cleanup-form &body body)
  (let ((cache-var (gensym "TEMP")))
    `(let ((,cache-var (make-temp-file)))
      (unwind-protect
        (progn ,@body)
        (delete-file ,cache-var)))))
```

`cache-var` — це змінна (символ), значенням якого є унікальний символ, створений `gensym`, який ніколи не конфліктуватиме зі змінними з зовнішнього контексту.

РОЗДІЛ 6

ОСОБЛИВОСТІ ФУНКЦІОНАЛЬНОГО ПРОГРАМУВАННЯ

Тема 6.1 Комбінування функціонального та об'єктного програмування

Як і було зазначено раніше, Common Lisp є однією з мов, яка крім парадигми функціонального програмування, реалізує багато інших парадигм. До таких парадигм відноситься і парадигма об'єктно-орієнтованого програмування. Завдяки цьому розробники мовою Common Lisp мають можливості обирати найкращий підхід для виконання конкретної задачі, замість того, щоб бути обмеженими однією парадигмою.

Common Lisp має вбудовану систему об'єктів — Common Lisp Object System (CLOS) [54-55]. Вона відрізняється від традиційних ООП-систем (таких як в С-подібних мовах — Java, C++, C#, тощо) такими особливостями, як:

- множинне спадкування — можливість класів спадкувати кілька батьківських класів;
- відділення методів від класів — методи завжди визначаються окремо від класів, з якими вони працюють, тобто класи *не містять* методів;
- метаоб'єктний прокол (Metaobject Protocol, MOP), який дає змогу змінювати поведінку самої об'єктної системи [55];
- тощо.

Комбінування функціонального та об'єктного підходів в Common Lisp може полягати у використанні рекурсії для створення ієрархічних структур об'єктів; визначення функцій вищого порядку при роботі з різними об'єктами; реалізацію макросів для створення типових класів і методів для роботи з ними, тощо.

Тема 6.2 Лінійні обчислення

Лінійні обчислення (lazy evaluation) — це стратегія обчислень, де вираз обчислюється лише тоді, коли його значення справді потрібне [57]. Це відрізняється від **жадібних обчислень** (eager evaluation), що є стратегією за замовчуванням у більшості мов, включаючи Common Lisp.

Перевагами лінійних обчислень є те, що вони дають змогу уникнути виконання обчислень «наперед» — до виникнення реальної потреби у даних, отриманих внаслідок таких обчислень. Крім того, це дає змогу створювати «нескінченні» послідовності даних, оскільки такі послідовності генеруватимуться не одразу повністю, а поступово — в разі виникнення потреби.

В Common Lisp не має вбудованої підтримки лінійних обчислень. Однак, завдяки своїй гнучкості та потужності макросів, цю функціональність можна реалізувати. Основний підхід полягає в створенні макросу, який обгортає вираз і відкладає його виконання.

Для реалізації лінійного обчислення потрібна структура, яка зберігатиме обчислюваний вираз і його результат. Наприклад:

```
(defstruct lazy-node
  (evaluated nil) ;; булеве значення, що вказує на статус обчислення значення
  (value nil)    ;; результат обчислення
  (expr nil))    ;; вираз, який необхідно обчислити
```

Маючи таку структуру, можна визначити lazy, який створюватиме екземпляр такої структури:

```
(defmacro lazy (expr)
  "Creates a 'node' for lazy evaluation."
  `(make-lazy-node :expr (lambda () ,expr)))
```

Цей макрос не обчислює `expr`, а лише створює `lambda`, яка містить цей вираз.

Щоб обчислити «лінивий» вираз, необхідна додаткова функція `force`. Вона перевіряє, чи було вже обчислено значення. Якщо так — повертає збережений результат; якщо ні — обчислює, зберігає результат і повертає його:

```
(defun force (node)
  "Forces the evaluation of a lazy expression."
  (unless (lazy-node-evaluated node)
    (setf (lazy-node-result thunk)
          (funcall (lazy-node-expr node)
                   (lazy-node-evaluated node) t))
    (lazy-node-result node))
```

Ця реалізація дасть змогу працювати з концепцією нескінченних послідовностей, що було б неможливо з жадібним підходом.

```
(defun lazy-integers-from (n)
  "Generates a lazy stream of integers starting from n."
  (cons n (lazy (lazy-integers-from (+ n 1)))))

;; Створення "потоків" натуральних чисел
(defvar *int-stream* (lazy-integers-from 1))

;; Отримання перших кількох елементів
CL-USER> (car *int-stream*)
1
CL-USER> (setf *int-stream* (force (cdr *int-stream*)))
(2 . #<LAZY-NODE>)
CL-USER> (car *int-stream*)
2
CL-USER> (setf *int-stream* (force (cdr *int-stream*)))
(3 . #<LAZY-NODE>)
```

У цьому прикладі `lazy-integers-from` повертає точкову пару. Поле `CAR` містить поточне число, а `CDR` — лінивий вираз, що представляє решту послідовності. Виклик `force` на хвості (`CDR`), призводить до обчислення наступної частини послідовності.

Тема 6.3 Комбінатори

Комбінатори — це функції, які приймають інші функції як аргументи та повертають нову функцію або значення. Тобто, комбінатори є функціями вищого порядку. Вони дають змогу створювати складніші операції з простих, без необхідності дублювати код.

Найпростішими прикладами комбінаторів є такі функції, як `compose` (розглядалась в р. 2.4.5), `curry` (створює функцію, що виконує виклики функції-аргументу з частково визначеними аргументами), та інші.

Ще одним прикладом комбінаторів є *парсер-комбінатори* (`parser combinators`) — спосіб визначення парсерів у функціональному стилі. Це функції вищого порядку, які приймають прості парсери як аргументи і повертають нові, більш складні парсери. Цей підхід дає змогу будувати складні парсери з простих, що робить процес розробки гнучким і модульним.

Парсер-комбінатори мають вигляд «будівельних блоків», які можна комбінувати. Типовими є такі комбінатори:

- `sequence` — послідовно застосовує парсери-аргументи;
- `or` — приймає два або більше парсерів і повертає новий парсер, який спочатку намагається застосувати перший парсер, якщо не вдалося — тоді другий, якщо і другий не вдалося — третій і т. д.
- `list` — приймає на вхід парсер, і повертає новий парсер, що застосовує парсер-аргумент до вхідних аргументів доти, доки можливо. Успішним вважається розбір навіть якщо парсер-аргумент жодного разу не повернув результату;
- `list+` — той самий `sequence`, але парсер аргумент має повернути результати принаймні раз;

- `optional` — отримує на вхід парсер і повертає новий парсер, який успішно виконується, навіть якщо парсер-аргумент не поверну результат.

Маючи такі комбінатори, можна визначити парсери для певних граматик мов, і виконувати розбір текстів, записаних цими мовами.

Тема 6.4 Простий вебсервер

Common Lisp як функційну мову програмування можна використати і для створення таких типових речей, як вебсервер. SBCL не має вбудованої реалізації вебсервера, тому для того, щоб створити свій вебсервер, варто використати одну з наявних бібліотек, наприклад Hunchentoot [56].

Зробити просту реалізацію вебсервера з Hunchentoot можна за ось так:

```
;; Завантажуємо бібліотеку Hunchentoot за допомогою Quicklisp
(ql:quickload "hunchentoot")

(defun start-server (&key (port 8080))
  "Start Hunchentoot webserver on a specific `port'"
  (hunchentoot:start (make-instance 'hunchentoot:easy-acceptor :port port)))

;; Визначаємо обробник запитів для кореневого шляху / URL ("/")
(hunchentoot:define-easy-handler (access-root :uri "/") ()
  "Access root - Success!")

;; Створюємо обробник для іншого шляху з параметром
(hunchentoot:define-easy-handler (greet-person :uri "/greet") (name)
  (setf (hunchentoot:content-type*) "text/plain; charset=utf-8")
  (format nil "Hello, ~a!" (or name "stranger")))

;; Запускаємо наш сервер на порту 8080
(start-server :port 8080)

(format t "Server started at http://localhost:8080")
```

`hunchentoot:easy-acceptor` — це простий варіант сервера, визначений в бібліотеці Hunchentoot.

РЕКОМЕНДОВАНА ЛІТЕРАТУРА

Базова література

1. Practical Common LISP. URL: <https://gigamonkeys.com/book/> (дата звернення 14.07.2025).
2. Paul Graham. ANSI Common Lisp. Prentice Hall, 1995, 432 pages. ISBN 0-13-370875-6. URL: <https://paulgraham.com/acl.html> (дата звернення 16.08.2025).
3. Common Lisp HyperSpec. URL: <https://www.lispworks.com/documentation/HyperSpec/Front/> (дата звернення 14.07.2025).

Додаткова література

1. В. М. Заяць, М. М. Заяць. Логічне і функціональне програмування. Системний підхід. Підручник. — 2-ге видання, випр. та доповн. — Рівне : НУБГП, 2018. — 422 с. URL: <https://www.lib.rv.ua/sections/items/11342#gsc.tab=0> (дата звернення: 16.11.2025).
2. Harold Abelson, Gerald Jay Sussman, and Julie Sussman. Structure and Interpretation of Computer Programs. 2nd edition. MIT Press, 1996, 855 pages. URL: <https://web.mit.edu/6.001/6.037/sicp.pdf> (дата звернення: 16.11.2025).

Інформаційні ресурси

1. The Common Lisp Cookbook. URL: <http://lispcookbook.github.io/cl-cookbook/> (дата звернення: 30.08.2025).

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. The Data Engineers Guide to Declarative vs Imperative for Data. URL: <https://www.dataops.live/the-data-engineers-guide-to-declarative-vs-imperative-for-data> (дата звернення 16.08.2025).
2. Wikipedia: Парадигма програмування. URL: https://uk.wikipedia.org/wiki/Парадигма_програмування (дата звернення 16.08.2025).
3. Top 10 Dying Programming Languages That Vanished Over Time. URL: <https://analyticsindiamag.com/ai-origins-evolution/10-dead-programming-languages/> (дата звернення 16.08.2025).
4. Wikipedia: Парадигма (значення). URL: [https://uk.wikipedia.org/wiki/Парадигма_\(значення\)](https://uk.wikipedia.org/wiki/Парадигма_(значення)) (дата звернення 16.08.2025).
5. Wikipedia: Comparison of multi-paradigm programming languages. URL: https://en.wikipedia.org/wiki/Comparison_of_multi-paradigm_programming_languages (дата звернення 16.08.2025).
6. Wikipedia: Functional programming. URL: https://en.wikipedia.org/wiki/Functional_programming (дата звернення 16.08.2025).
7. Immutability in Functional Programming. URL: <https://adabeat.com/fp/immutability-in-functional-programming/> (дата звернення 16.08.2025).
8. Wikipedia: Lisp (programming language). URL: [https://en.wikipedia.org/wiki/Lisp_\(programming_language\)](https://en.wikipedia.org/wiki/Lisp_(programming_language)) (дата звернення 16.08.2025).

9. GitHub: dundalek / Awesome Lisp Languages. URL:
<https://github.com/dundalek/awesome-lisp-languages> (дата звернення 16.08.2025).
10. Paul Graham. ANSI Common Lisp. Prentice Hall, 1995, 432 pages. ISBN 0-13-370875-6. URL: <https://paulgraham.com/acl.html> (дата звернення 16.08.2025).
11. Steel Bank Common Lisp. URL: <https://www.sbcl.org/> (дата звернення 16.08.2025).
12. SLIME: The Superior Lisp Interaction Mode for Emacs. URL:
<https://slime.common-lisp.dev/> (дата звернення 16.08.2025).
13. Quora: Why has the programming language LISP lost popularity? URL:
<https://www.quora.com/Why-has-the-programming-language-LISP-lost-popularity#eIFcu> (дата звернення 16.08.2025).
14. StackOverflow 2024 Developer Survey. URL:
<https://survey.stackoverflow.co/2024> (дата звернення 16.08.2025).
15. GNU Emacs. URL: <https://www.gnu.org/software/emacs/> (дата звернення 16.08.2025).
16. GitHub: Enerccio / SLT. URL: <https://github.com/Enerccio/SLT> (дата звернення 16.08.2025).
17. Common Lisp HyperSpec. URL:
<https://www.lispworks.com/documentation/HyperSpec/Front/> (дата звернення: 16.08.2025).
18. Portacle. URL: <https://portacle.github.io/> (дата звернення: 16.08.2025).
19. Emacs Refcard. URL:
<https://www.gnu.org/software/emacs/refcards/pdf/refcard.pdf> (дата звернення: 16.08.2025).

20. Definition of symbolic programming. URL:
https://www.allbusiness.com/barrons_dictionary/dictionary-symbolic-programming-4950308-1.html (дата звернення: 16.08.2025).
21. Style Guide. URL: <https://lisp-lang.org/style-guide/> (дата звернення: 16.08.2025).
22. Google Common Lisp Style Guide. URL:
<https://google.github.io/styleguide/lispguide.xml> (дата звернення: 16.08.2025).
23. Practical Common Lisp: Syntax and Semantics. URL:
<https://gigamonkeys.com/book/syntax-and-semantics> (дата звернення: 16.08.2025).
24. Practical Common Lisp: Numbers, Characters, and Strings. URL:
<https://gigamonkeys.com/book/numbers-characters-and-strings> (дата звернення: 16.08.2025).
25. The Common Lisp Cookbook — Numbers. URL:
<http://lispcookbook.github.io/cl-cookbook/numbers.html> (дата звернення: 16.08.2025).
26. ParEdit. URL: <https://www.emacswiki.org/emacs/ParEdit> (дата звернення: 16.08.2025).
27. Common Lisp the Language, 2nd Edition: Special Forms. URL:
<https://www.cs.cmu.edu/Groups/AI/html/cltl/clm/node59.html#SECTION00913000000000000000> (дата звернення: 16.08.2025).
28. Practical Common Lisp: The Special Operators. URL:
<https://gigamonkeys.com/book/the-special-operators> (дата звернення: 16.08.2025).
29. Common Lisp HyperSpec: Use of Triple Semicolon. URL:
https://www.lispworks.com/documentation/HyperSpec/Body/02_ddbc.htm (дата звернення: 16.08.2025).

30. В. М. Заяць, М. М. Заяць. Логічне і функціональне програмування. Системний підхід. Підручник. — 2-ге видання, випр. та доповн. — Рівне : НУВГП, 2018. — 422 с. URL: <https://www.lib.rv.ua/sections/items/11342#gsc.tab=0> (дата звернення: 16.11.2025).
31. The Common Lisp Cookbook — Equality. URL: <https://lispcookbook.github.io/cl-cookbook/equality.html> (дата звернення: 16.08.2025).
32. Practical Common Lisp: They Called It LISP for a Reason: List Processing. URL: <https://gigamonkeys.com/book/they-called-it-lisp-for-a-reason-list-processing> (дата звернення: 16.08.2025).
33. Wikipedia: Lambda calculus. URL: https://en.wikipedia.org/wiki/Lambda_calculus (дата звернення: 16.08.2025).
34. Common Lisp HyperSpec: Symbol LAMBDA. URL: https://www.lispworks.com/documentation/HyperSpec/Body/s_lambda.htm (дата звернення: 16.08.2025).
35. Wikipedia: Recursion. URL: <https://en.wikipedia.org/wiki/Recursion> (дата звернення: 16.08.2025).
36. Practical Common Lisp: Functions. URL: <https://gigamonkeys.com/book/functions> (дата звернення: 16.08.2025).
37. Common Lisp HyperSpec: Macro LOOP. URL: https://www.lispworks.com/documentation/HyperSpec/Body/m_loop.htm#loop (дата звернення: 16.08.2025).
38. Practical Common Lisp: LOOP for Black Belts. URL: <https://gigamonkeys.com/book/loop-for-black-belts> (дата звернення: 16.08.2025).

39. Common Lisp HyperSpec: Overview of Places and Generalized Reference. URL:
https://www.lispworks.com/documentation/HyperSpec/Body/05_aa.htm (дата звернення: 16.08.2025).
40. GeeksForGeeks: Sequences in LISP. URL:
<https://www.geeksforgeeks.org/sequences-in-lisp/> (дата звернення: 16.08.2025).
41. Lookup tables in Common Lisp - alists and plists. URL:
<http://dnaeon.github.io/common-lisp-lookup-tables-alists-and-plists/> (дата звернення: 16.08.2025).
42. ASDF: Function split-string. URL:
https://asdf.common-lisp.dev/uiop.html#index-split_002dstring (дата звернення: 16.08.2025).
43. Common Lisp HyperSpec: The Strings Dictionary. URL:
https://www.lispworks.com/documentation/HyperSpec/Body/c_string.htm (дата звернення: 16.08.2025).
44. Common Lisp HyperSpec: Printer. URL:
https://www.lispworks.com/documentation/HyperSpec/Body/22_.htm (дата звернення: 16.08.2025).
45. Common Lisp HyperSpec: Formatted Output. URL:
https://www.lispworks.com/documentation/HyperSpec/Body/22_c.htm (дата звернення: 16.08.2025).
46. The Common Lisp Cookbook — Data structures: Hash Table. URL:
<https://lispcookbook.github.io/cl-cookbook/data-structures.html#hash-table> (дата звернення: 16.08.2025).
47. Quora: What is the meaning of core programming in C, C++, Java, etc.?. URL:
<https://www.quora.com/What-is-the-meaning-of-core-programming-in-C-C++-Java-etc> (дата звернення: 16.08.2025).

48. Practical Common Lisp: Macros: Standard Control Constructs. URL: <https://gigamonkeys.com/book/macros-standard-control-constructs> (дата звернення: 16.08.2025).
49. The Common Lisp Cookbook — Macros. URL: <https://lispcookbook.github.io/cl-cookbook/macros.html> (дата звернення: 16.08.2025).
50. Practical Common Lisp: Lather, Rinse, Repeat: A Tour of the REPL. URL: <https://gigamonkeys.com/book/lather-rinse-repeat-a-tour-of-the-repl> (дата звернення: 17.08.2025).
51. The Common Lisp Cookbook — Packages. URL: <https://lispcookbook.github.io/cl-cookbook/packages.html> (дата звернення: 23.08.2025).
52. SLIME Features. URL: <https://www.cliki.net/slime%20features> (дата звернення: 23.08.2025).
53. CUA mode for Emacs. URL: https://www.gnu.org/software/emacs/manual/html_node/emacs/CUA-Bindings.html (дата звернення: 23.08.2025).
54. Practical Common Lisp: Object Reorientation: Generic Functions. URL: <https://gigamonkeys.com/book/object-reorientation-generic-functions> (дата звернення: 02.09.2025).
55. The Common Lisp Cookbook — Fundamentals of CLOS. URL: <https://lispcookbook.github.io/cl-cookbook/clos.html> (дата звернення: 02.09.2025).
56. GitHub: Hunchentoot. URL: <https://edicl.github.io/hunchentoot/> (дата звернення: 04.09.2025).
57. Wikipedia: Lazy Evaluation. URL: https://en.wikipedia.org/wiki/Lazy_evaluation (дата звернення: 09.11.2025).

ДОДАТОК 1

ФОРМАТУВАННЯ КОДУ МОВОЮ COMMON LISP

1. Іменування символів

1.1 Верхній та нижній регістри

SBCL дає змогу писати тексти програм в довільному регістрі — символи в будь-якому разі будуть приведені у верхній регістр. Крім спеціальної форм запису символів з використанням символів `|` (bar) (напр. `|a b|`). Попри це, використовувати різний регістр при написанні програм не варто, оскільки це псує загальний вигляд тексту програм. Зазвичай (і це є рекомендація) код мовою Common Lisp пишеться у нижньому регістрі. Верхній регістр часом застосовується для позначення аббревіатур, але не завжди.

Варто писати: `car`, `reduce`, `append`

НЕ варто писати: `Car`, `REDUCE`, `aPpEnd`

1.2 Багатослівні символи

Символи, що складаються з двох і більше слів, записуються з дефісами між словами. Наприклад: `bring-me-home`. Такий стиль запису назв ідентифікаторів, який називається `kebab case` (або також `spinal case`, `lisp-case`). На противагу такому стилю запису, в інших мовах зазвичай застосовуються `snake case` (`bring_me_home`) та `camel case` (`bringMeHome`).

Варто писати: `insertion-sort`, `binary-search-1`

НЕ варто писати: `insertion_sort`, `binartSearch1`

1.3 Небажані імена

При написанні програм мовою Common Lisp (як, загалом, і іншими мовами програмування) слід уникати таких імен змінних, як `l` (маленька «L»), `o` (велика літера «O»), оскільки їх досить просто переплутати з одиницею `1` і нулем `0`.

2. Пробіли у списках

Варто дотримуватись таких правил написання пробілів (перед, після та між формами) та переносів рядка:

- після відкриваючої дужки списку перед першим елементом (символом, підсписком, тощо) та після останнього елементу списку перед закриваючою дужкою пробіл не ставиться;
- між елементами списку ставиться по одному пробілу або переносу рядка;
- між послідовними закриваючими дужками, якщо між ними немає інших форм, пробіли та/або переноси рядків не додаються.

Варто писати:

```
(list 1 (1+ arg) (- k 5))  
(list 1  
  (1+ arg)  
  (- k 5))
```

НЕ варто писати:

```
( list 1      (1+ arg) (- k 5)  )  
(list 1  
  (1+ arg)  
  (- k 5  
)  
)
```

3. Відступи вкладених форм

Вкладені форми (елементи списку) мають мати певний відступ відносно батьківської (зовнішньої) форми — її початку, відкриваючої дужки. Таким чином

форма матиме вигляд ієрархічної структури, де вкладені форми мають більший відступ, ніж «зовнішні» форми. Зазвичай цей відступ рівний двом пробілам — для «тіл» таких форм, як `defun`, `progn`, тощо. Для частин інших форм — одному (напр. Для тіл гілок `case`, `cond`, тощо). У випадку передачі параметрів у функцію, параметри записуються з тим самим відступом, який має перший параметр функції, який, у свою чергу, розміщується через пробіл після функції, або одразу під нею (з одиничним відступом відносно відкриваючої дужки). Для деяких стандартних форм відступи можуть бути 4 пробіли для першого аргументу і 2 — для наступних (наприклад, `multiple-value-bind`), або деяку визначену кількість пробілів (4 для `if`).

Варто писати:

```
(cons 1 nil)
(foo 1 2 3)
(foo 1
    2
    3)
(foo
 1
 2
 3)
(defun foo (arg j)
  "Docstring..."
  (if (eql arg j)
      (1+ arg)
      (case j
        (1 "ok")
        ((2 3)
         "better")
        (otherwise "bad")))))
```

НЕ варто писати:

```
(cons 1
      nil)
(foo 1
```

```

2 3)
(foo 1
  2
  3)
(foo
  1
    2
    3)
(defun foo (arg k)
  "Some comment about what this function does"
  (if (eql arg k)
      (1+ arg)
      (case k
        (1 "ok")
        ((2 3) "better")
        (otherwise "bad")))))

```

Автоматично виставити «правильні» відступи в Emacs можна за допомогою комбінації C-M-q (Ctrl+Alt+q), застосованій на відкриваючій дужці форми — встановлення відступів відбувається для усієї форми, до якої ця комбінація застосована, в т. ч. для вкладених форм.

4. Розміщення гілок конструкцій керування

Гілки керуючих форм та вкладені форми слід переносити на наступний рядок, якщо всі вони не поміщаються в один. Немає стандарту “максимальної довжини рядка”, й взагалі його довжина не є обмеженою. Але краще писати код так, щоб довжина його рядків не перевищувала 90-100 символів і не було переносів на наступний рядок. 90-100 символів — це приблизна кількість символів, яка поміститься у половині сучасного широкоформатного монітора, або ж у не широкоформатному моніторі. Хоча, звісно, все залежить від розширення екрану, розміру шрифту, тощо. Якщо ж форми доведеться переносити, то переносити потрібно всі, окрім однієї-двох перших. Не можна частину форм переносити, а частину ні, якщо це не є усталеним способом запису конкретної форми.

Окрім того слід зазначити, що не потрібно робити кілька перенесень.
Достатньо одного.

Варто писати:

```
(if (eq1 arg k)
    (1+ arg)
    (case k
      (1 "ok")))

(if cond
    t-branch
    nil-branch)

(if (eq1 x 1) 0 (error "X != 1"))
```

НЕ варто писати:

```
(if (eq1 arg k)
    (1+ arg)
    (case k
      (1 "ok")))

(if cond t-branch
    nil-branch)

(if cond
    t-branch nil-branch)
```

5. Визначення функцій

Між окремими визначеннями функцій має бути відступ — один пустий рядок. Додатково можна додати коментарі — якщо є потреба додатково розділити визначення (наприклад, якщо групи функцій мають логічно різне призначення).

Варто писати:

```
(defun foo ()
  ...)

(defun bar ()
  ...)
```

НЕ варто писати:

```
(defun foo ()  
  ...)  
(defun bar ()  
  ...)
```

При визначенні функції, в разі потреби пояснити що ця функція робить, можна написати рядок документації (docstring). Для простих функцій такої потреби, зазвичай не виникає, проте для складніших може з'явитись. Приклад:

```
(defun my-sort (lst)  
  "Sort input list"  
  ...)
```

В разі потреби визначити додаткову локальну функцію або замикання всередині іншої функції, не варто використовувати для цього `defun`, оскільки він створить функцію, і асоціює її з певним символом, який буде доступний в загальному контексті (на верхньому рівні). У більшості випадків така поведінка не потрібна. Тому у зазначених випадках варто визначати локальні функції або лямбда-вирази.

Варто писати:

```
(defun print-message (arg)  
  (flet ((%print (text)  
          (format t "> ~A~%" text))))  
  (%print arg)))
```

НЕ варто робити:

```
(defun print-message (arg)  
  (defun print-inner (text)  
    (format t "> ~A~%" text))  
  (print-inner arg))
```

ДОДАТОК 2

РОБОТА З EMACS/PORTACLE І SLIME

Буфером в Emacs називається текст, який редагується. Можна назвати це "вкладками" в Emacs. Наприклад, для кожного файлу створюється окремий буфер, для REPL — теж свій буфер і т. д.

Мінібуфер — буфер внизу екрану, що використовується для запуску команд Emacs та/або передачі аргументи у команди.

Вікном називається частина всього вікна Emacs, в якому розміщується буфер.

Деякі комбінації клавіш для роботи з вікнами, буферами та файлами (C — Ctrl, M — Alt, S — Shift):

- C-x b — обрати буфер в мінібуфері
- C-x C-b — відкрити список всіх активних буферів
- C-x o — переключитись в наступне вікно
- C-x 1 — сховати всі вінка, крім поточного
- C-x 2 — розділити поточне вікно на два горизонтальних вікна
- C-x 3 — розділити поточне вікно на два вертикальних вікна
- C-x 0 — видалити поточне вікно
- C-x C-f — відкрити файл (шлях записується через мінібуфер)
- C-x C-s — зберегти поточний буфер (якщо це файл — збережеться файл)
- C-x k — видалити буфер (буфер треба вибрати у мінібуфері)

cua-mode

M-x cua-mode RET — активує cua-mode в поточному буфері.

cua-mode додає такі комбінації клавіш:

- C-c — копіювати виділений текст
- C-v — вставити текст
- C-x — вирізати текст
- C-z — відмінити попередню дію

paredit-mode

За замовчуванням в Portacle цей мод активовано. Для явної активації або деактивації використовується команда:

M-x paredit-mode RET

Цей мод контролює створення парних дужок. І якщо не видаляти одну з дужок комбінацією клавіш для вирізання виділеного коду, тоді видалити одну з пари дужок не вдасться.

Робота з текстом в буфері

- C-Space — почати виділення з поточного місця. Після цього клавішами "вгору", "вниз", "вліво" та "вправо" можна переводити курсор. При цьому весь текст від початкової точки до поточної буде виділено. Виділивши потрібний фрагмент тексту, необхідно натиснути комбінацію клавіш для копіювання/вирізання/вставки тощо, щоб щось зробити з виділеним фрагментом. Зупинити режим виділення тексту можна натисканням C-Space двічі.
- C-M-Space — виділити весь текст від курсора до кінця наступного символьного виразу.
- C-M-b — перевести курсор до початку попереднього (або зовнішнього, якщо попереднього немає) символьного виразу.

- C-M-f — аналогічно C-M-b, але до "наступного" символного виразу.
- M-r — вибрати символний вираз починаючи від курсора і до завершення поточного символного виразу і "винести" за межі зовнішнього символного виразу, видаливши зовнішній.
- C-<left> винести останній елемент поточного (в контексті якого знаходиться курсор) списку за межі списку.
- C-<right> внести наступний елемент, що знаходиться за межами поточного списку у список.
- C-s — перейти в мінібуфер для пошуку текстового збігу. Якщо натискати C-s після знаходження — перейти до наступного знайденого збігу.
- C-r — пошук у зворотному напрямку.
- C-M-q (якщо використати на відкриваючій дужці списку) — автоматично ідентує вміст списку/форми.

REPL

Для виконання команди/форми в REPL необхідно написати форму й натиснути RET (Enter). В Portacle, якщо натиснути RET в "тілі" якоїсь форми, відбудеться перенесення курсора (і всього наступного тексту) на новий рядок. Щоб виконати форму в Portacle, треба вивести курсор за межі форми.

Деякі інші команди для роботи з REPL:

- C-j — додає символ перенесення на новий рядок
- C-c C-c — припинити поточний процес обчислення
- C-c M-o — очистити буфер REPL-a
- C-c C-k — скопіювати весь поточний буфер
- C-c C-c — скопіювати всю високорівневу форму, в якій або перед якою знаходиться курсор

- C-u C-c C-k — скомпілювати весь поточний буфер з додатковою інформацією для віджагодження (debugging)
- C-u C-c C-c — скомпілювати всю високорівневу форму, в якій або перед якою знаходиться курсор, з додатковою інформацією для відлагодження

Slime

Slime — це один з major модів для інтерактивної розробки мовою Common Lisp. В Portacle він за замовчуванням увімкнений. Для явного запуску slime використовується команда:

```
M-x slime RET
```

Деякі команди та комбінацій клавіш:

- M-. (slime-edit-definition) — перейти на визначення функції/змінної (додає позицію, з якої виконано перехід у стек визначень).
- M-, (slime-pop-find-definition-stack) — повернутись з визначення (в позицію в голові стеку визначень).
- C-c C-w C-c (slime-who-calls) визначає місця виклику функції, на імені якої в поточний момент знаходиться курсор.
- C-c C-w C-r (slime-who-references) визначає місця посилання / використання змінної
- C-c C-w C-b (slime-who-binds) визначає місця зв'язування значення змінної
- C-c C-w m (slime-who-macroexpands) визначає місця, де відбувається макророзширення макроса